

Test-Driven Development (TDD) Java and Spring Boot



Somkiat
Home

Somkiat Puisungnoen

Update Info 1

View Activity Log 10+

...

Timeline

About

Friends 3,138

Photos

More ▾

When did you work at Opendream?

×

...

22 Pending Items

Intro

Software Craftsmanship

Software Practitioner at สยามชำนานุกิจ พ.ศ. 2556

Agile Practitioner and Technical at SPRINT3r

Post

Photo/Video

Live Video

Life Event

What's on your mind?

Public ▾

Post

Somkiat Puisungnoen

15 mins · Bangkok · 🌐 ▾

Java and Bigdata

...



Facebook interface for the page **somkiat.cc**. The top navigation bar includes the Facebook logo, the page name **somkiat.cc**, a search icon, and user profile icons for **Somkiat** and **Home**. The main navigation bar features **Page** (selected), **Messages**, **Notifications** (3), **Insights**, **Publishing Tools**, **Settings**, and **Help**.

The page cover image shows a man in a white Superman t-shirt with "SOMKIAT.CC" printed on it, posing against a white wall. The profile picture is a smaller version of the same image. The page name **somkiat.cc** and handle **@somkiat.cc** are displayed below the profile picture.

The left sidebar contains a menu with **Home** (selected), **Posts**, **Videos**, and **Photos**.

The main content area displays the cover image with a blue call-to-action button that says "Help people take action on this Page." and a blue button labeled "+ Add a Button". Below the cover image, there are buttons for **Liked**, **Following**, **Share**, and a three-dot menu icon.



**[https://github.com/up1/
course-java-spring-2026](https://github.com/up1/course-java-spring-2026)**



Test-Driven Development (TDD)

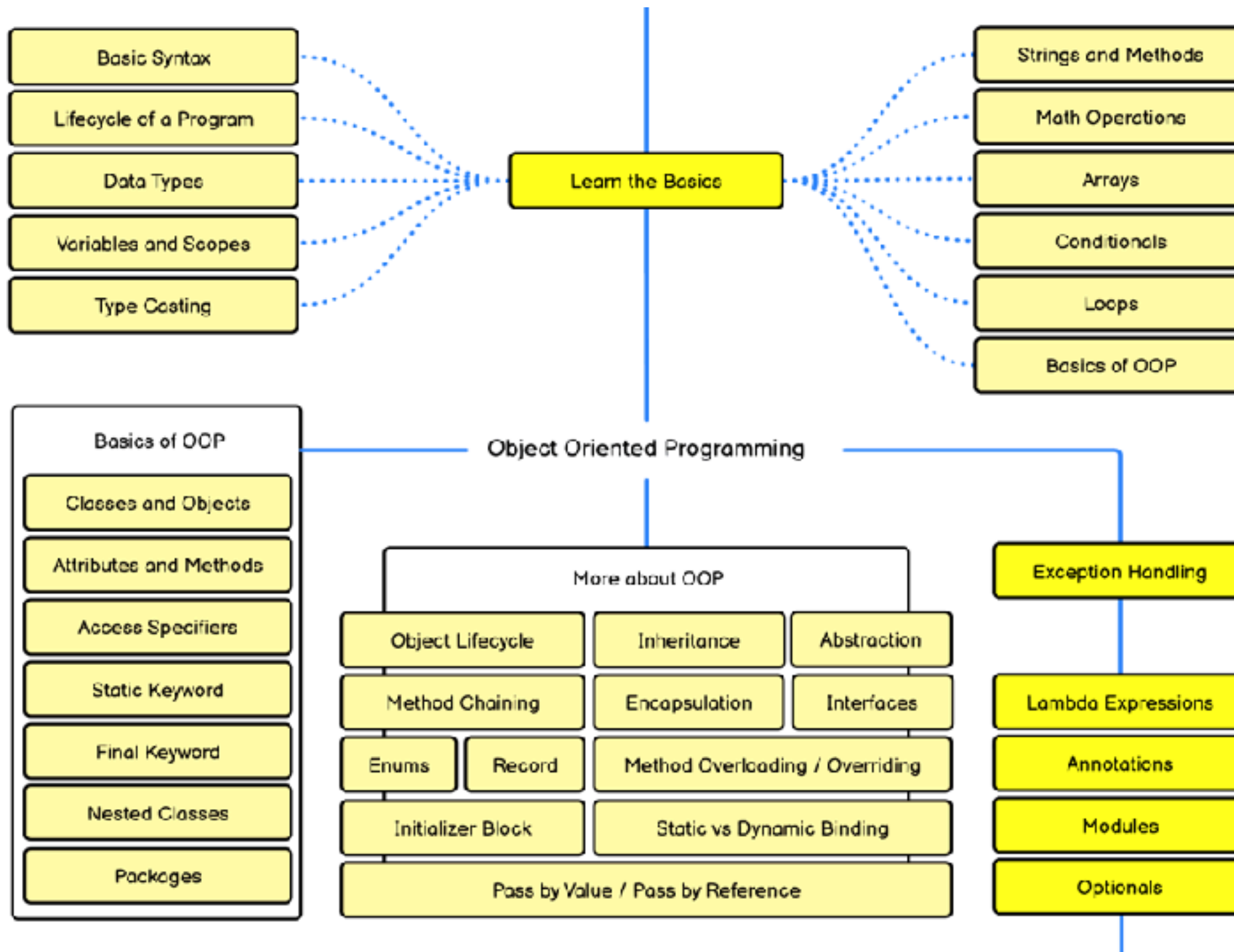


Leaning Java

Testing with Java
Testing strategies
Test-Driven Development (TDD)
TDD Life Cycle
Thinking before Coding
Testing with JUnit and Mockito
Workshop



Java Developer Roadmap



<https://roadmap.sh/java>



Spring Boot

Better project structure of Spring Boot

Develop RESTful APIs

Error handling

Working with external system (DB, API)

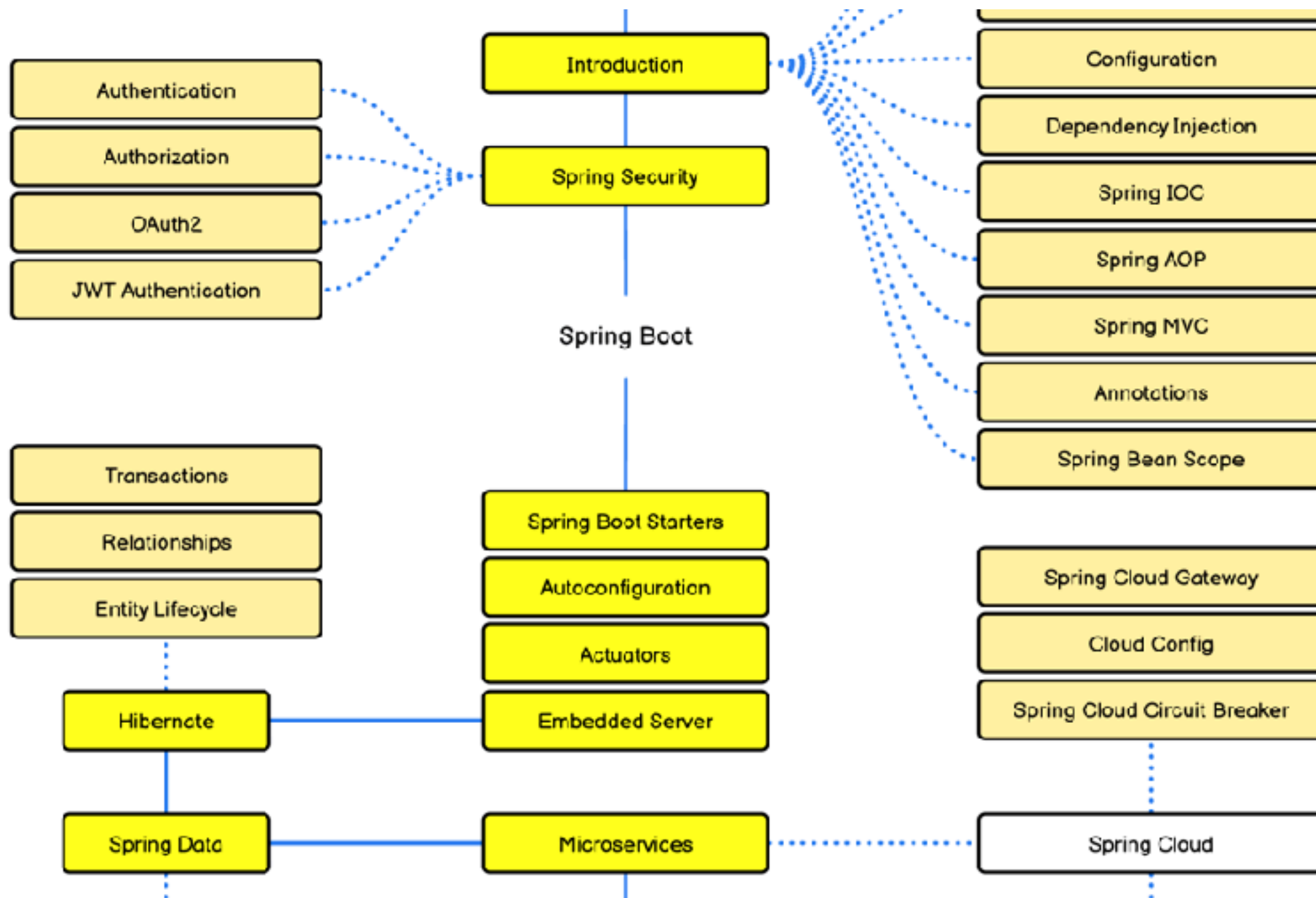
Testing with Spring Boot

Types of testing in Spring Boot

Workshop



Spring Boot Roadmap



<https://roadmap.sh/spring-boot>



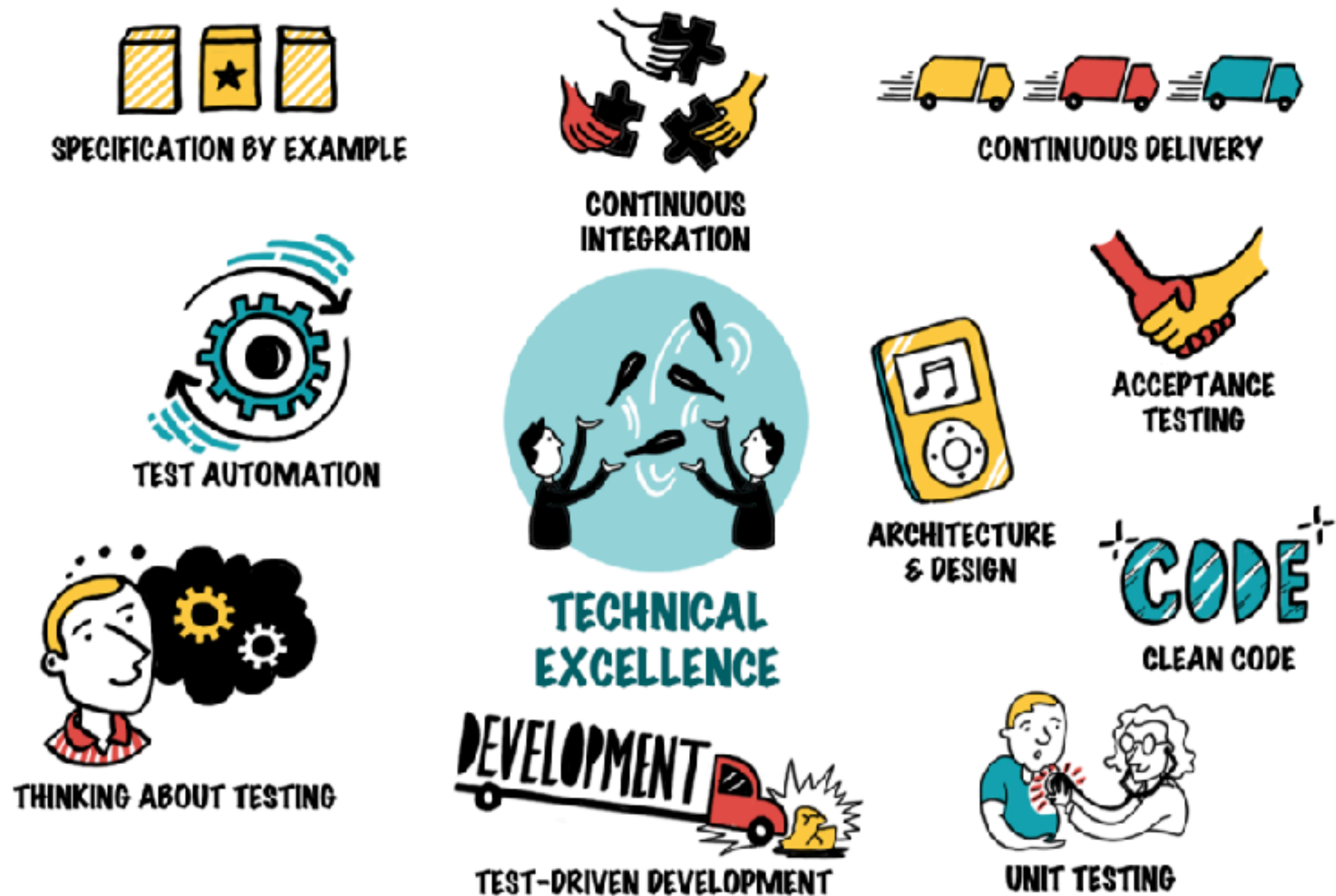
Software Development



Technical Excellence



Technical Excellence



<https://less.works/less/technical-excellence>



Learning Path

Java

Testing

Spring Boot

JUnit

Types of
Testing

Spring Boot
Testing

Good
testing

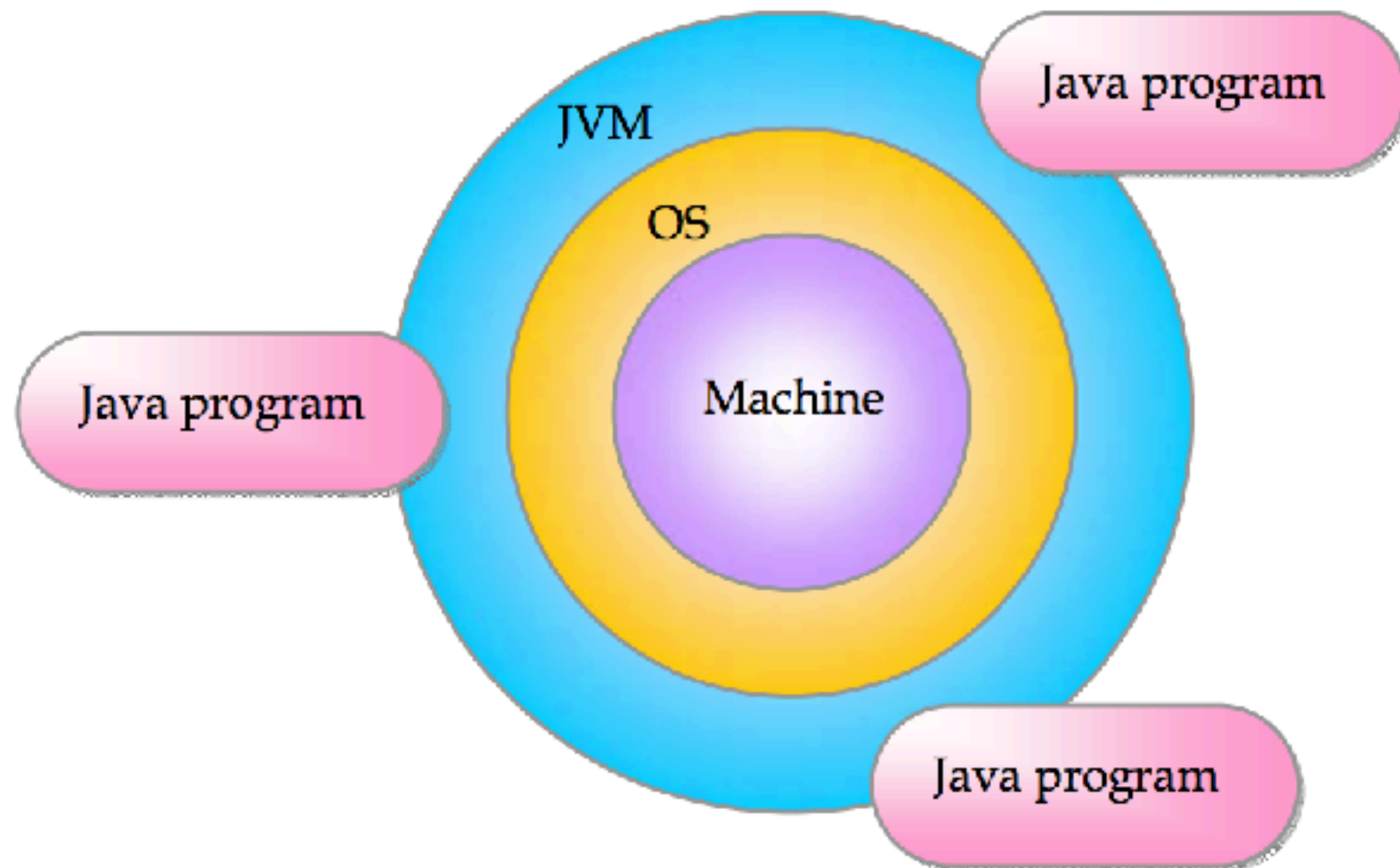
TDD



Modern Java



Write Once Run Anywhere ?



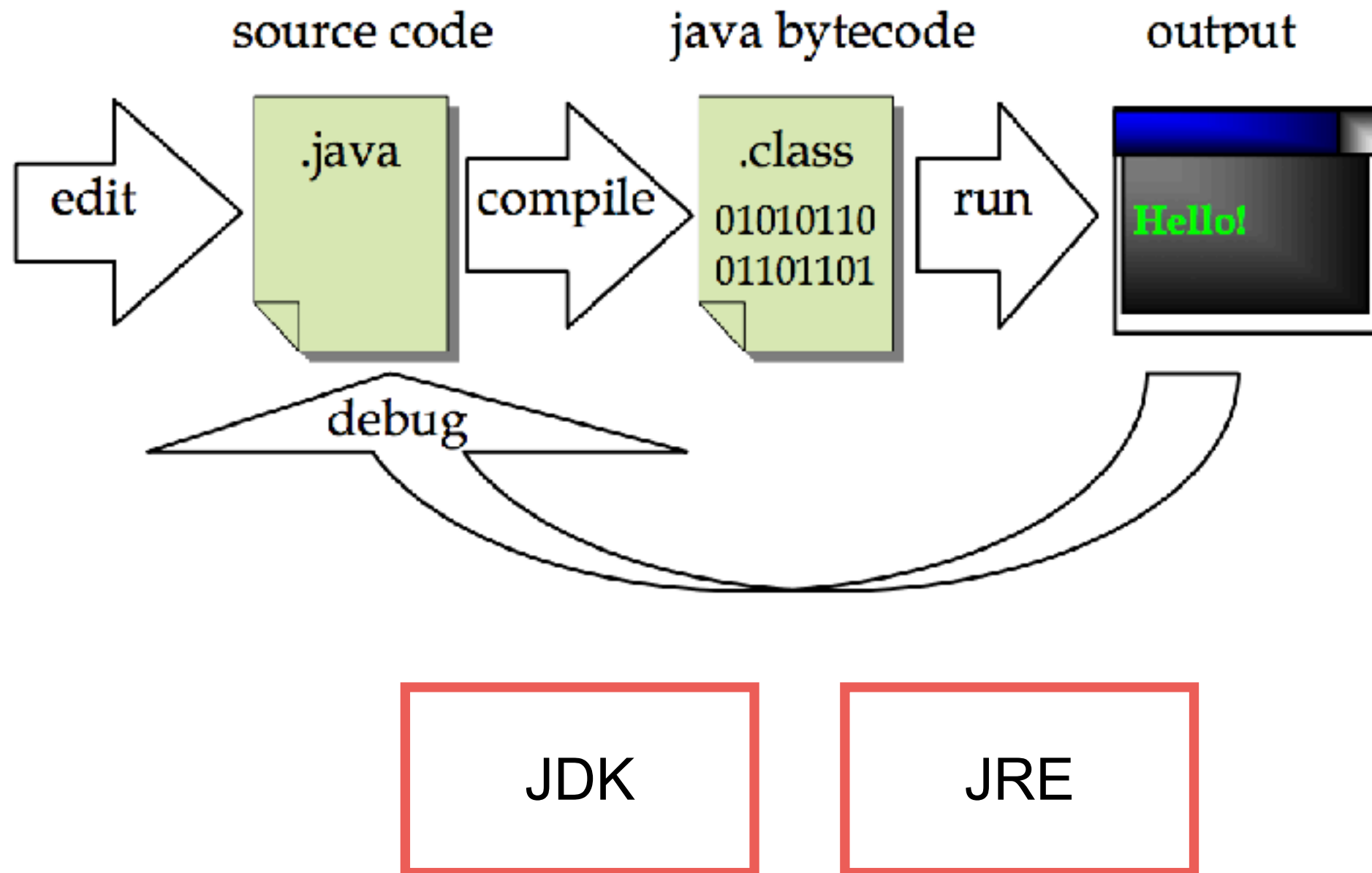
Structure of Java program

Create file HelloWorld.java

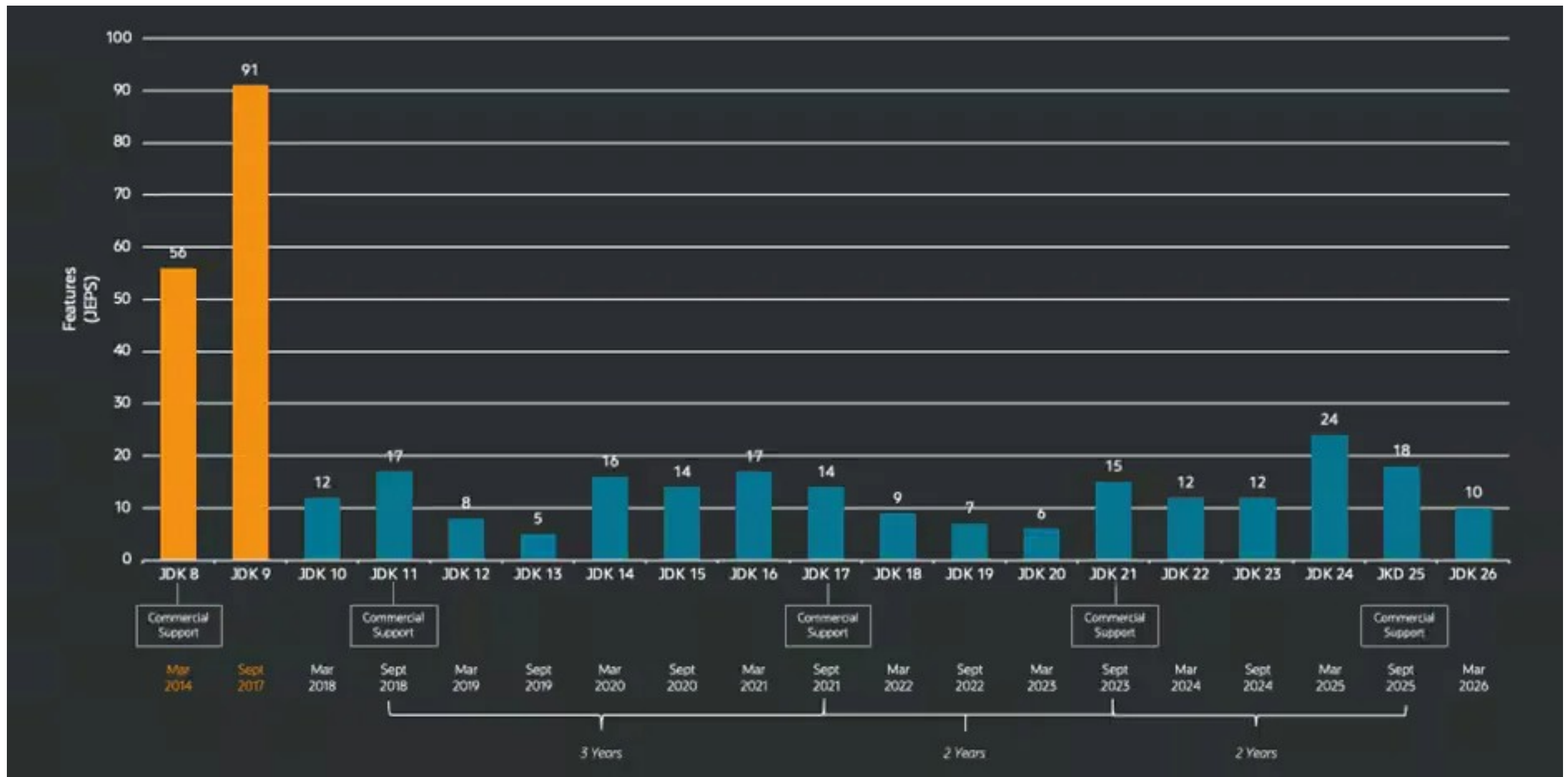
```
1 public class HelloWorld {  
2  
3     // Show message in a console  
4     public static void main(String[] args) {  
5         System.out.println("Hello World");  
6  
7     }  
8  
9 }
```



Java Development Workflow



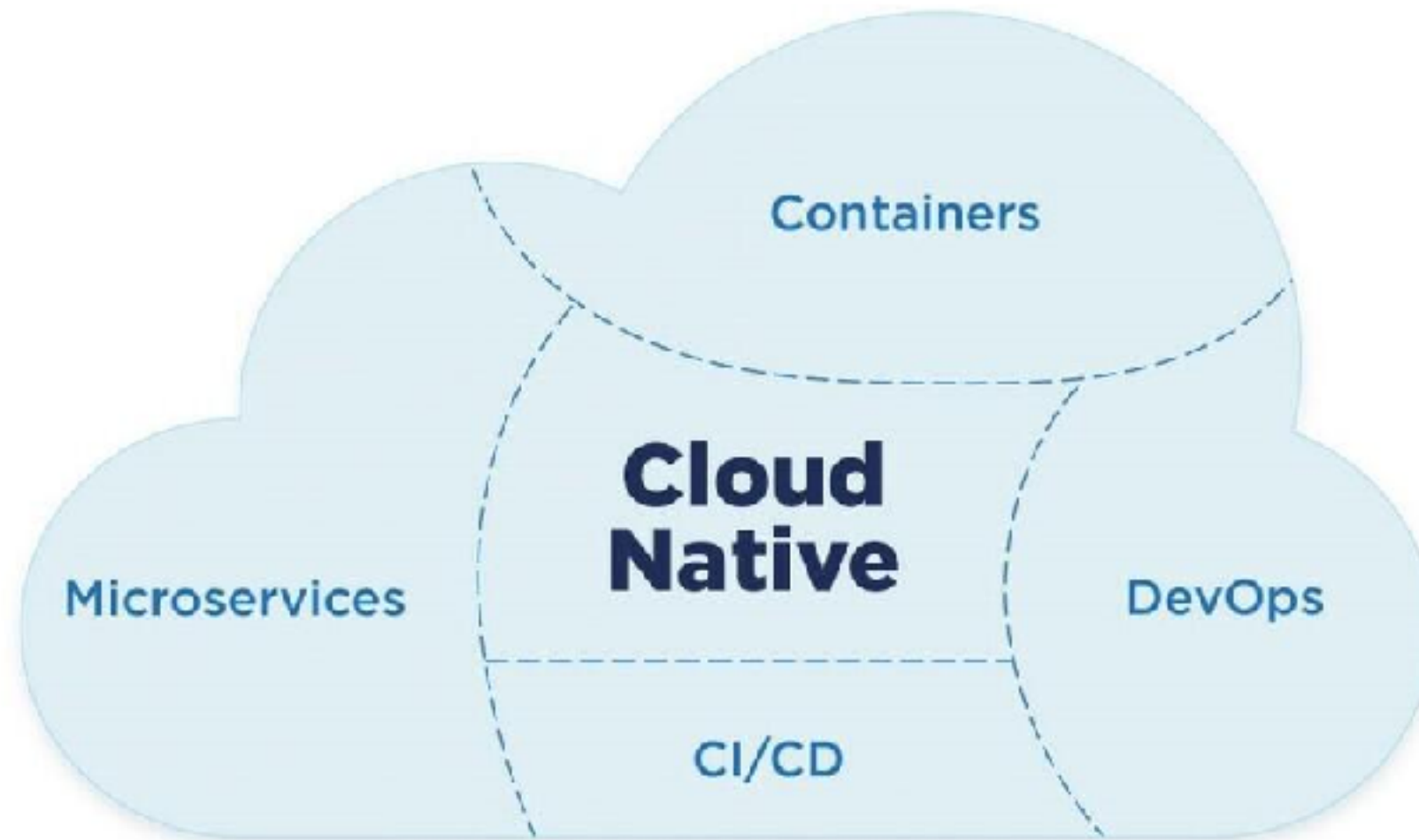
History of Java from 8 to 26



<https://www.infoq.com/news/2026/03/java26-released/>



Cloud Native Architecture



Java in Cloud Native App

Improve performance and resource usages

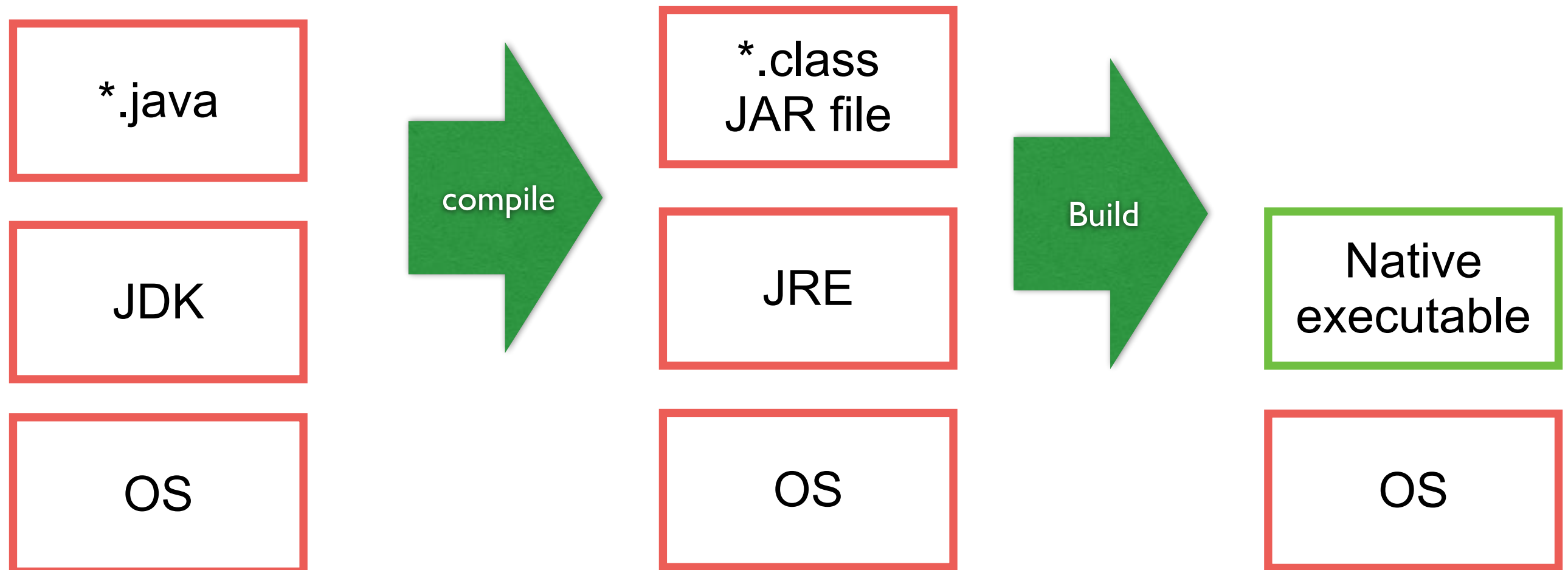


QUARKUS

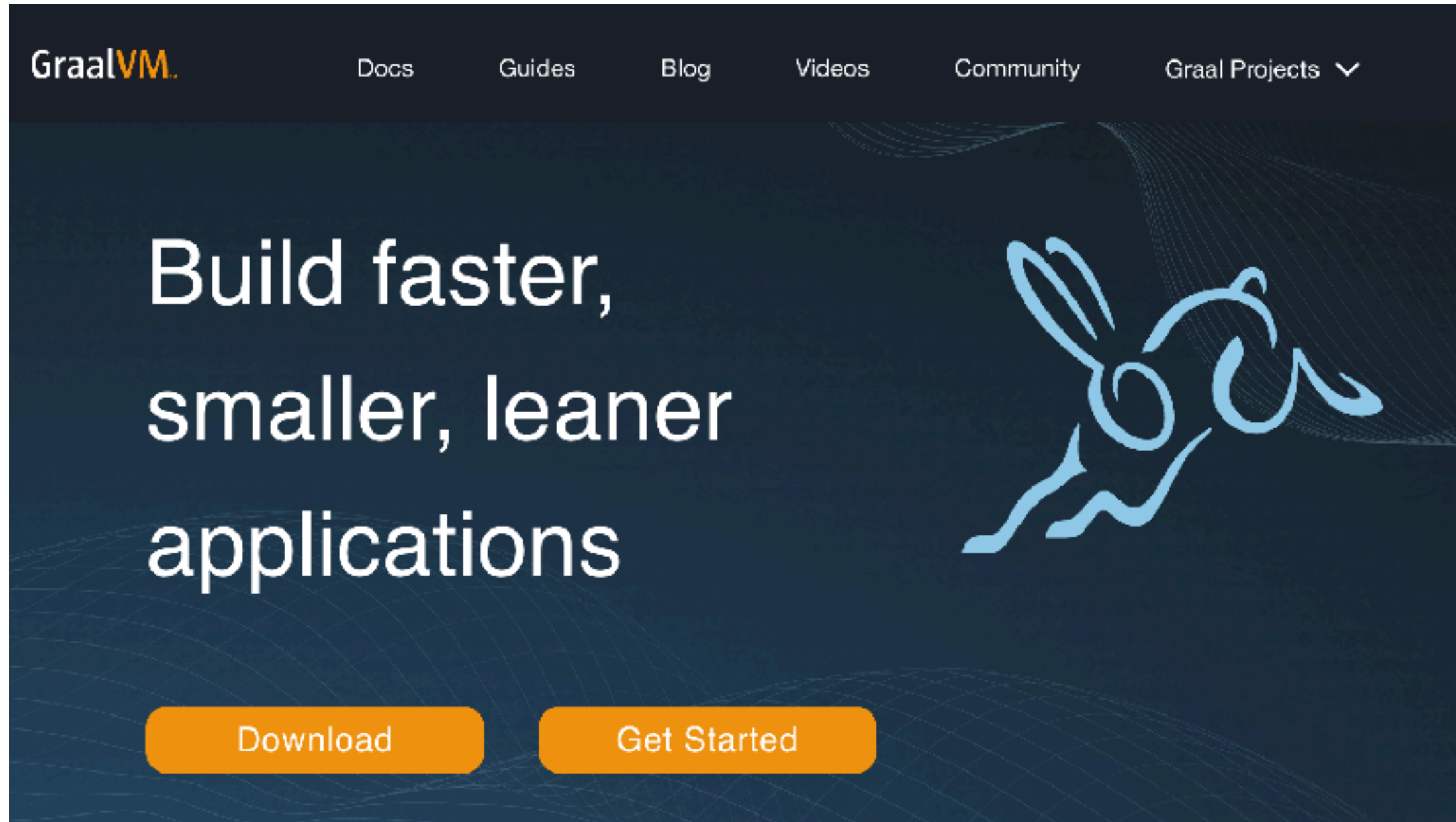
GraalVM™



Java in Cloud Native



Working with GraalVM



<https://www.graalvm.org/>



Native Image of GraalVM

Use less resources (compare with JRE)

Start in milliseconds

Deliver peak performance immediately (no warmup)

Lightweight package

Reduce attack surface

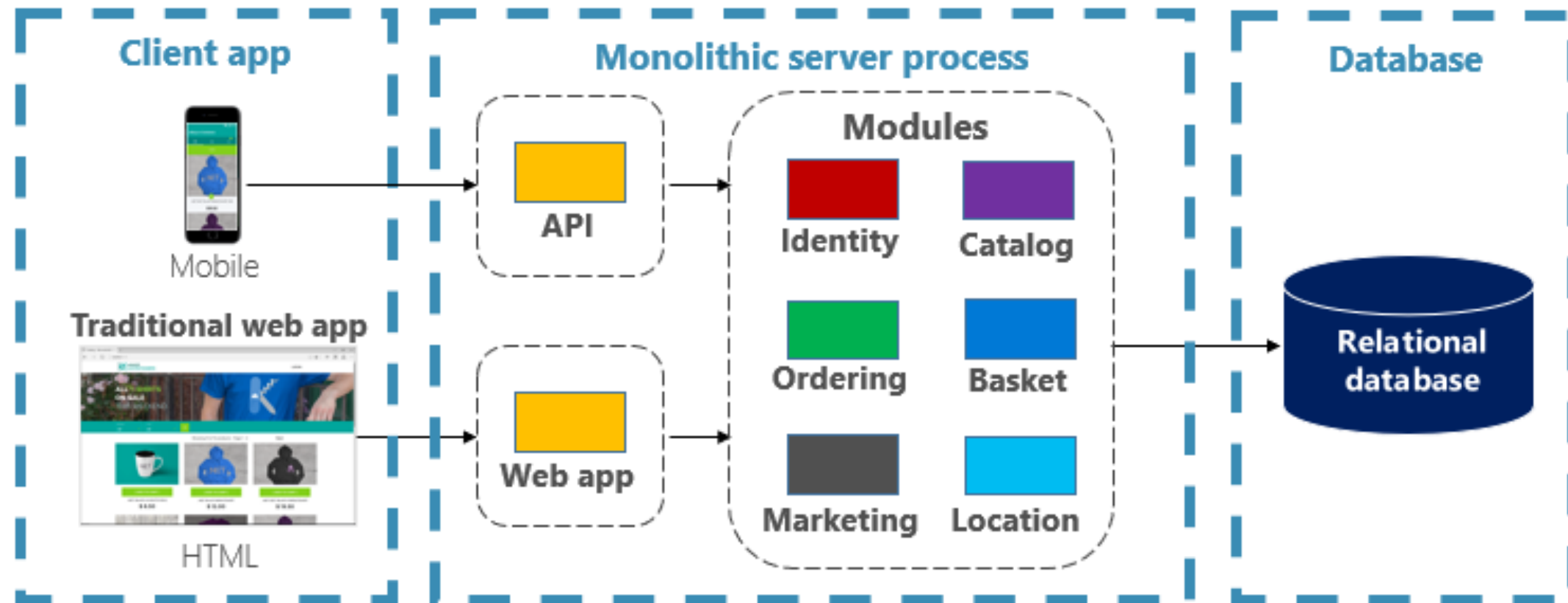
<https://www.graalvm.org/latest/reference-manual/native-image/>



Cloud Native Architecture Design



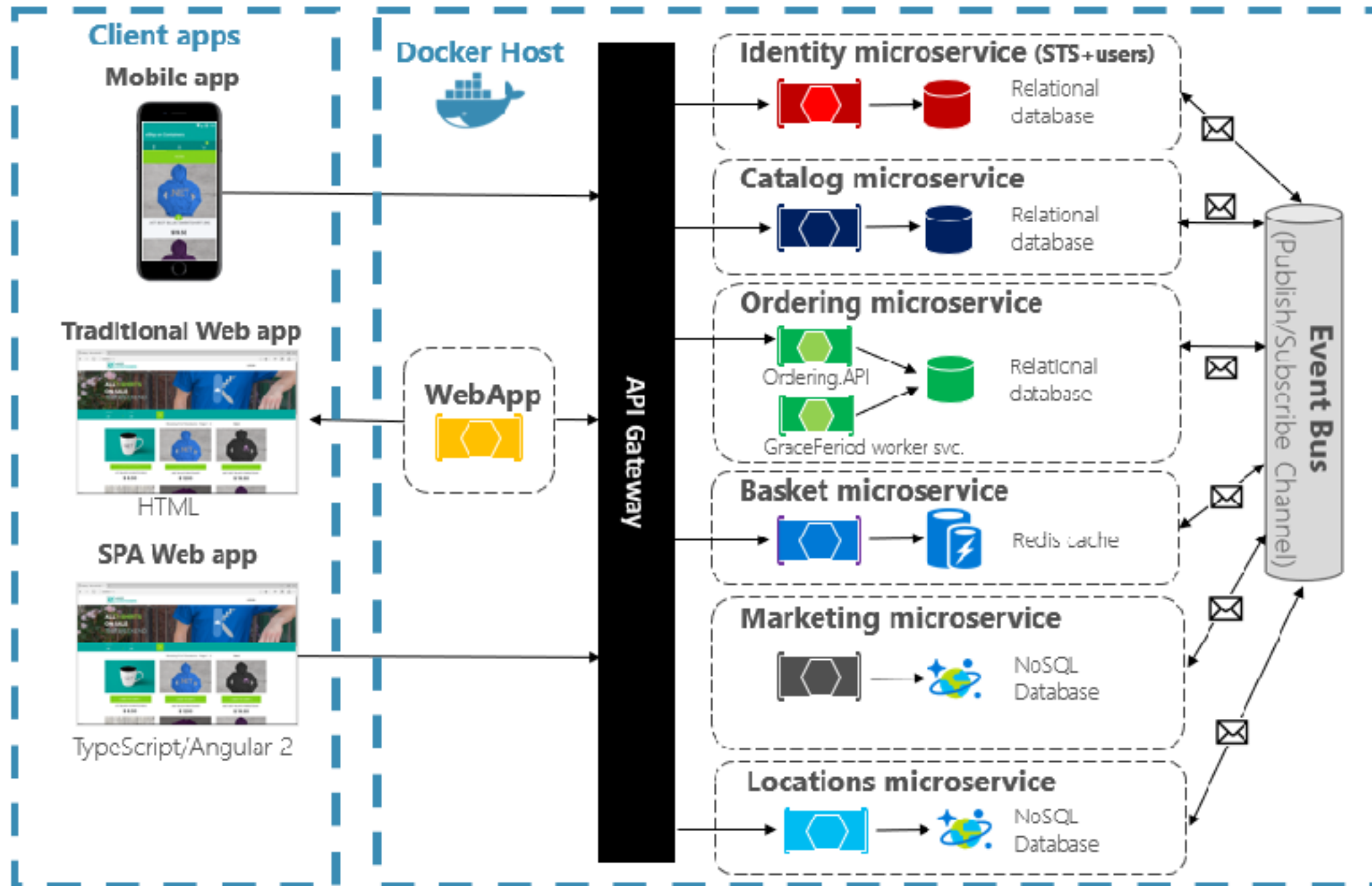
Monolithic



<https://learn.microsoft.com/th-th/dotnet/architecture/cloud-native/introduction>



Microservices



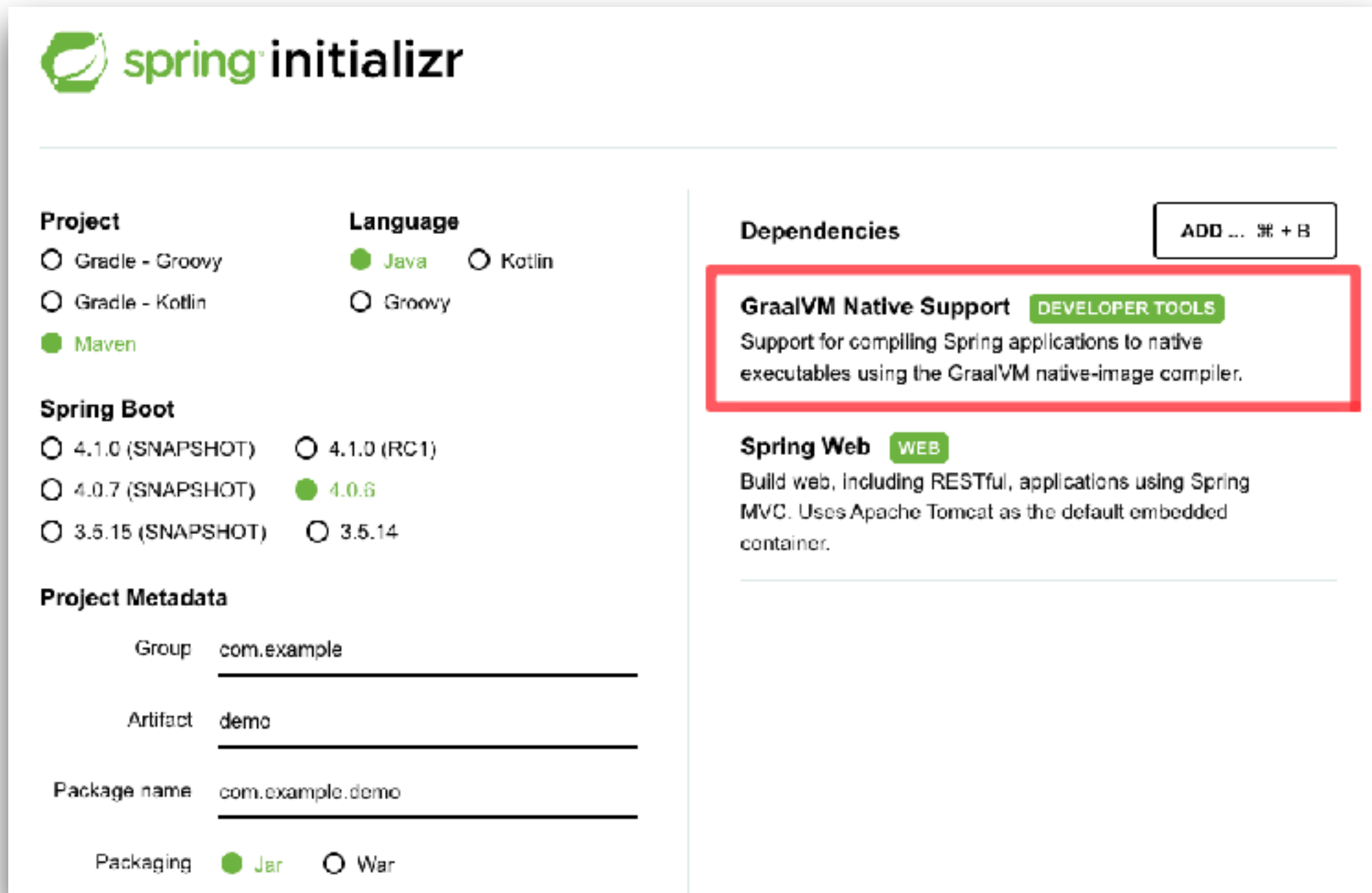
<https://learn.microsoft.com/th-th/dotnet/architecture/cloud-native/introduction>



GraalVM in Spring Boot



GraalVM in Spring Boot



The image shows a screenshot of the Spring Initializr web form. The form is divided into several sections: Project, Language, Spring Boot, Project Metadata, Dependencies, and a right-hand sidebar. The 'Project' section has radio buttons for 'Gradle - Groovy', 'Gradle - Kotlin', and 'Maven' (selected). The 'Language' section has radio buttons for 'Java' (selected), 'Kotlin', and 'Groovy'. The 'Spring Boot' section has radio buttons for '4.1.0 (SNAPSHOT)', '4.1.0 (RC1)', '4.0.7 (SNAPSHOT)', '4.0.6' (selected), '3.5.15 (SNAPSHOT)', and '3.5.14'. The 'Project Metadata' section has text input fields for 'Group' (com.example), 'Artifact' (demo), and 'Package name' (com.example.demo). The 'Packaging' section has radio buttons for 'Jar' (selected) and 'War'. The 'Dependencies' section has a button 'ADD ... + B'. The right-hand sidebar contains two sections: 'GraalVM Native Support' with a 'DEVELOPER TOOLS' tag and a description 'Support for compiling Spring applications to native executables using the GraalVM native-image compiler.', and 'Spring Web' with a 'WEB' tag and a description 'Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.'

Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Maven

Language

☒ Java ☐ Kotlin

☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT) ☐ 4.1.0 (RC1)

☐ 4.0.7 (SNAPSHOT) ☒ 4.0.6

☐ 3.5.15 (SNAPSHOT) ☐ 3.5.14

Project Metadata

Group

Artifact

Package name

Packaging ☒ Jar ☐ War

Dependencies ADD ... + B

GraalVM Native Support DEVELOPER TOOLS

Support for compiling Spring applications to native executables using the GraalVM native-image compiler.

Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

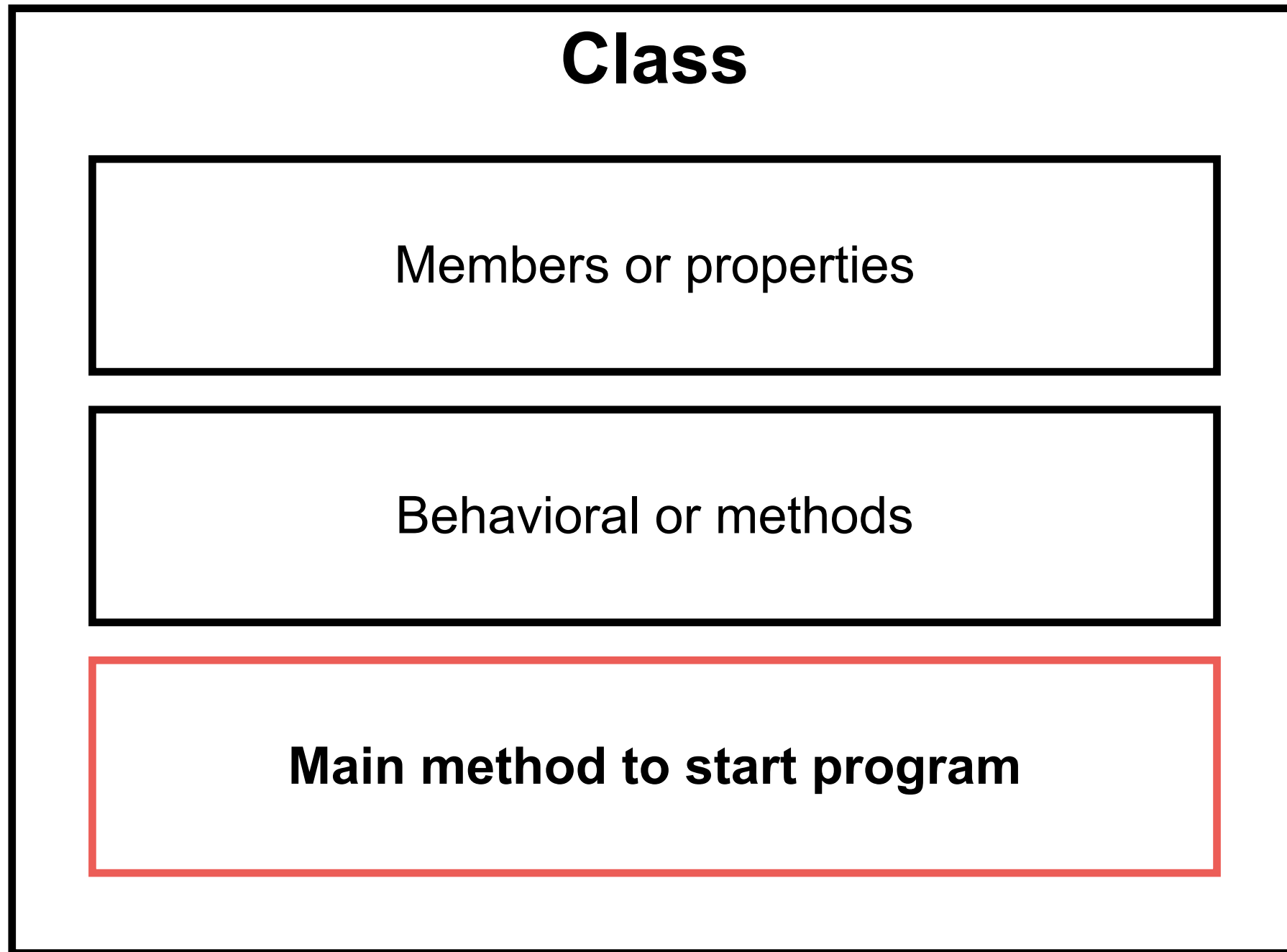
<https://start.spring.io/>



Object-Oriented Programming



Structure of Java program



Thinking before create class



Thinking before create class



Single Responsibility Principle



Single Responsibility Principle

Just because you *can* doesn't mean you *should*.



Open-Closed Principle



Open-Closed Principle

Open-chest surgery isn't needed when putting on a coat.

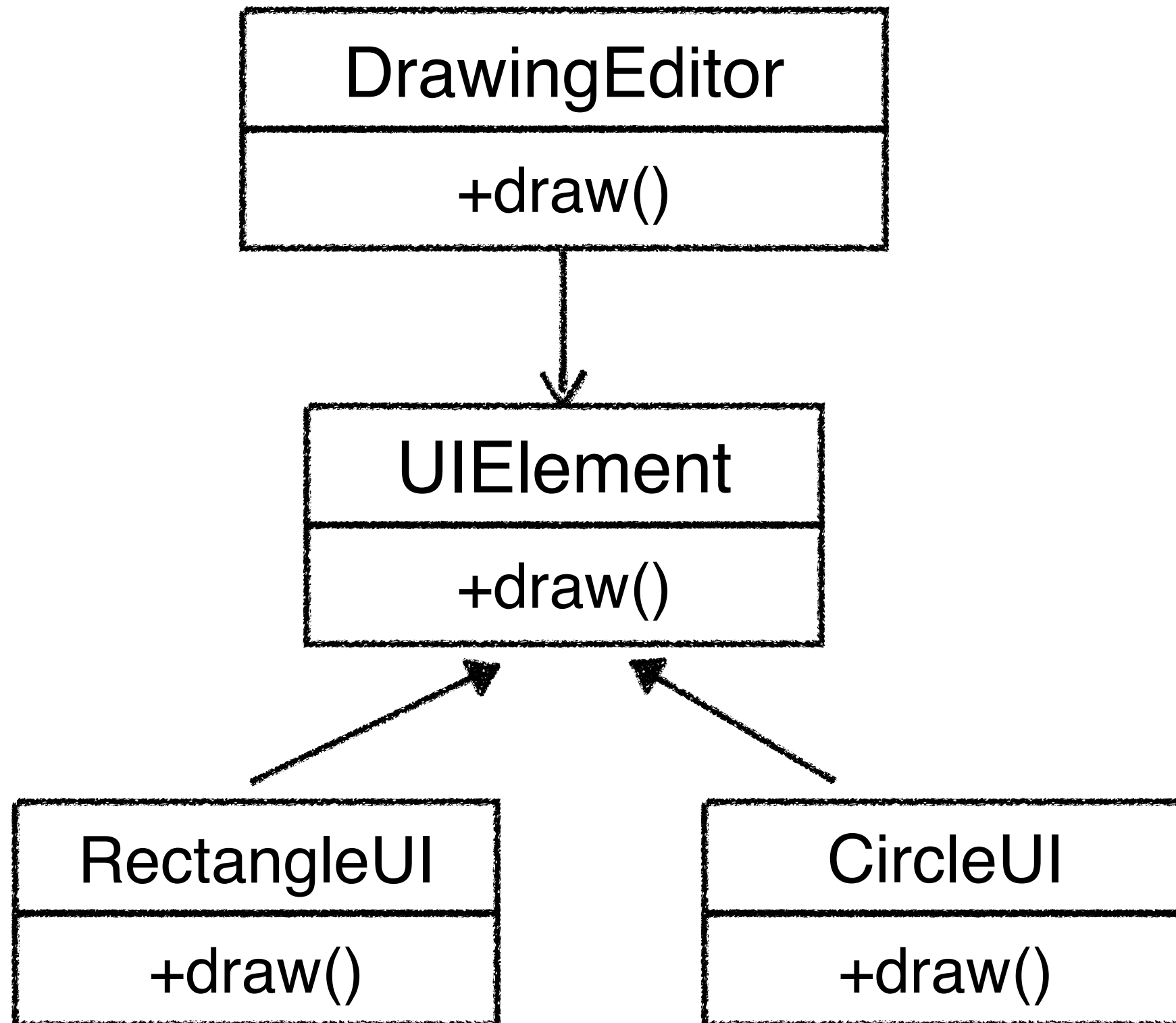


Example

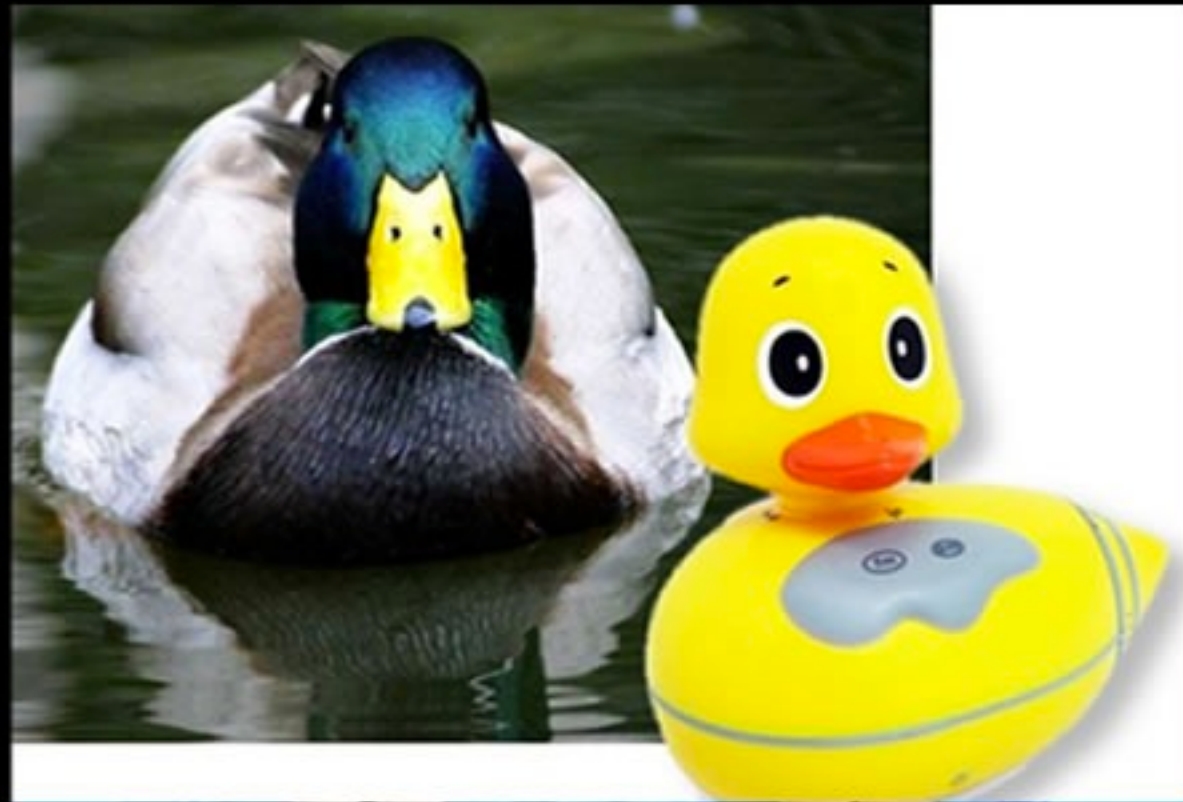
DrawingEditor

+draw()
-drawCircle()
-drawRectangle()





Liskov Substitution Principle



Liskov Substitution Principle

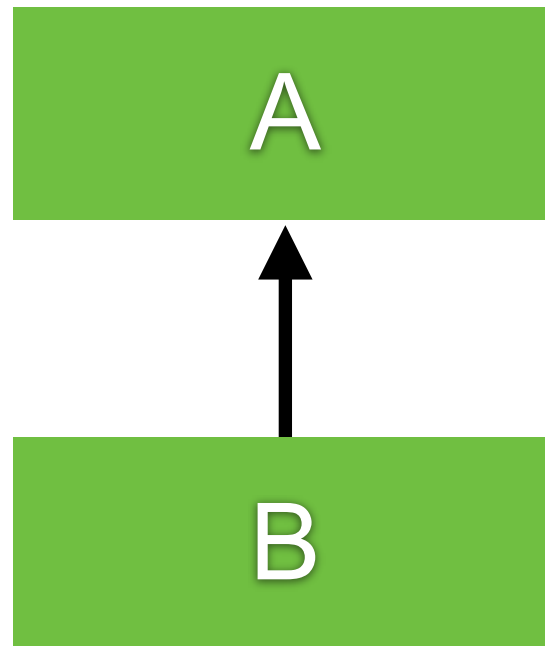
If it looks like a duck and quacks like a duck but needs batteries, you probably have the wrong abstraction.



Inheritance

Ability to derive a new class from existing class

superclass / base class



subclass / derived class

Additional properties and behaviors



How to create ?

Extends from base class only one class

```
class BaseClass {  
  
}
```

```
class SubClass extends BaseClass {  
  
}
```



More OOP with Java

Abstract classes
Interfaces



Abstract class



Interface



Abstract classes

Placeholder for a method that fully defined in a subclass

Class have abstract modifier

Method can have abstract modifier

Abstract method can't be private

Abstract method can't have body

*“Class that contains an abstract method is called
Abstract class”*



Classes

*“Class that contains an abstract method is called
Abstract class”*

*“Class that not contains an abstract method is called
Concrete class”*



Abstract classes

```
abstract class Employee {
```

```
    private int id;
```

```
    public abstract double getSalary();
```

```
    public boolean sameSalary(Employee other) {  
        return this.getSalary() == other.getSalary();  
    }
```

```
}
```



Pitfall of abstract class

You can't create instances of an abstract class
Abstract class can only use to derive more classes



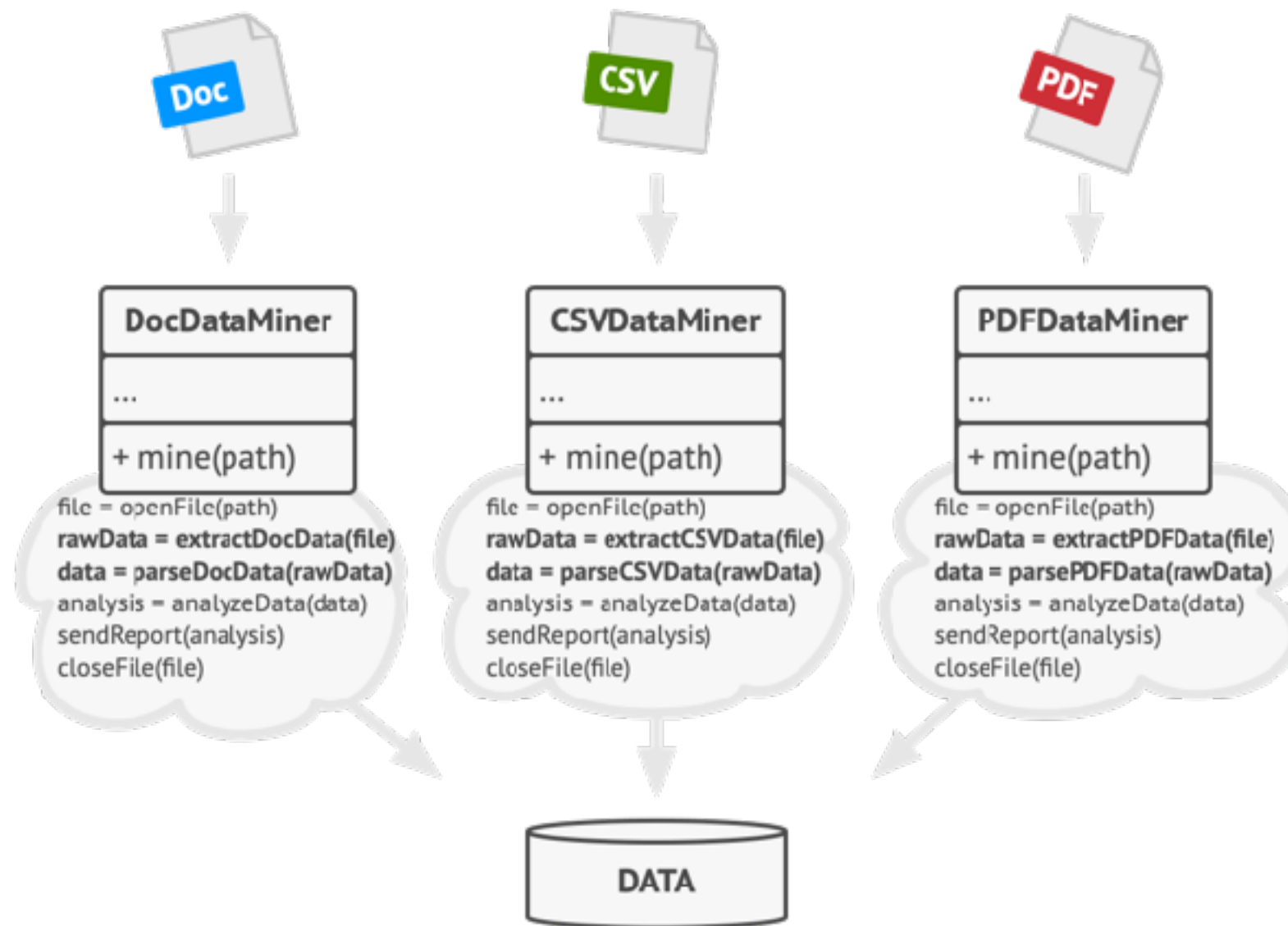
Purpose of abstract class

Create a function as base class
But can be extended by sub class

```
abstract class Report {  
    public abstract void createHeader();  
    public abstract void createBody();  
    public abstract void createFooter();  
  
    public void generate() {  
        createHeader();  
        createBody();  
        createFooter();  
    }  
}
```



Template Method Pattern



<https://refactoring.guru/design-patterns/template-method>



Interfaces

Like an extreme abstract class

All of methods in an interface are abstract by default

Not a class

No instance variables

```
interface Animal {  
    void eat();  
    void sleep();  
}
```



Using interface from class (1)

```
class Cat implements Animal {
```

```
    @Override  
    public void eat() {  
    }
```

```
    @Override  
    public void sleep() {  
    }
```

```
}
```



Using interface from class (2)

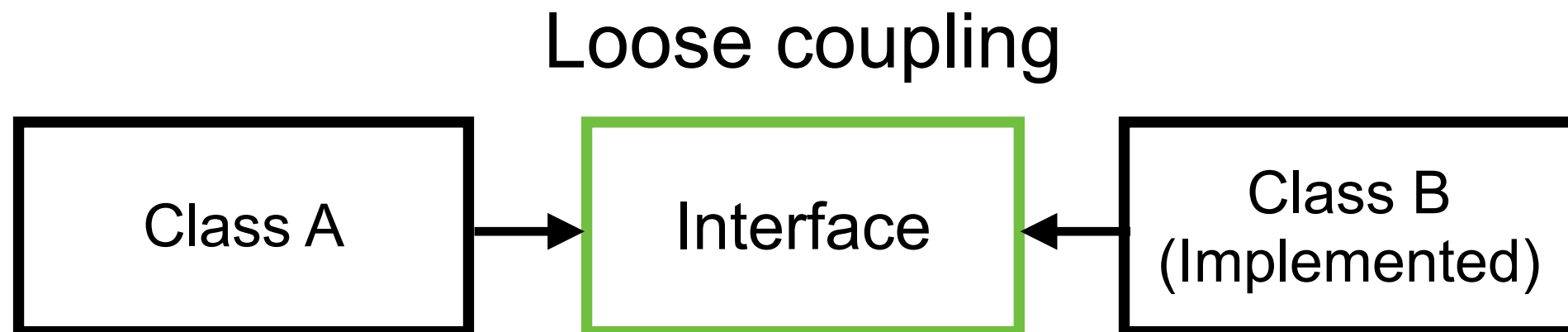
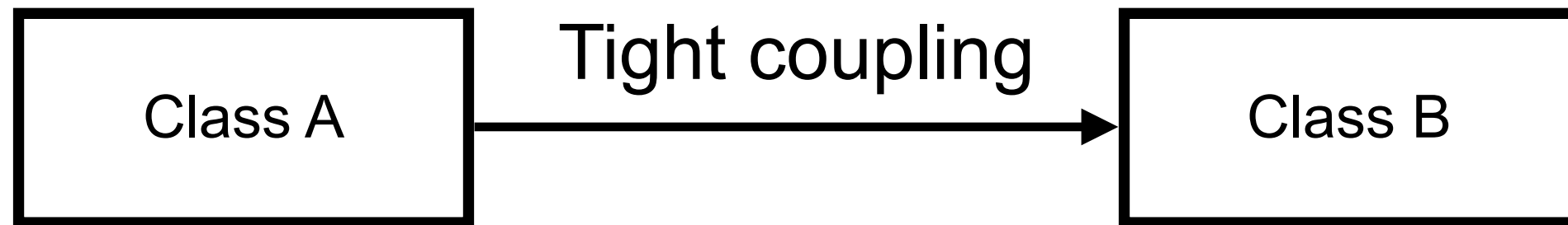
```
class Cat implements Animal {
```

```
    @Override  
    public void eat() {  
    }  
  
    @Override  
    public void sleep() {  
    }  
}
```

Need to implements all method from interface



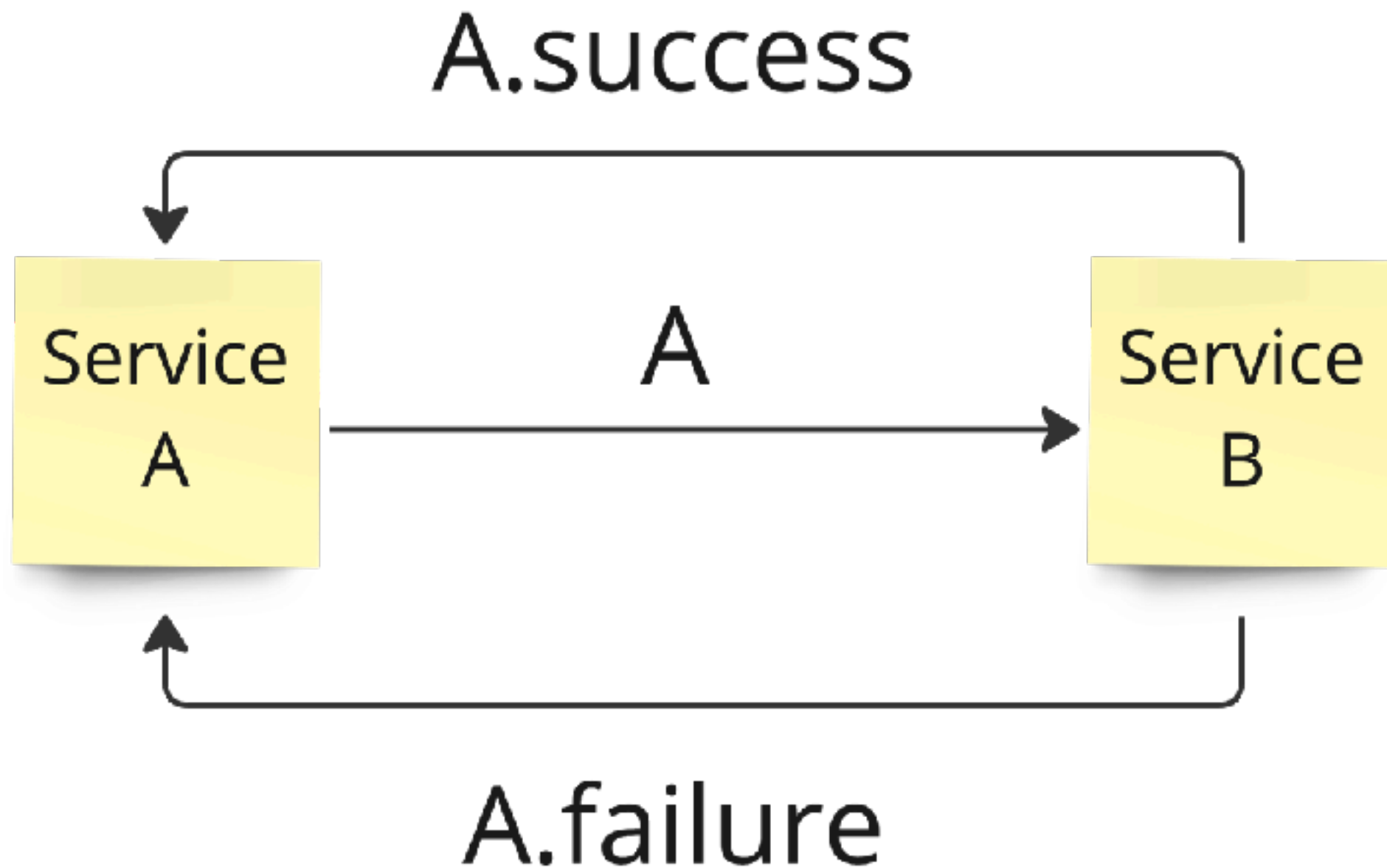
When to use interface ?



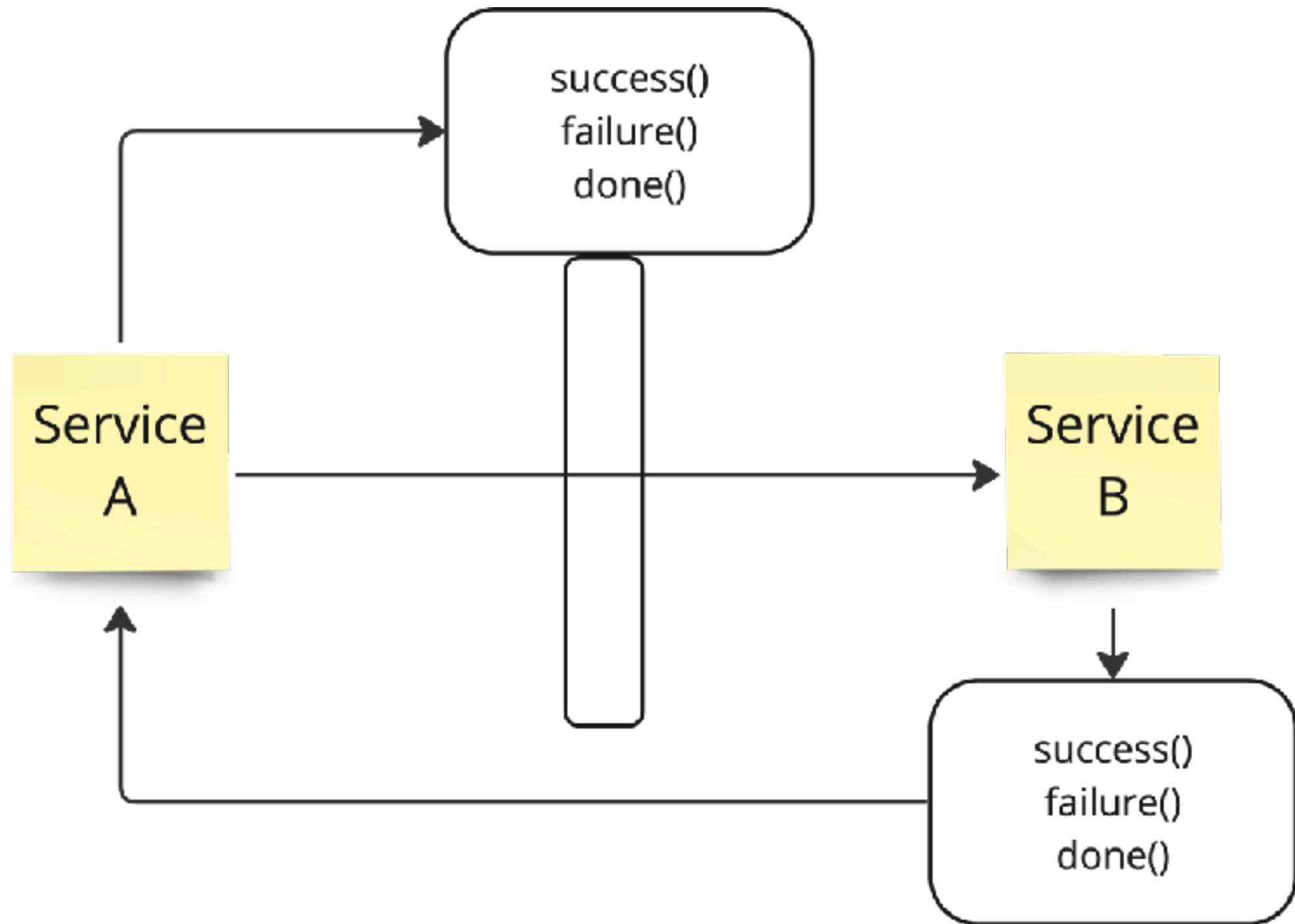
Callback with Interface



Call back with class (1)



Call back with interface (2)



Interface Segregation Principle



Interface Segregation Principle

You want me to plug this in *where?*



Dependency Inversion Principle

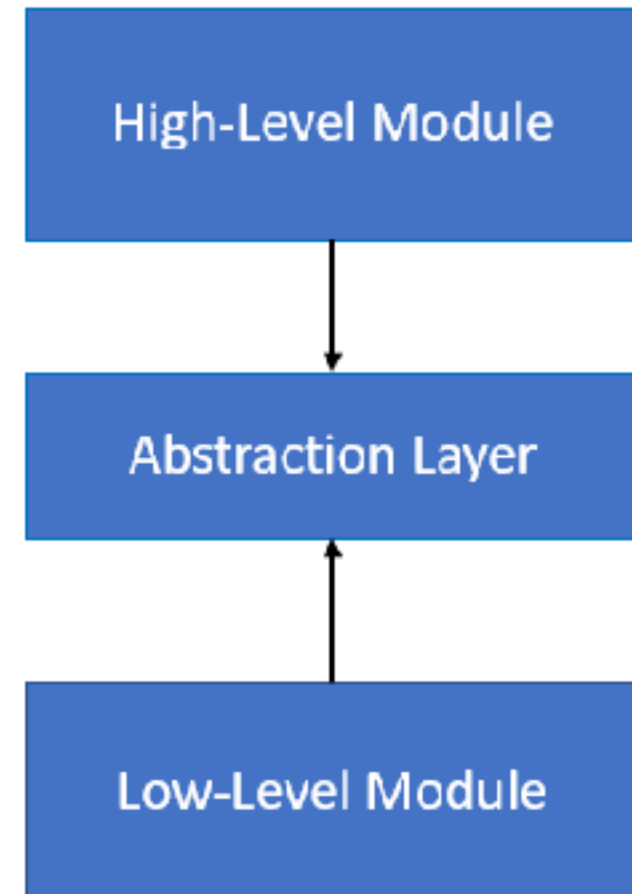
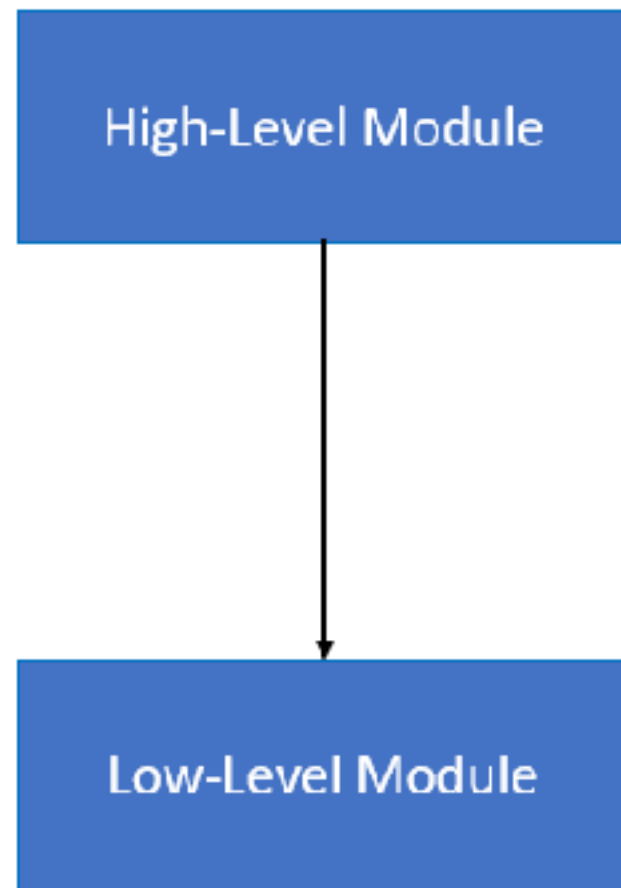


Dependency Inversion Principle

Would you solder a lamp directly
to the electrical wiring in a wall?

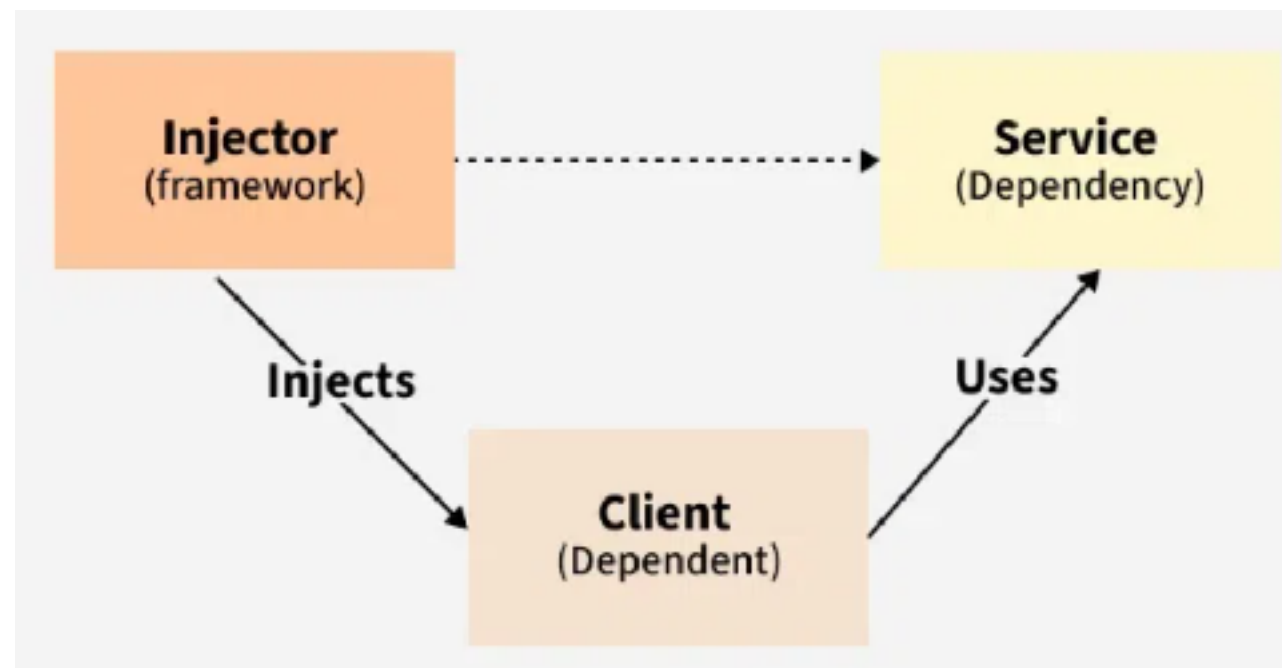


Depend on abstraction



Dependency Injection

Property injection
Constructor injection
Method injection
Interface injection

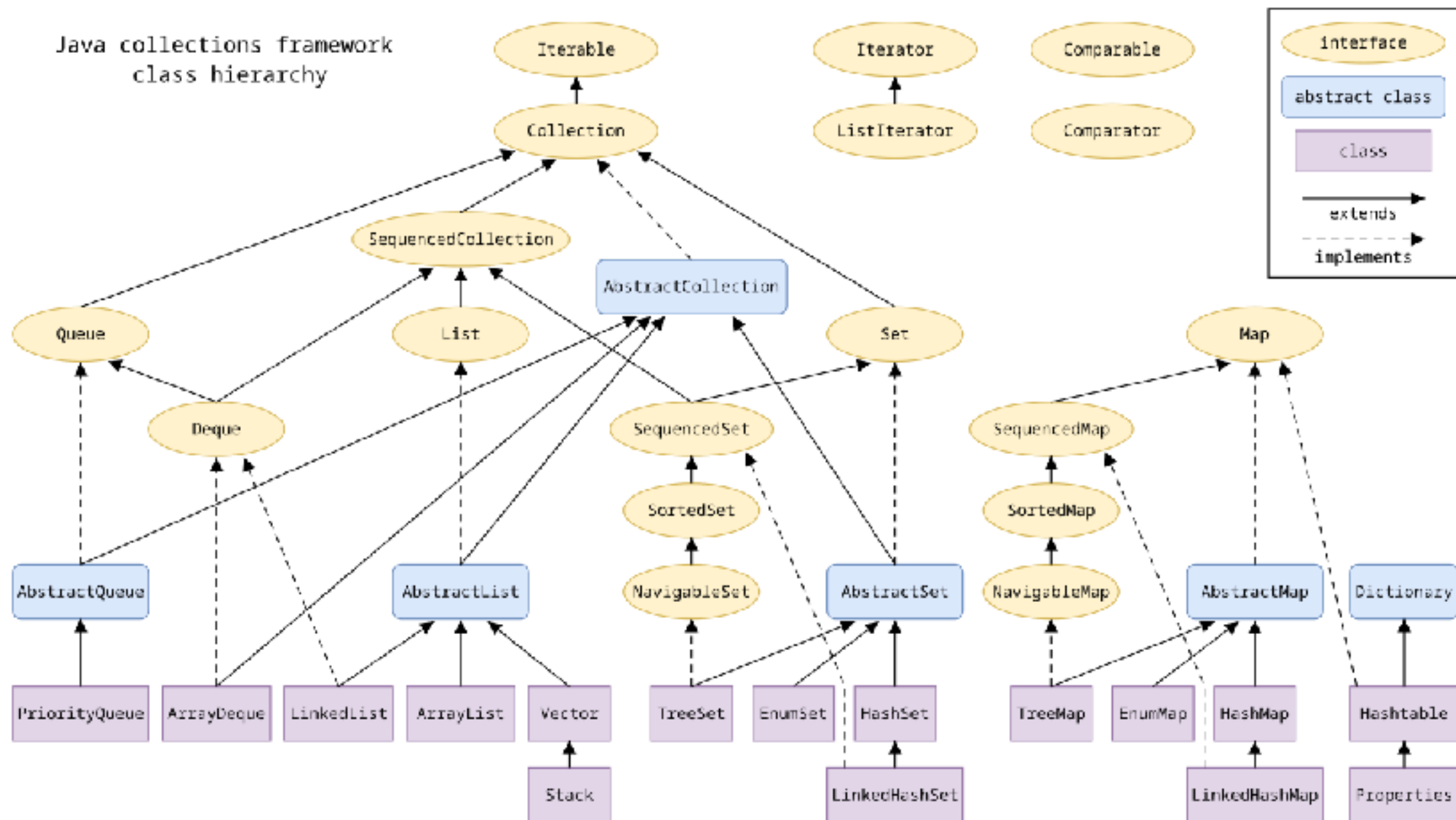


Java Collection Framework

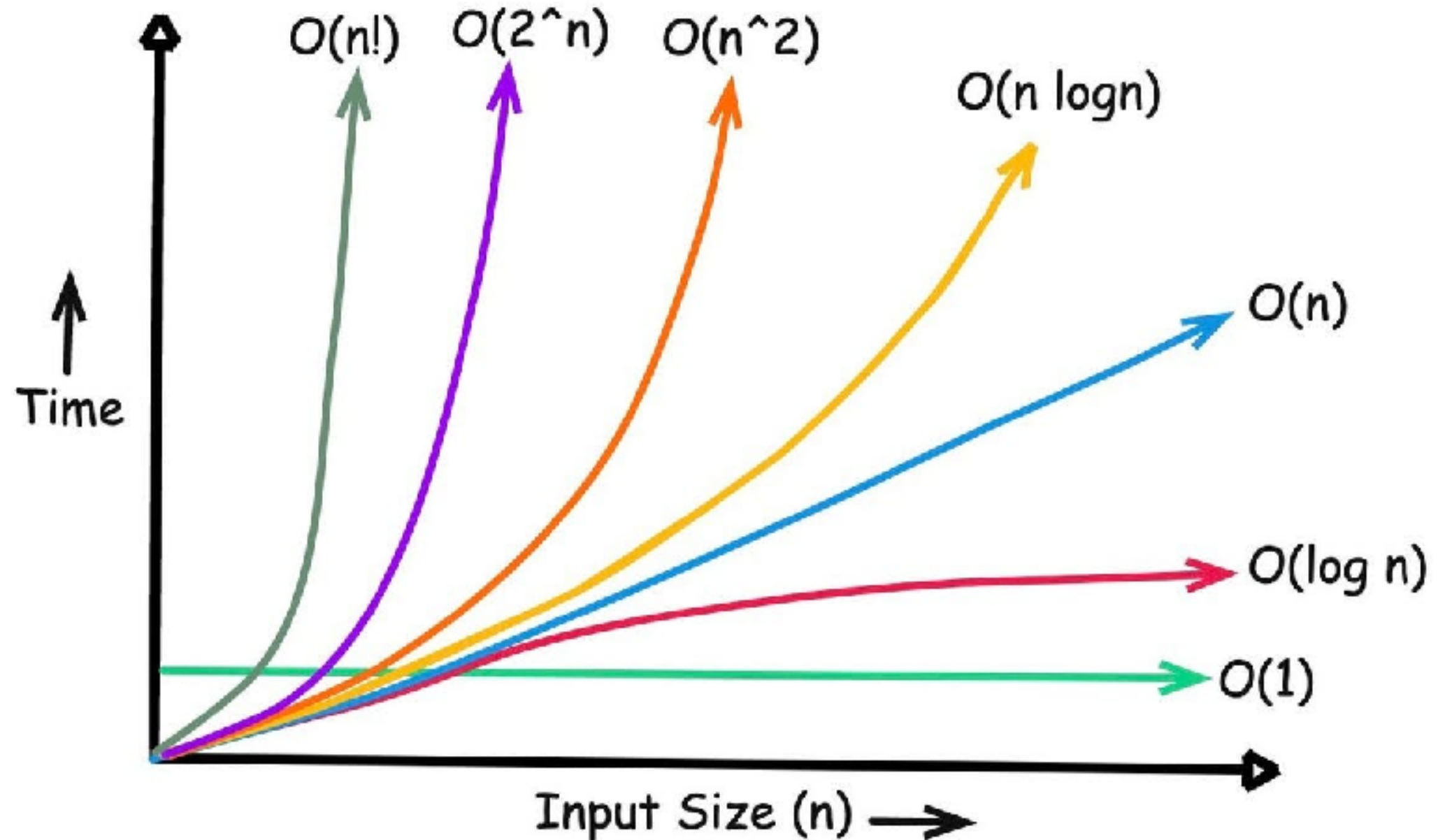
<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/doc-files/coll-overview.html>



Java Collection Framework



Big O Notation

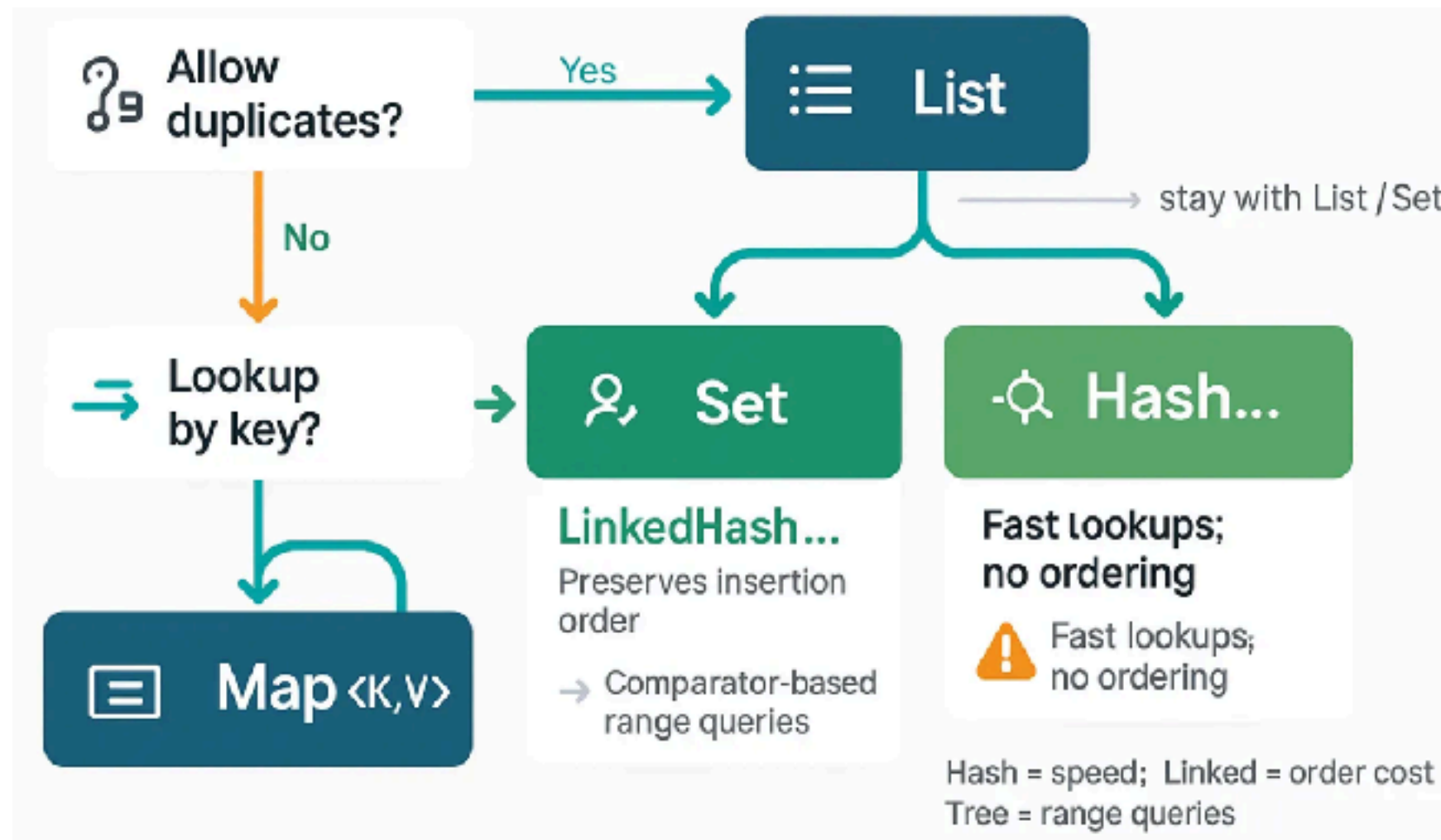


Working with use cases



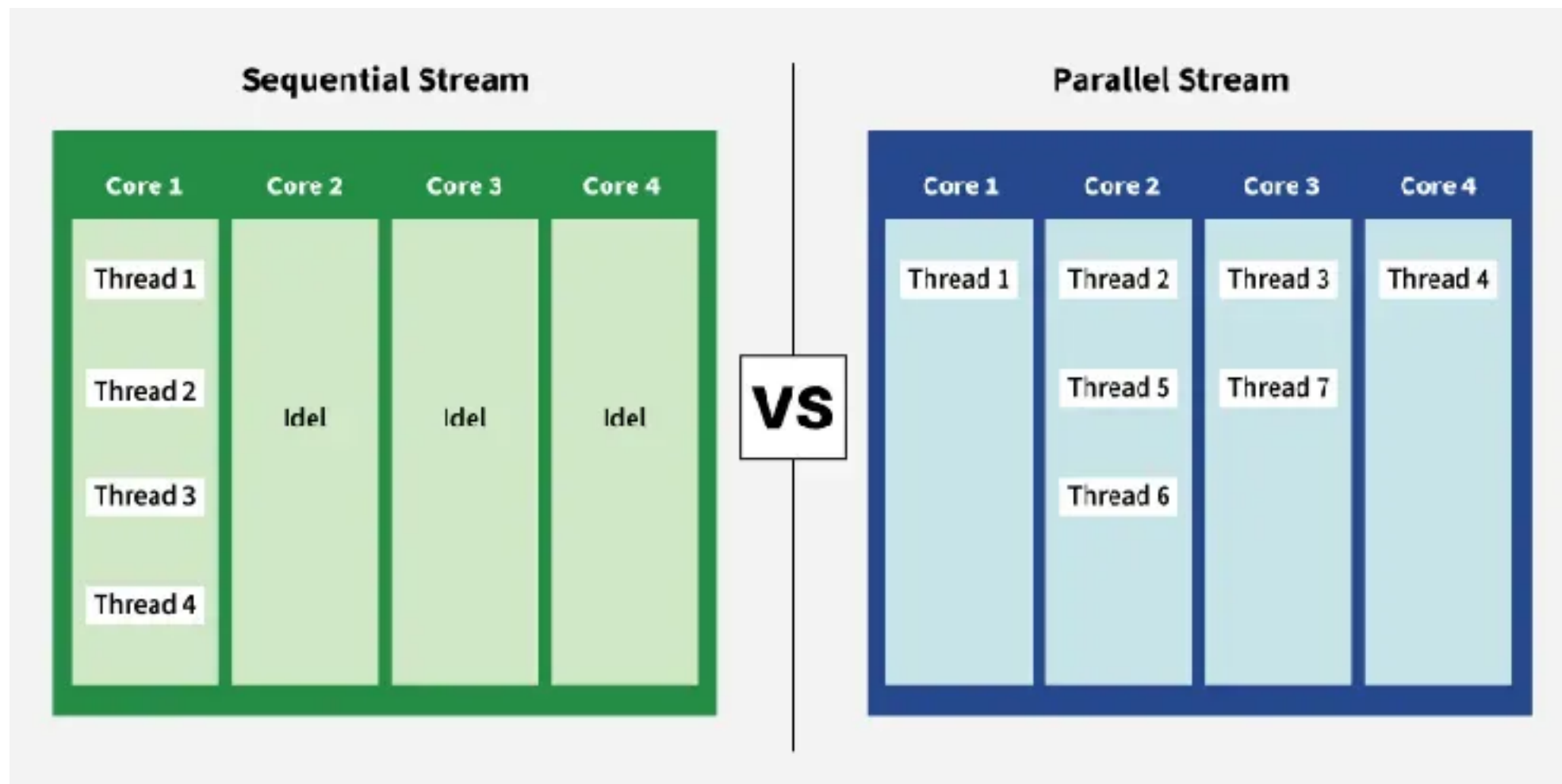
List vs Map

Lookup optimization



Java Stream

Sequential vs Parallel



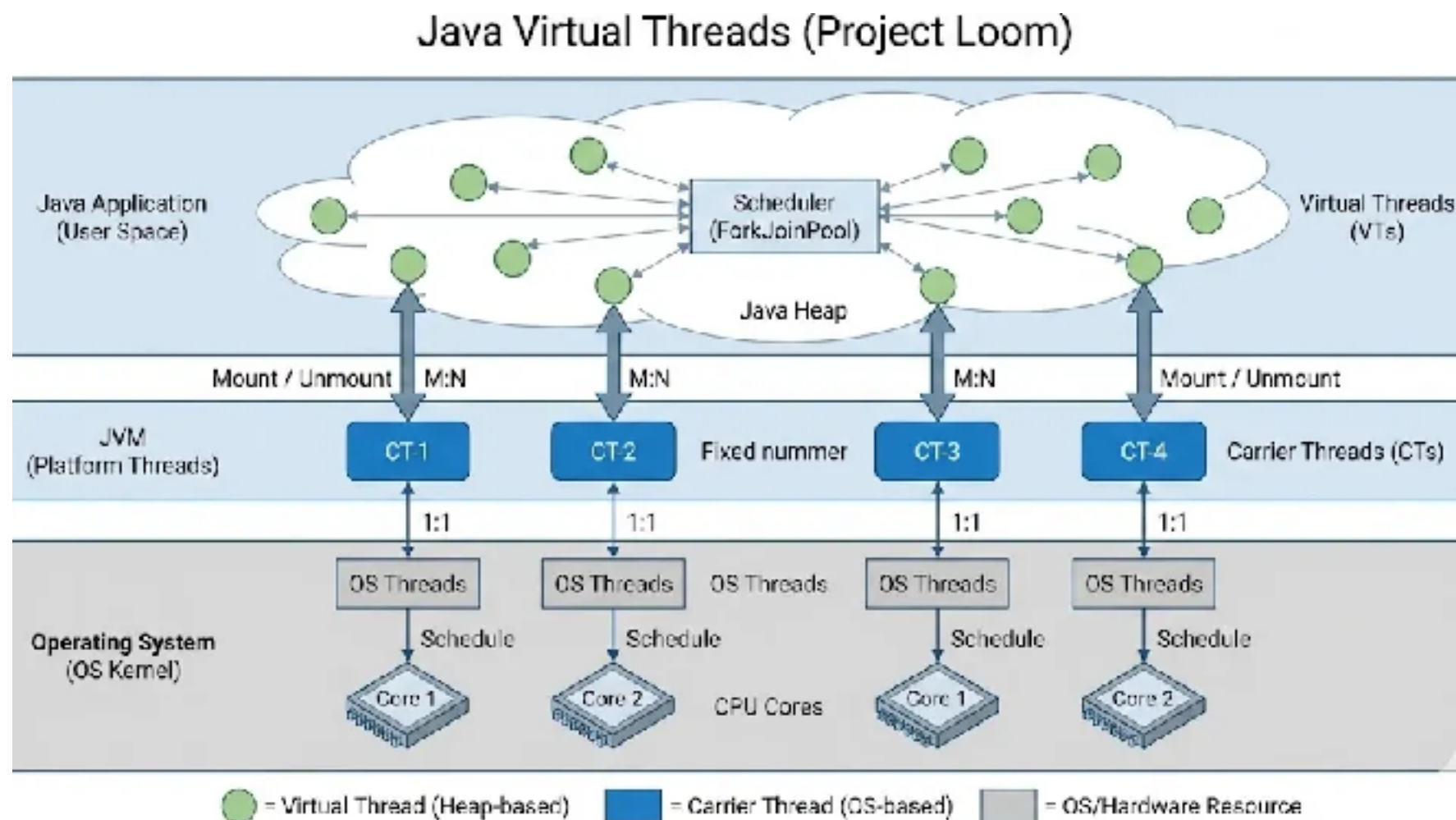
<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/stream/Stream.html>



Java Virtual Threads (Project Loom)

Java 21

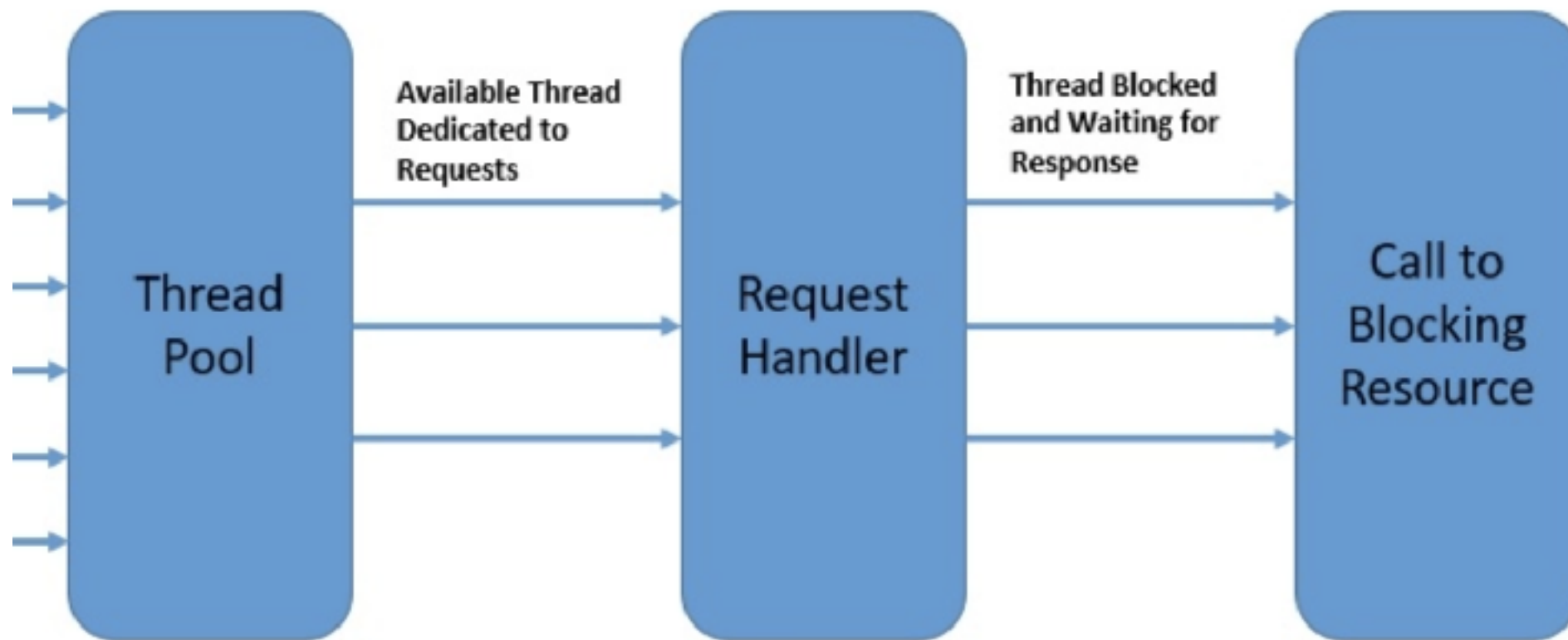
Lightweight, JVM-managed thread



<https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html>



Spring Boot with Virtual Thread !!



<https://www.somkiat.cc/virtual-thread-in-spring-boot-3/>

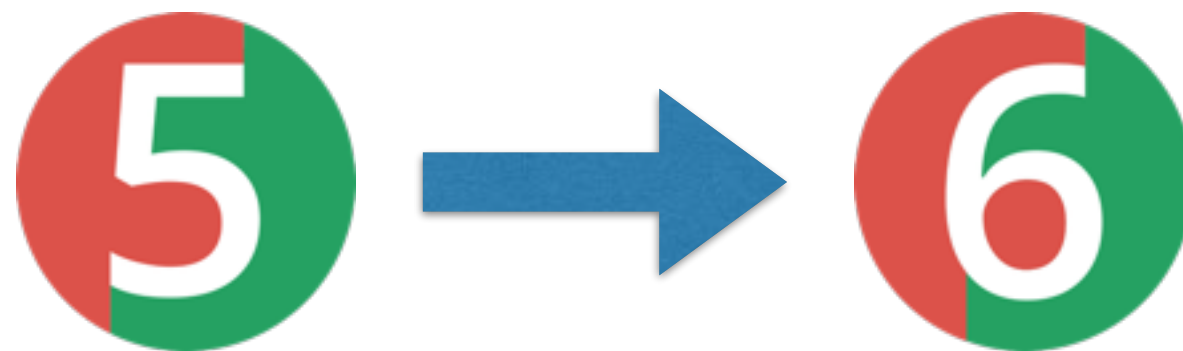


Platform vs Virtual Thread

Feature	Platform thread	Virtual thread (Java21+)
Mapping	1:1 with OS threads	M:N Many virtual threads to few OS threads
Creation cost	High memory and CPU	Low
Memory footprint	1 MB per thread (Stack size)	< 1 KB (Stored on heap)
Use case	CPU-intensive tasks	I/O intensive tasks (blocking call)
Scalability	Limit by OS resources	Scalable to millions
Pooling ?	Required to prevent resource exhaustion	Create a new one per task



Testing with JUnit



<https://junit.org/>



Good Testing ?

F.I.R.S.T + U

Fast

Isolate

Repeat

Self-verify

Timely

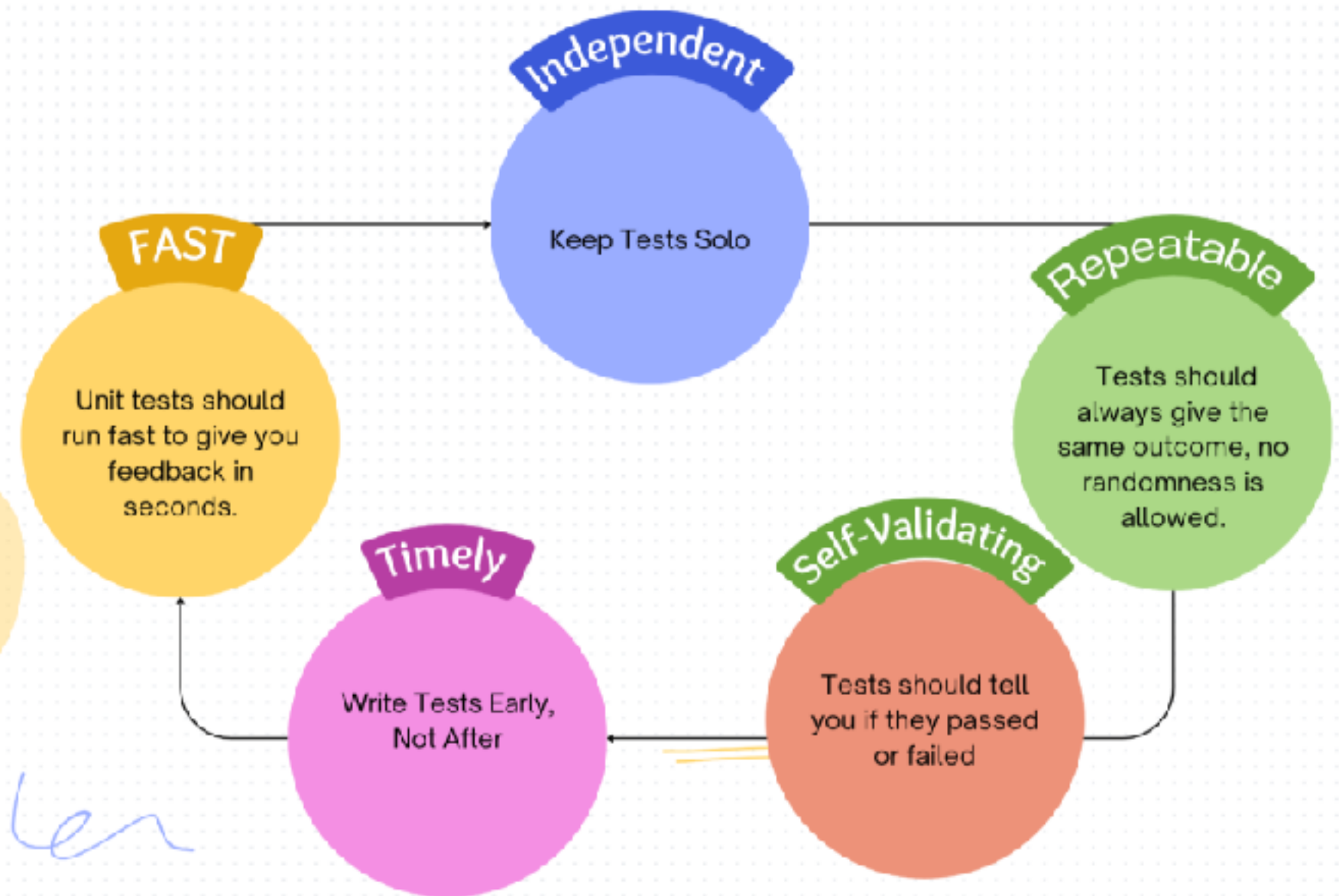
Understand



F.I.R.S.T

F.I.R.S.T

The FIRST Principles of Unit Testing



Test Case Structure

AAA (Arrange Act Assert)

```
public class HelloTest {  
  
    @Test  
    @DisplayName("ทดสอบอะไร มีขั้นตอนอย่างไร")  
    public void case01(){  
        // Arrange  
        Hello hello = new Hello();  
        // Act  
        String actualResult = hello.sayHi("java");  
        // Assert  
        assertNotNull(actualResult);  
        assertEquals("Hello java", actualResult);  
    }  
}
```



Tests Cases
=
Business Criteria
+
Examples (real data)



Test Early and Often ?

Test first
Test last
Test later



Test Early and Often ?

TEST-FIRST (TDD)

Think and write
test before coding

1. Think about
the feature



2. Write a test



3. Run the test



4. Write code to make
the test pass



5. Run the test
again



6. Refactor
(improve code)



Repeat steps 2–6 for next small increment

TEST-LAST

Think and coding
feature by feature
and write test

1. Think about
a feature



2. Write code
for the feature



3. Test the feature



4. Fix issues
if any



5. Write tests
for the feature



Repeat for next feature

TEST-LATER

Think, develop
many features and
write tests for all
feature in once

1. Think and plan
many features



2. Develop all
features



3. Write tests for
all features



4. Run all tests



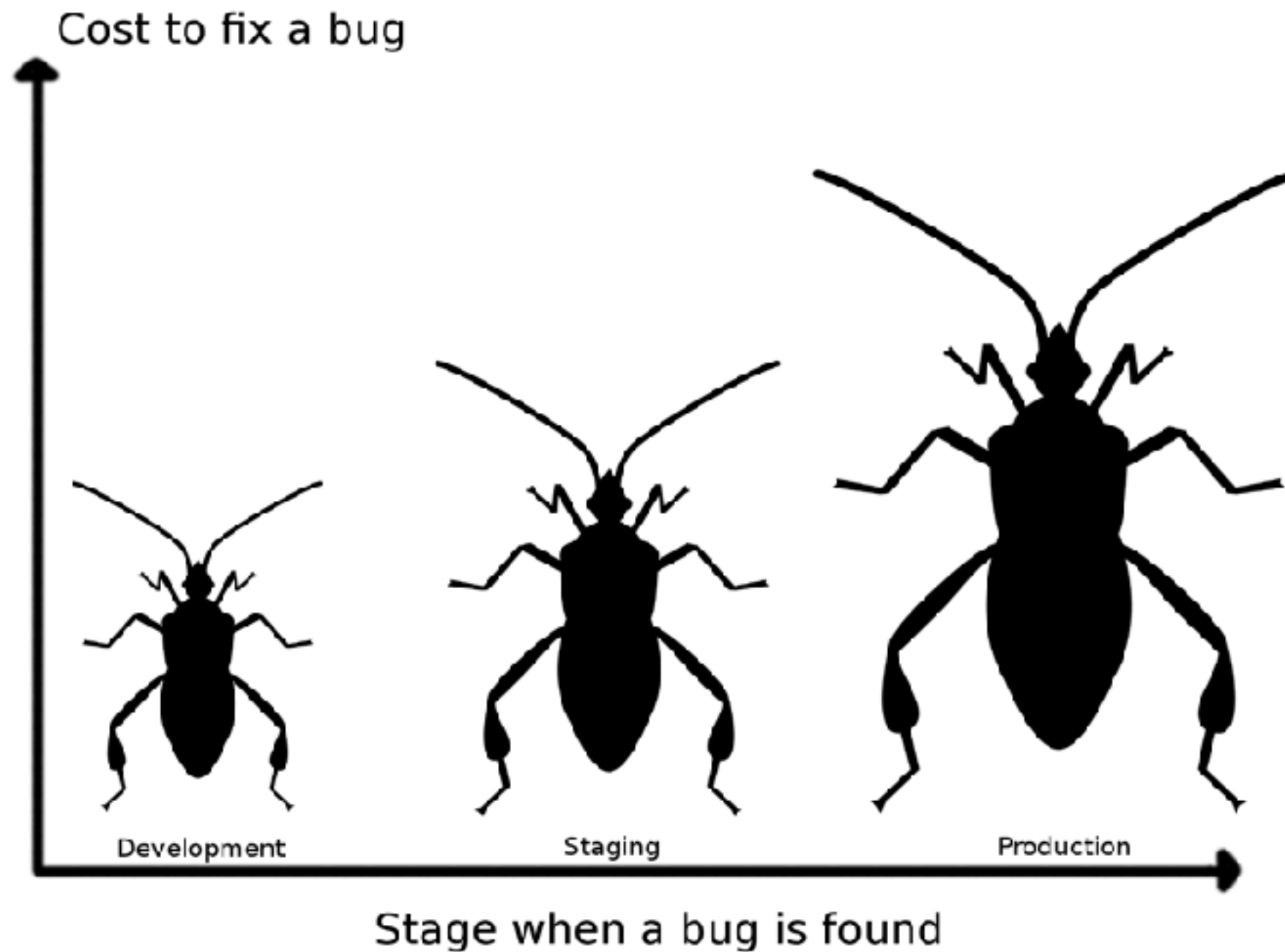
5. Fix issues



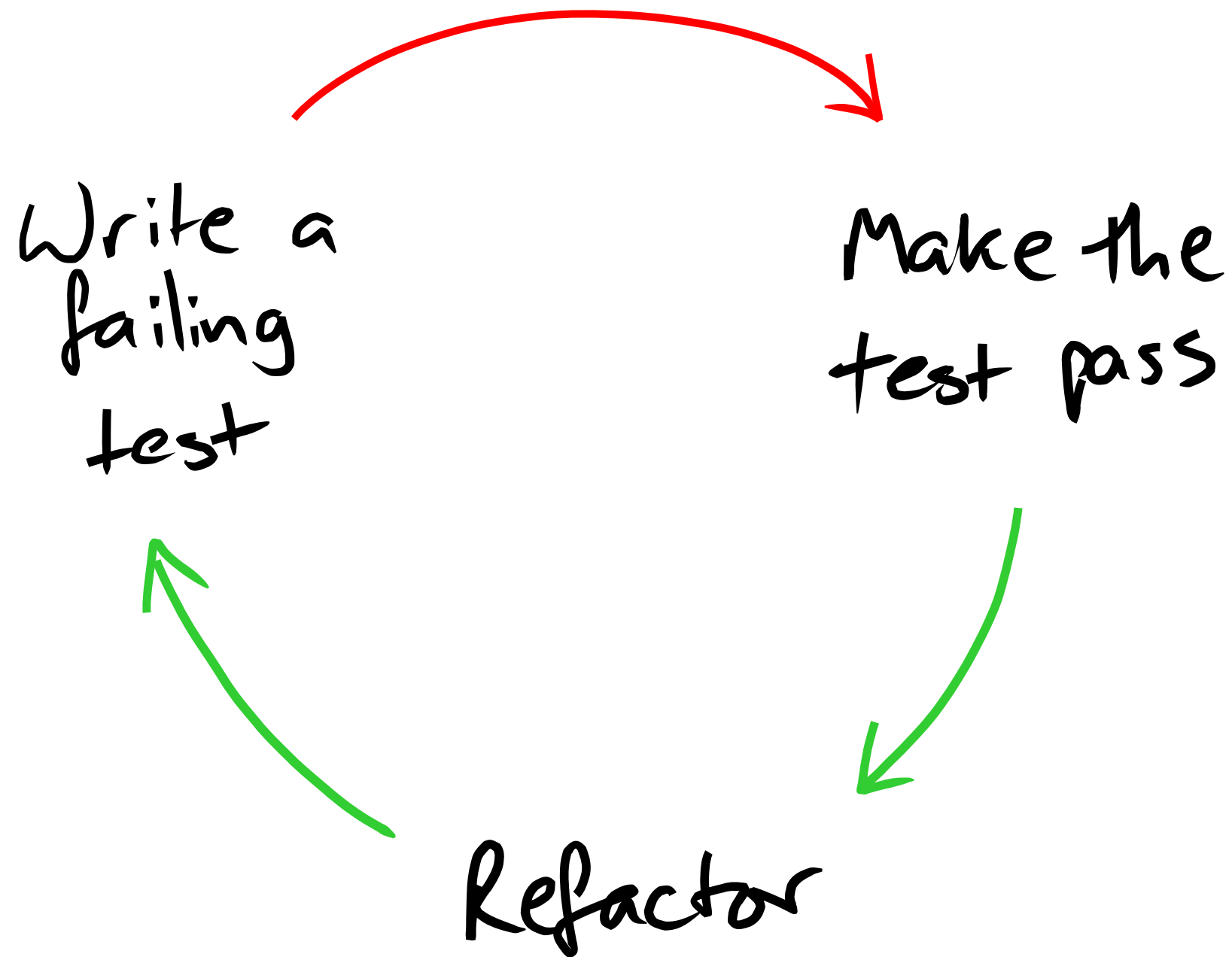
Tests written after development



Test Early and Often ?



Test-Driven Development



<https://growing-object-oriented-software.com/>



Think before Coding

Why & What ?

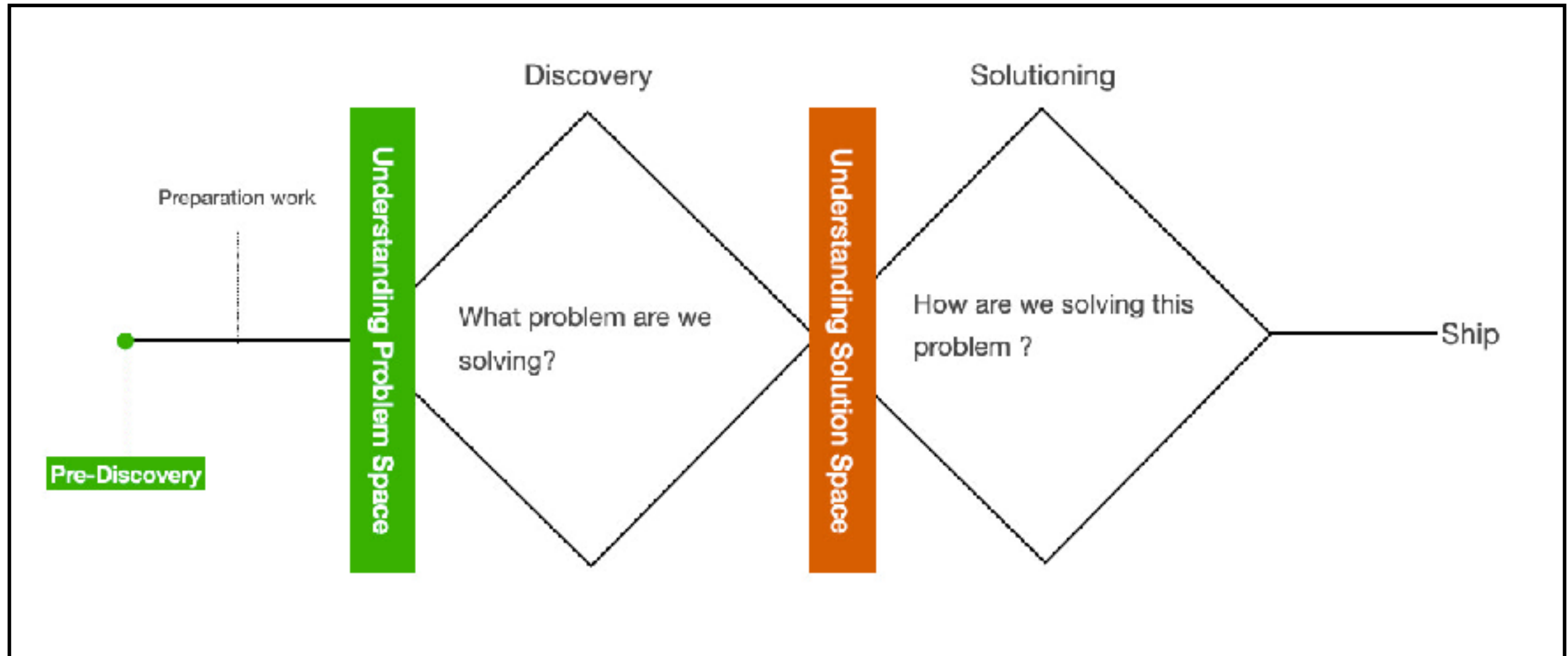
Write a
failing
test

Make the
test pass

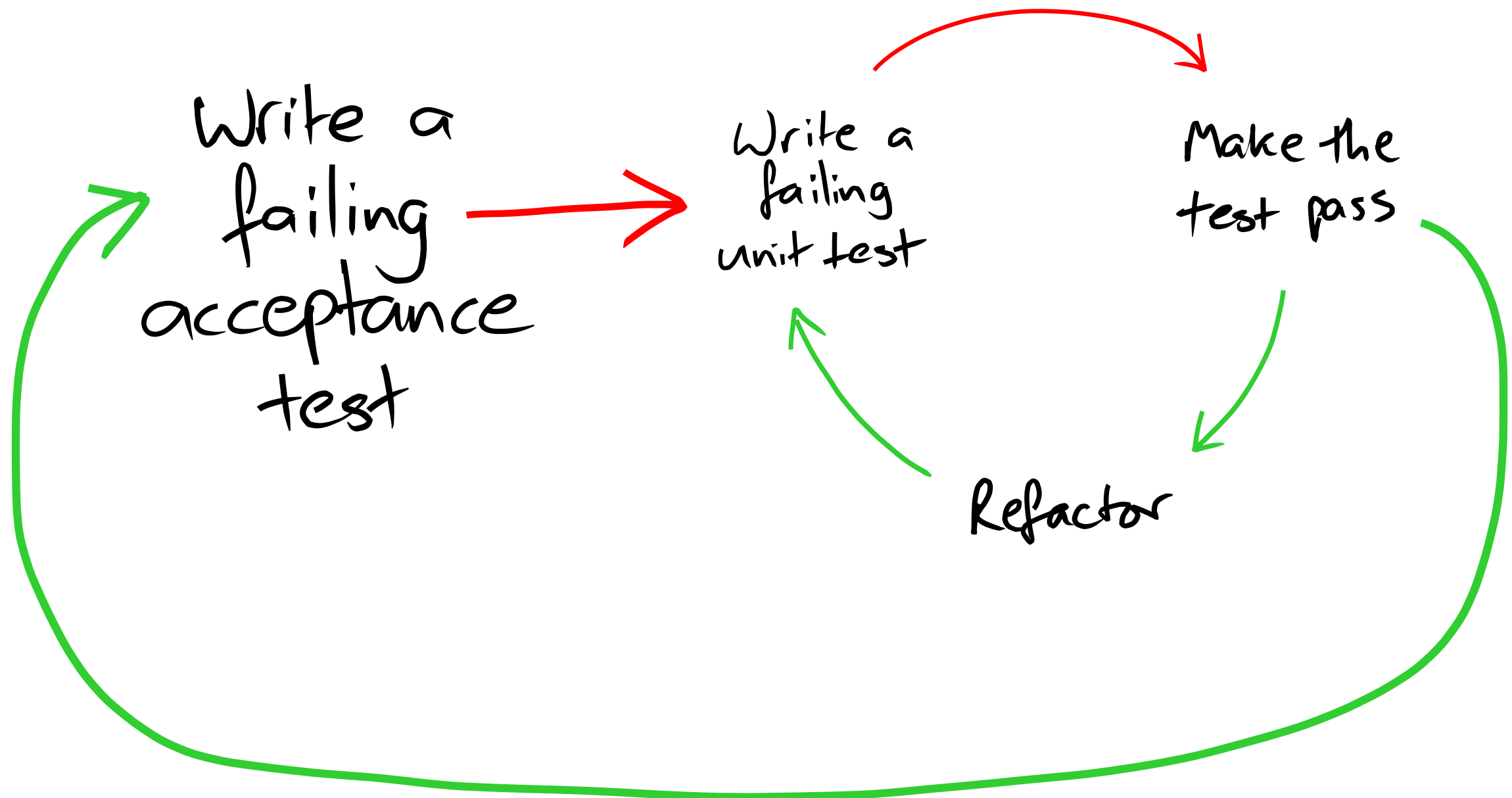
Refactor



Problem vs Solution



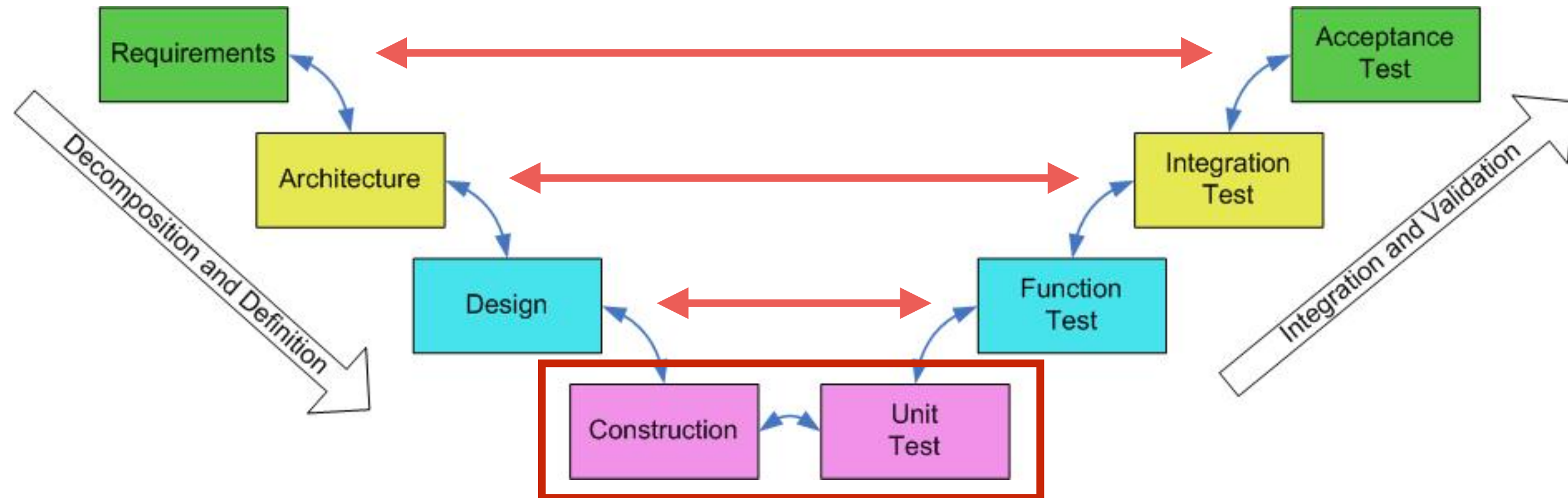
TDD with Acceptance test



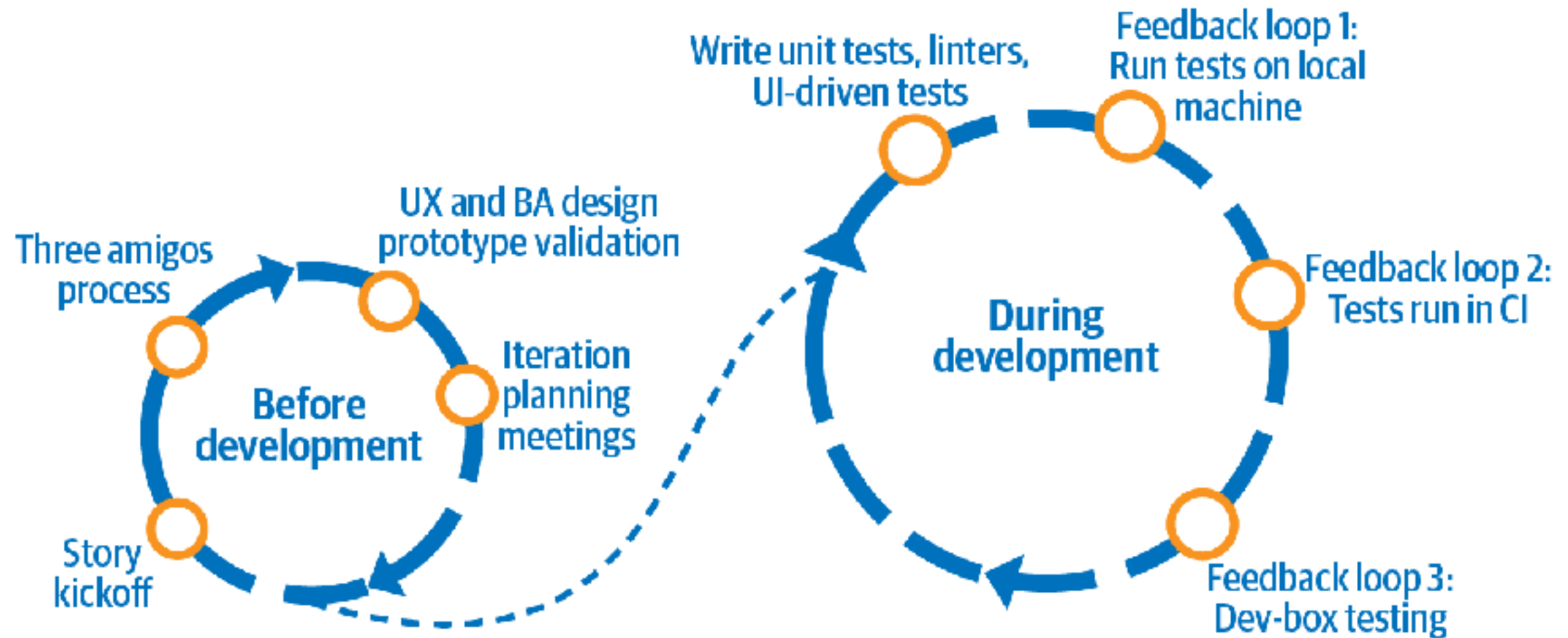
<https://growing-object-oriented-software.com/>



V Model



Quality Check ?

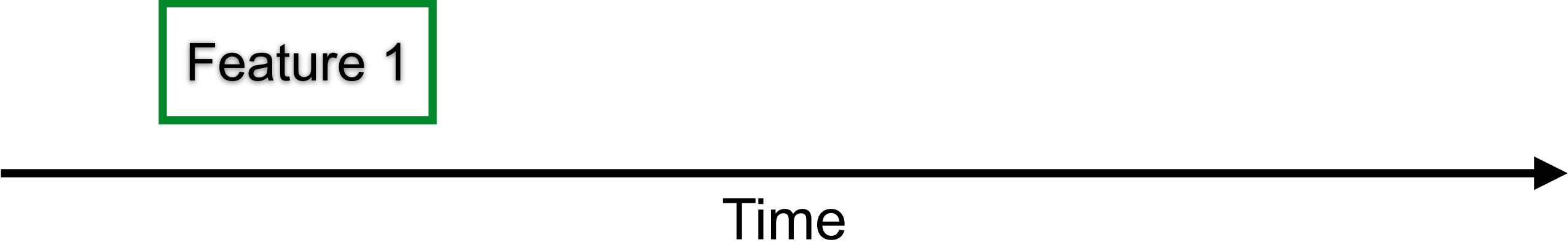


Iterative and Incremental process



Iterative and incremental process

Done = coded and tested



The diagram illustrates an iterative and incremental process. It features a horizontal black arrow pointing to the right, representing the progression of time. Above the arrow, on the left side, is a green rectangular box containing the text "Feature 1". Below the arrow, centered, is the word "Time".

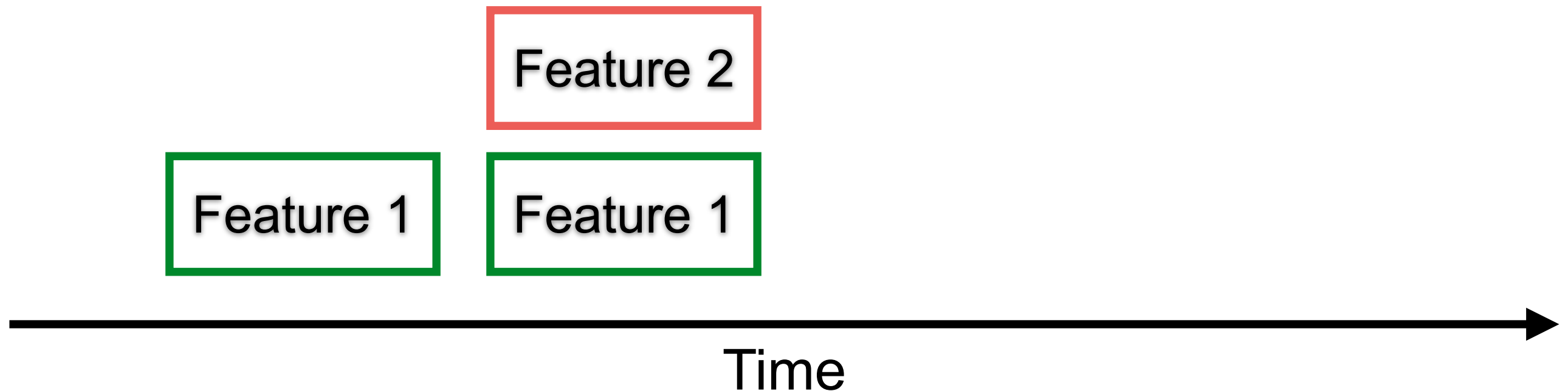
Feature 1

Time



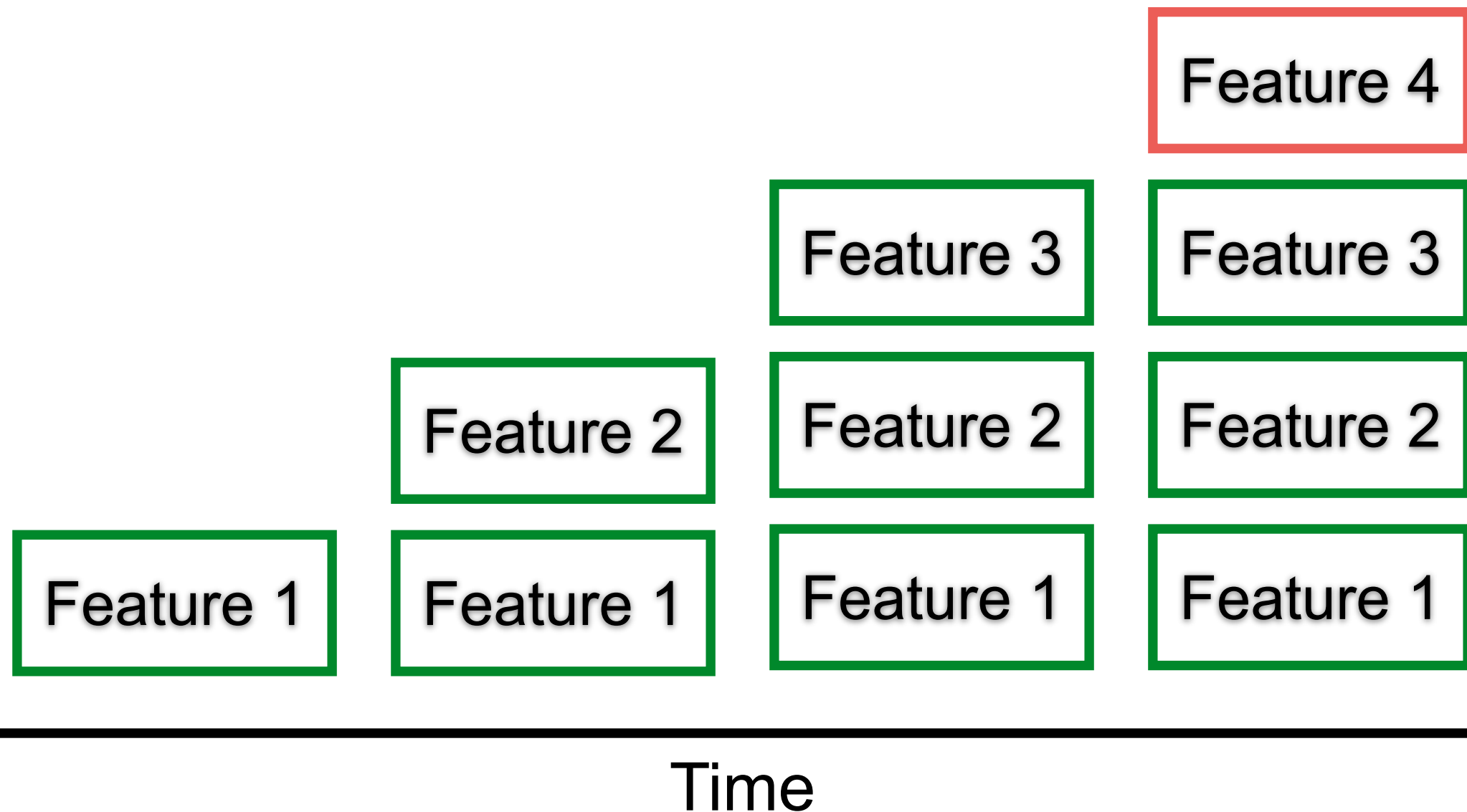
Iterative and incremental process

Done = coded and tested



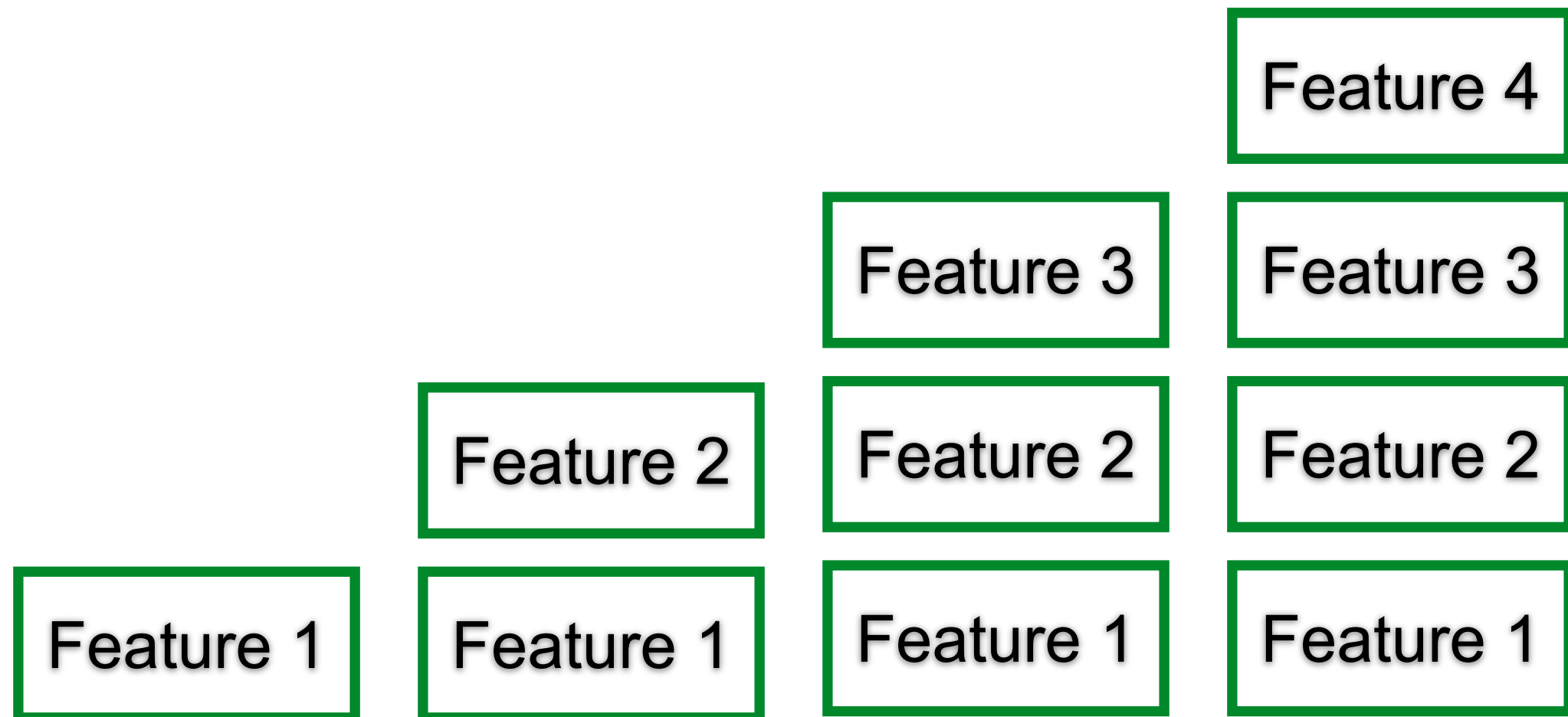
Iterative and incremental process

Done = coded and tested



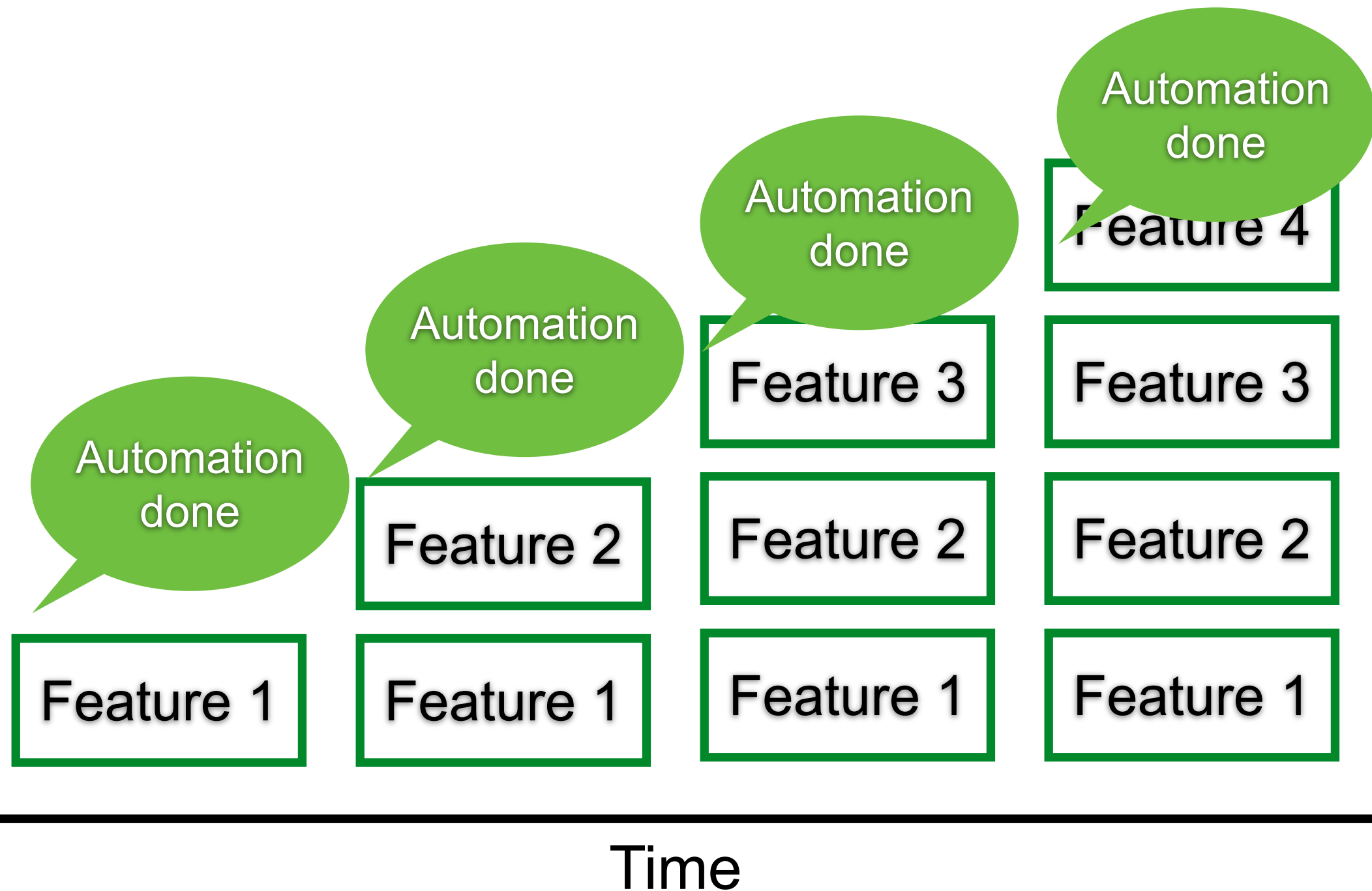
Iterative and incremental process

Done = coded and tested



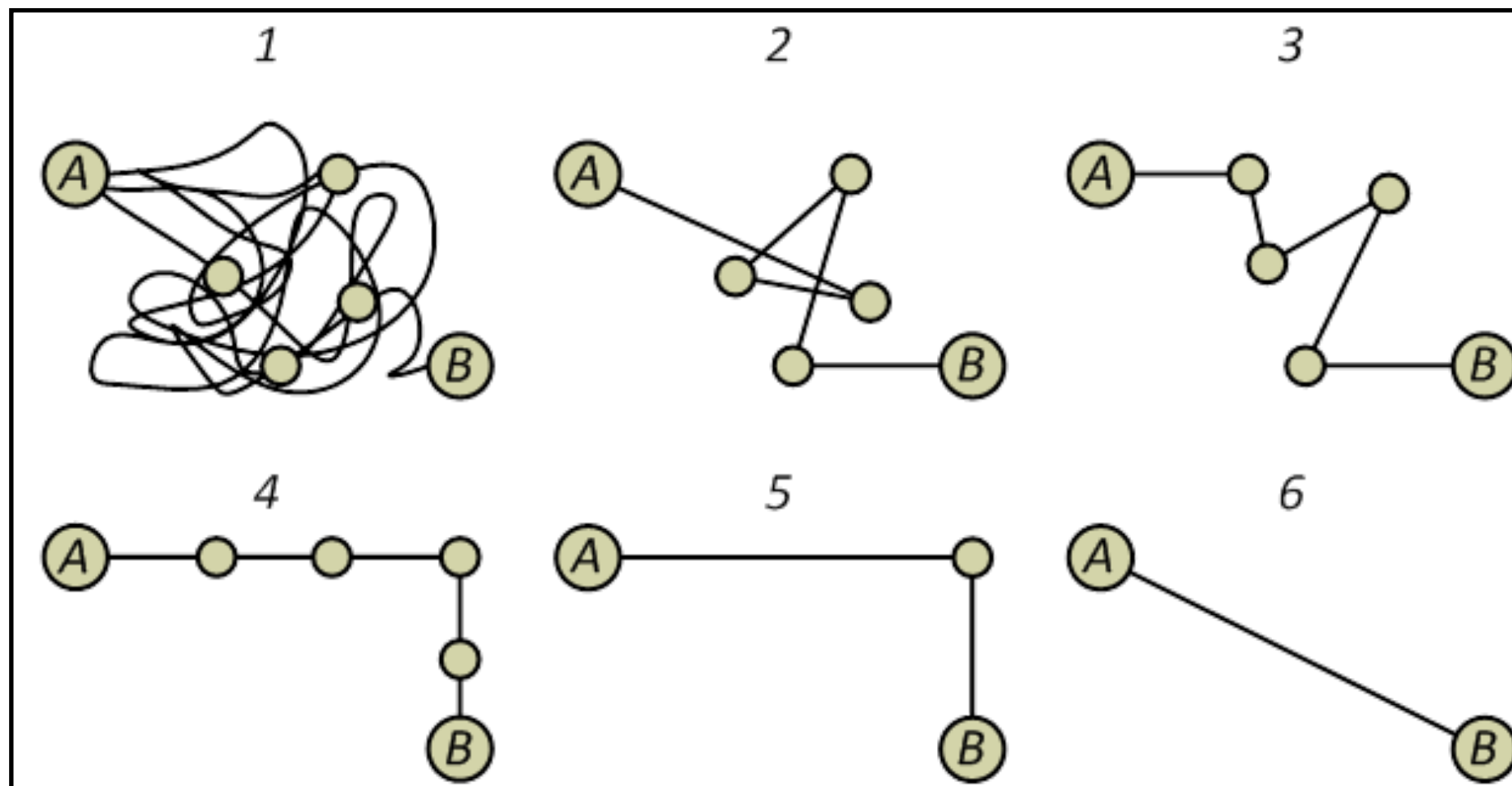
Iterative and incremental process

Done = coded and tested



Refactor ?

Change the structure of code without change the behaviour of code

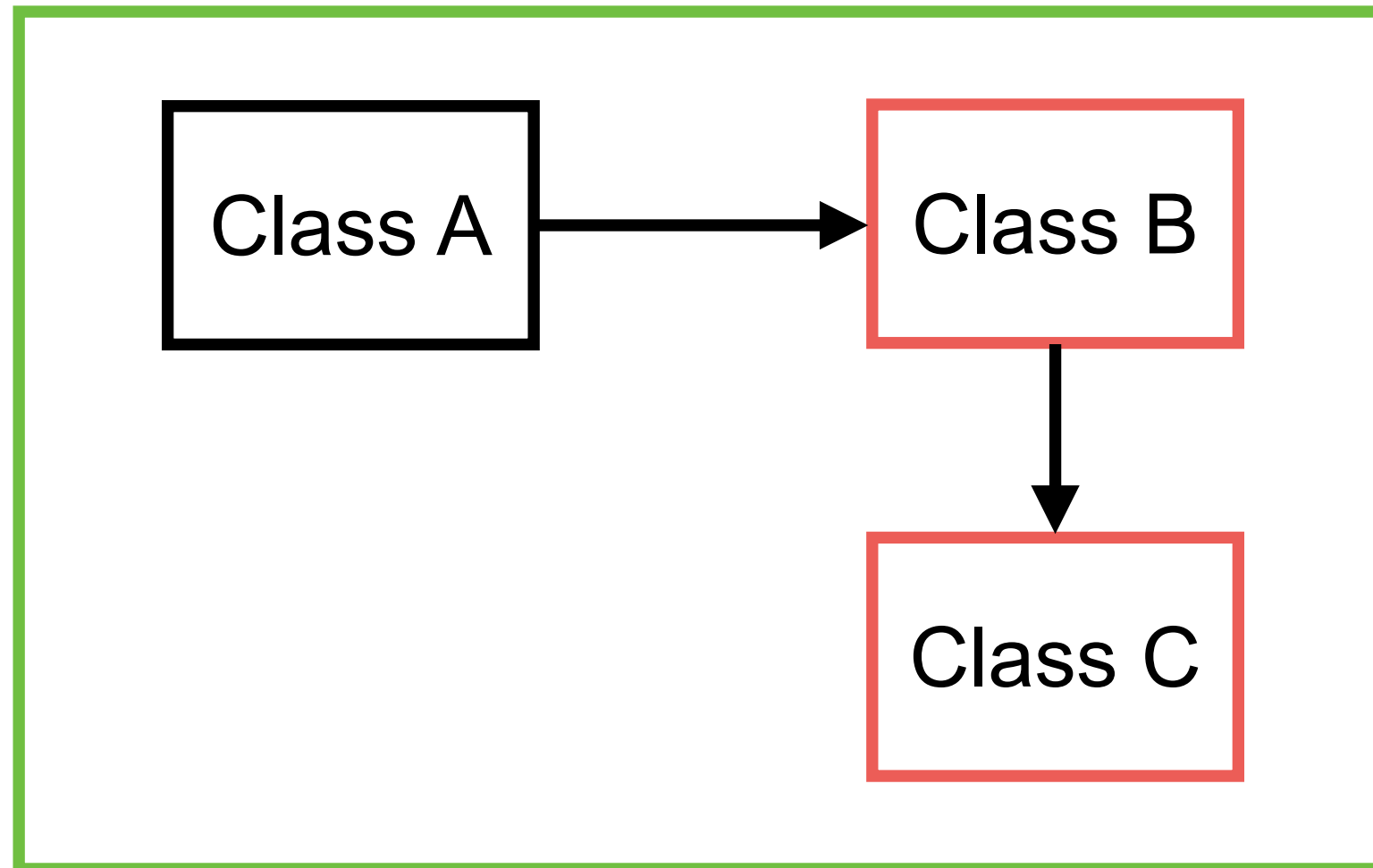


Working with dependencies !!



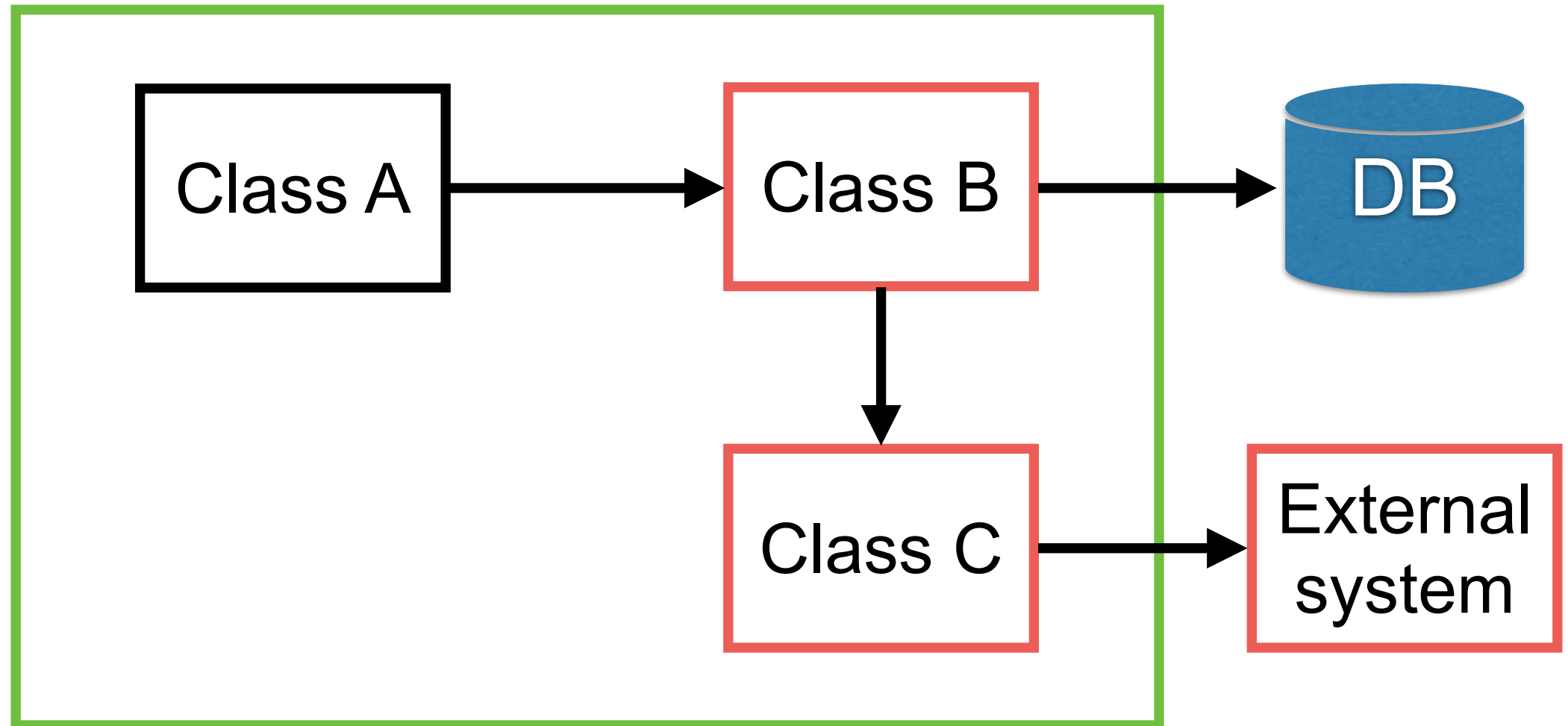
Working with dependencies !!

Internal dependencies



Working with dependencies !!

External dependencies



Test Double

Dummy

Stub

Spy

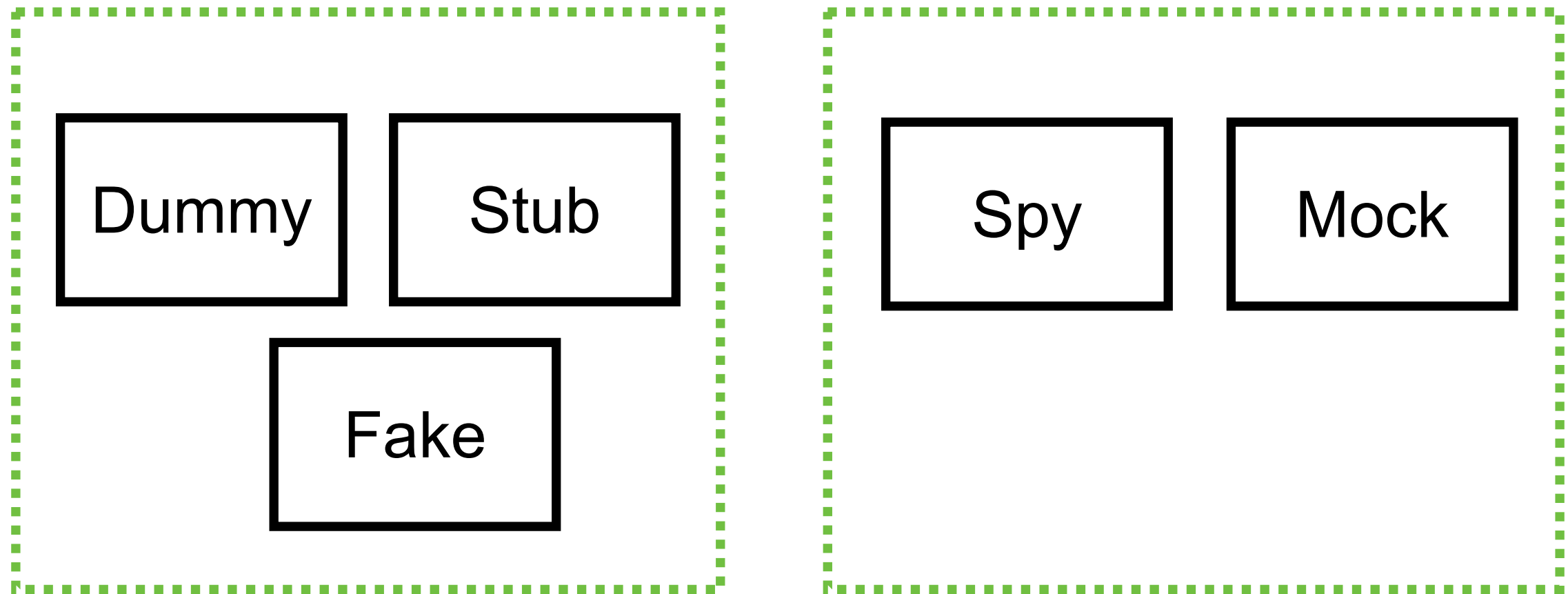
Mock

Fake

<http://xunitpatterns.com/Test%20Double.html>



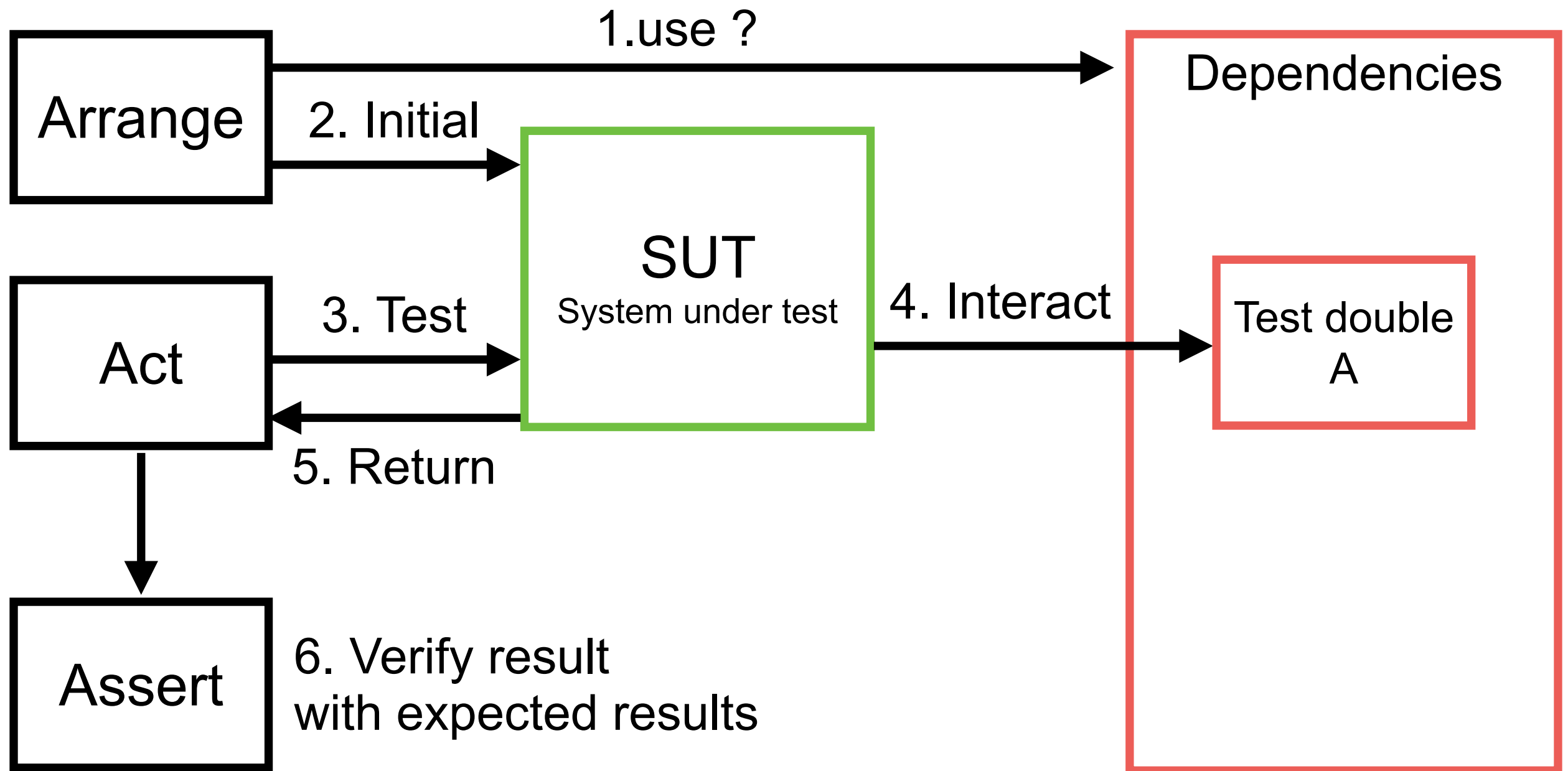
Test Double



<http://xunitpatterns.com/Test%20Double.html>



Flow of testing ?



Tools and Frameworks



Demo code with Mockito



Unit 5 + Mockito (1)

```
public class Hello {  
  
    private final Random random;  
  
    public Hello(Random random) {  
        this.random = random;  
    }  
  
    public String sayHiWithDependency(String name) {  
        int number = random.nextInt(10);  
        return "Hello " + name + number;  
    }  
  
}
```



Unit 5 + Mockito (2)

Find dependency in your code

```
public class Hello {  
    private final Random random;  
  
    public Hello(Random random) {  
        this.random = random;  
    }  
  
    public String sayHiWithDependency(String name) {  
        int number = random.nextInt(10);  
        return "Hello " + name + number;  
    }  
}
```



Unit 5 + Mockito (3)

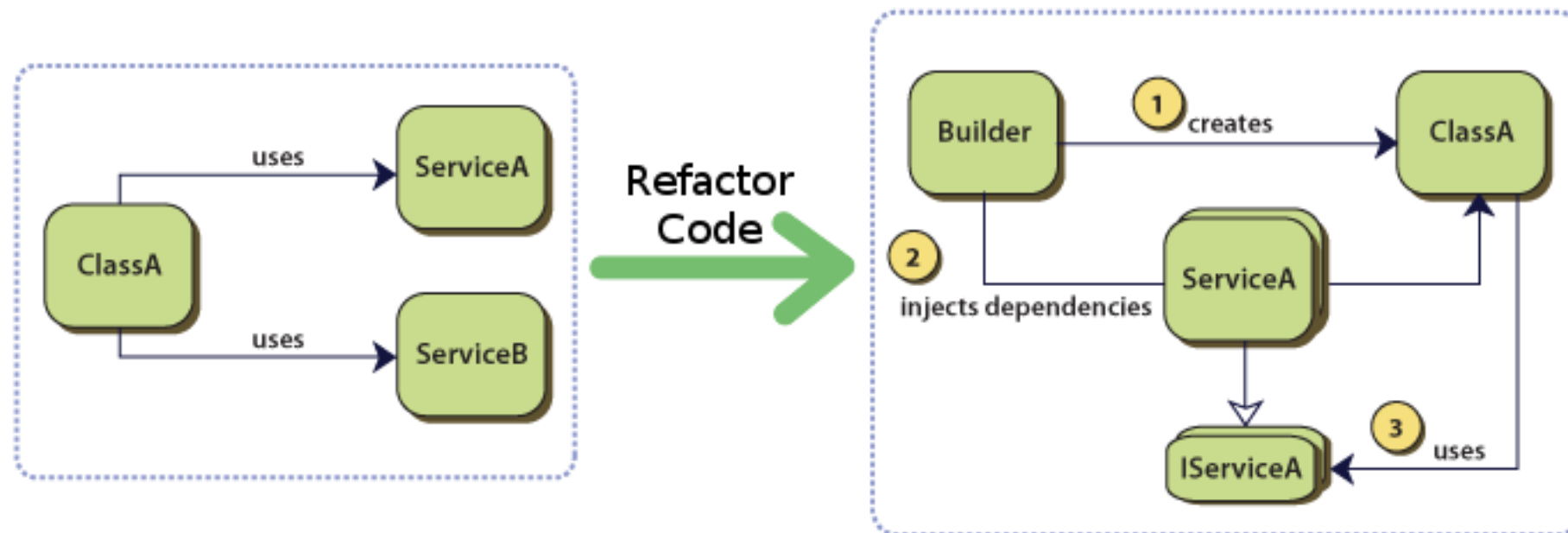
How to inject dependency to your code ?

```
public class Hello {  
  
    private final Random random;  
  
    public Hello(Random random) {  
        this.random = random;  
    }  
  
    public String sayHiWithDependency(String name) {  
        int number = random.nextInt(10);  
        return "Hello " + name + number;  
    }  
  
}
```



Types of Dependency Injection

Field/Property injection
Constructor injection
Method injection
Interface injection



Unit 5 + Mockito (4)

```
@ExtendWith(MockitoExtension.class)
public class HelloTest {

    @Mock
    private Random random;

    @InjectMocks
    private Hello hello;

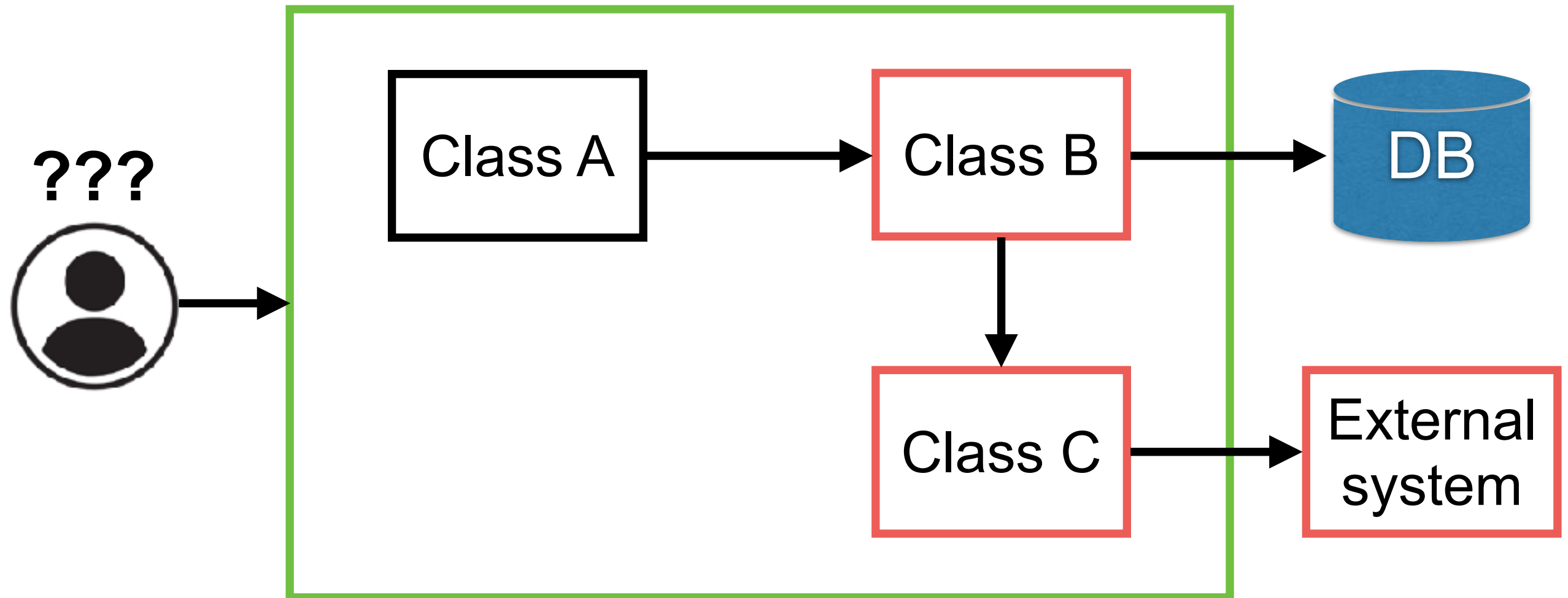
    @Test
    public void case02(){
        // Arrange
        when(random.nextInt(anyInt())).thenReturn(5);
        // Act
        String actualResult = hello.sayHiWithDependency("java");
        // Assert
        assertEquals("Hello java5", actualResult);
    }
}
```



Types of Testing ?



Types of Testing ?



Types of Testing ?

End-to-End testing

Integration testing

Service component testing

Contract testing

Unit testing



Respect your tests

Don't ignore it if it fails
Fix the code or fix the test
Always refactor or improve

**100% of regression tests
must pass all the time**



REST API with Spring Boot

<https://spring.io/>





Spring Boot

Version 3 => 4 ?



Spring Initializr



Project
☐ Gradle - Groovy
☐ Gradle - Kotlin
☒ Maven

Language
☒ Java
☐ Kotlin
☐ Groovy

Spring Boot
☐ 4.1.0 (SNAPSHOT) ☐ 4.1.0 (RC1) ☐ 4.0.7 (SNAPSHOT)
☒ 4.0.6 ☐ 3.5.15 (SNAPSHOT) ☐ 3.5.14

Project Metadata

Group

com.example

Artifact

demo

Package name

com.example.demo

Packaging

☒ Jar ☐ War

Configuration

☒ Properties ☐ YAML

Java

☐ 28 ☒ 25 ☐ 21 ☐ 17

Dependencies

ADD DEPENDENCIES... ⌕ + B

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

GraalVM Native Support DEVELOPER TOOLS
Support for compiling Spring applications to native executables using the GraalVM native-image compiler.

Spring Data JPA SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

<https://start.spring.io/>



© 2022 - 2026 Siam Chamnankit Company Limited. All rights reserved.

Workshop

107

Spring Boot 4

Spring Framework 7

<https://spring.io/projects/release-highlights>



Changes and Recommendation

Java 17+

Java 21 + Virtual Thread

Build-in resilience (retry, concurrency limit)

Native API versioning

Null-safety with JSpecify

Modularization

Faster and smart testing lifecycle

<https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide>



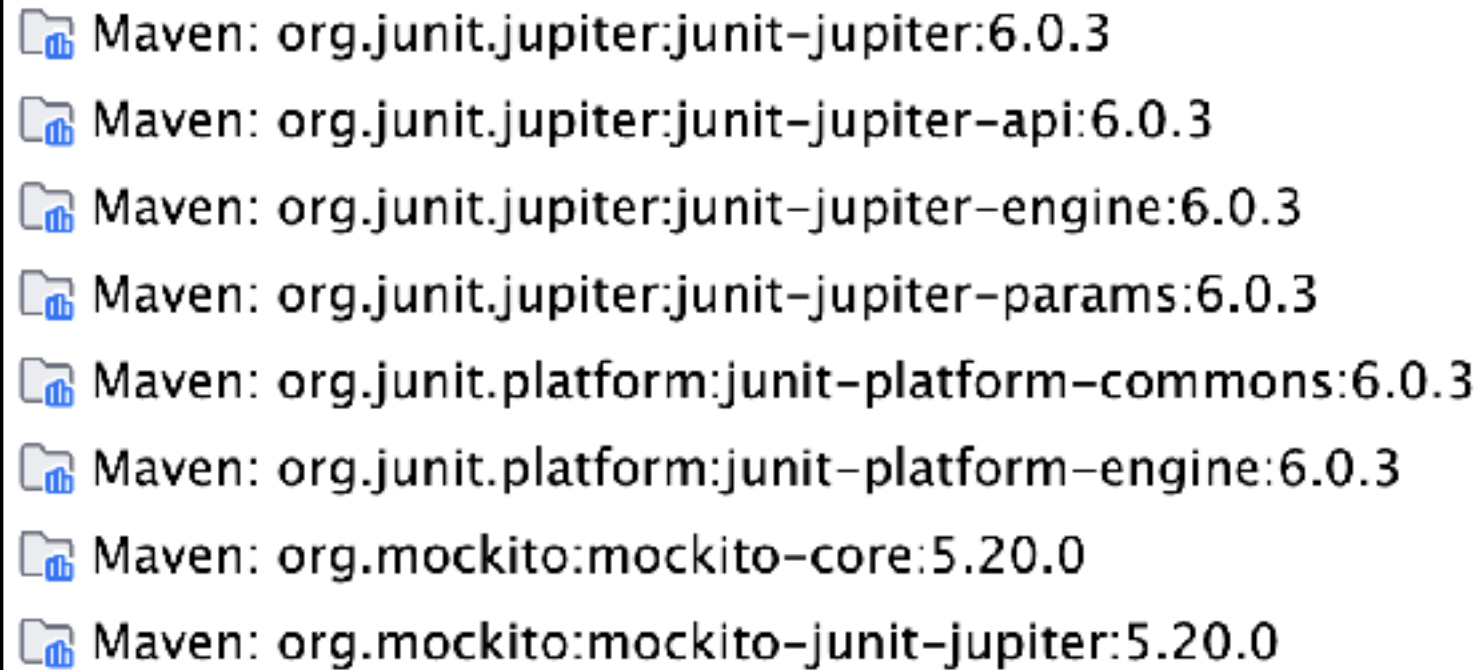
Testing Module

Technology	Main dependency	Test dependency
RestClient RestTemplate	spring-boot-starter-restclient	spring-boot-starter-restclient-test
WebClient (Reactive)	spring-boot-starter-webclient	spring-boot-starter-webclient-test
Spring Web MVC	spring-boot-starter-webmvc	spring-boot-starter-webmvc-test
Spring Data JPA	spring-boot-starter-data-jpa	spring-boot-starter-data-jpa-test

<https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide>



Testing with Spring Boot (1)



A screenshot of a Maven dependency list, enclosed in a black rectangular border. It lists eight dependencies, each preceded by a small folder icon with a blue bar chart. The dependencies are:

- Maven: org.junit.jupiter:junit-jupiter:6.0.3
- Maven: org.junit.jupiter:junit-jupiter-api:6.0.3
- Maven: org.junit.jupiter:junit-jupiter-engine:6.0.3
- Maven: org.junit.jupiter:junit-jupiter-params:6.0.3
- Maven: org.junit.platform:junit-platform-commons:6.0.3
- Maven: org.junit.platform:junit-platform-engine:6.0.3
- Maven: org.mockito:mockito-core:5.20.0
- Maven: org.mockito:mockito-junit-jupiter:5.20.0

<https://docs.spring.io/spring-boot/reference/testing/index.html>



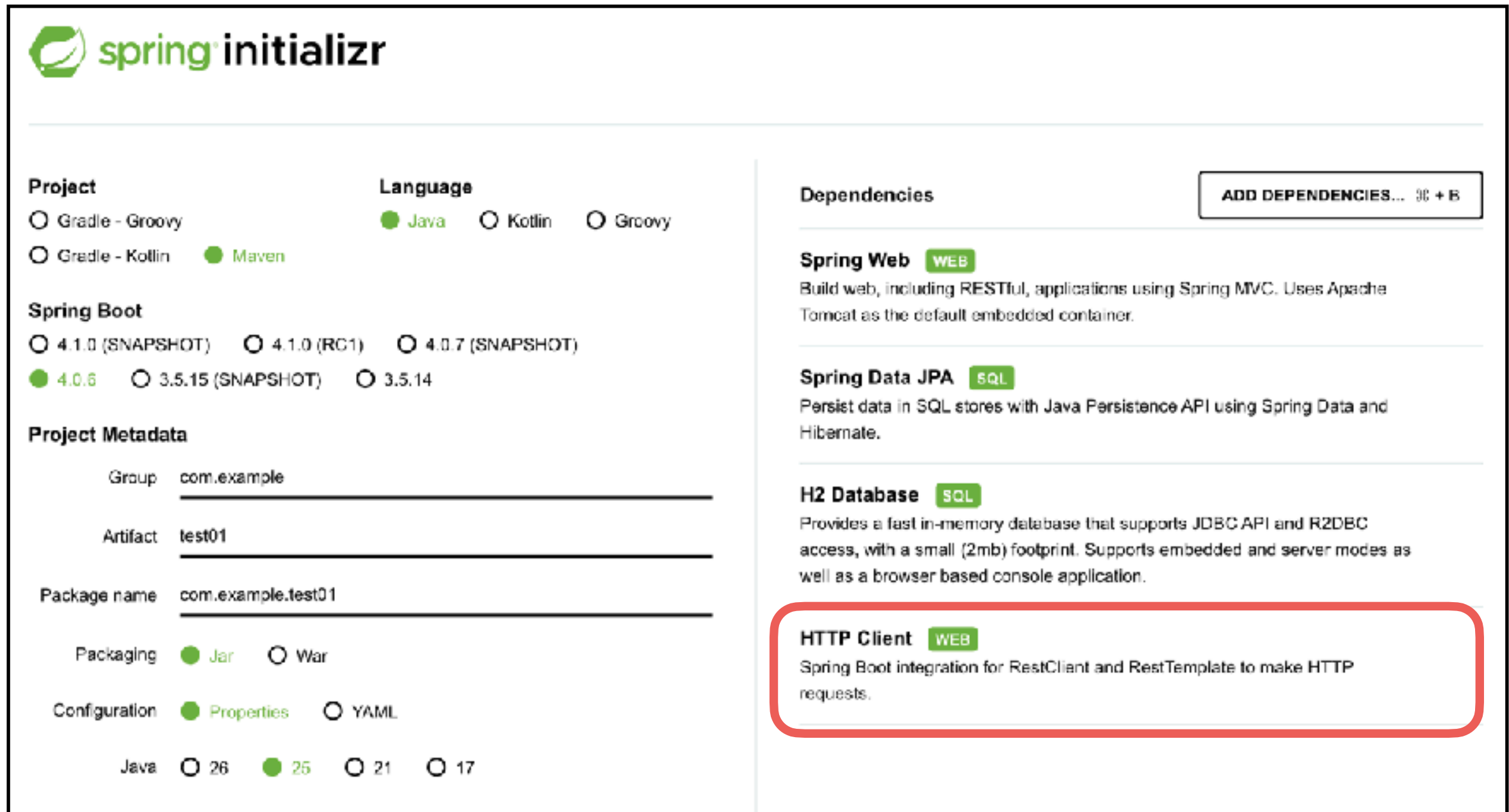
Testing with Spring Boot (2)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>
```

<https://docs.spring.io/spring-boot/reference/testing/index.html>



REST Template with HttpClient



The image shows the Spring Initializr web interface for configuring a new project. The interface is divided into several sections:

- Project:** Includes radio buttons for `Gradle - Groovy`, `Gradle - Kotlin`, and `Maven` (which is selected).
- Language:** Includes radio buttons for `Java` (selected), `Kotlin`, and `Groovy`.
- Spring Boot:** Includes radio buttons for various versions: `4.1.0 (SNAPSHOT)`, `4.1.0 (RC1)`, `4.0.7 (SNAPSHOT)`, `4.0.6` (selected), `3.5.15 (SNAPSHOT)`, and `3.5.14`.
- Project Metadata:** Includes text input fields for `Group` (com.example), `Artifact` (test01), and `Package name` (com.example.test01).
- Packaging:** Includes radio buttons for `Jar` (selected) and `War`.
- Configuration:** Includes radio buttons for `Properties` (selected) and `YAML`.
- Java:** Includes radio buttons for versions `26`, `25` (selected), `21`, and `17`.
- Dependencies:** A section on the right with a button `ADD DEPENDENCIES... 96 + 6`. It lists several dependencies:
 - Spring Web** (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
 - Spring Data JPA** (SQL): Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
 - H2 Database** (SQL): Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.
 - HTTP Client** (WEB): Spring Boot integration for RestClient and RestTemplate to make HTTP requests. This dependency is highlighted with a red rounded rectangle.

<https://docs.spring.io/spring-boot/reference/testing/index.html>



Pom.xml for HTTP Client

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-restclient</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-restclient-test</artifactId>
  <scope>test</scope>
</dependency>
```



Example in Spring Boot 4

TestRestTemplate

```
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@AutoConfigureTestRestTemplate
class HelloControllerTest {

    @Autowired
    private TestRestTemplate testRestTemplate;

    @Test
    void case01() {
        // Act
        String input = "java";
        String actualResult
            = testRestTemplate.getForObject("/api/hello/" + input,
                                           String.class);

        // Assert
        assertEquals("Hello java", actualResult);
    }
}
```



Example in Spring Boot 4

RestTestClient in Spring framework 7

```
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@AutoConfigureRestTestClient
class HelloController2Test {

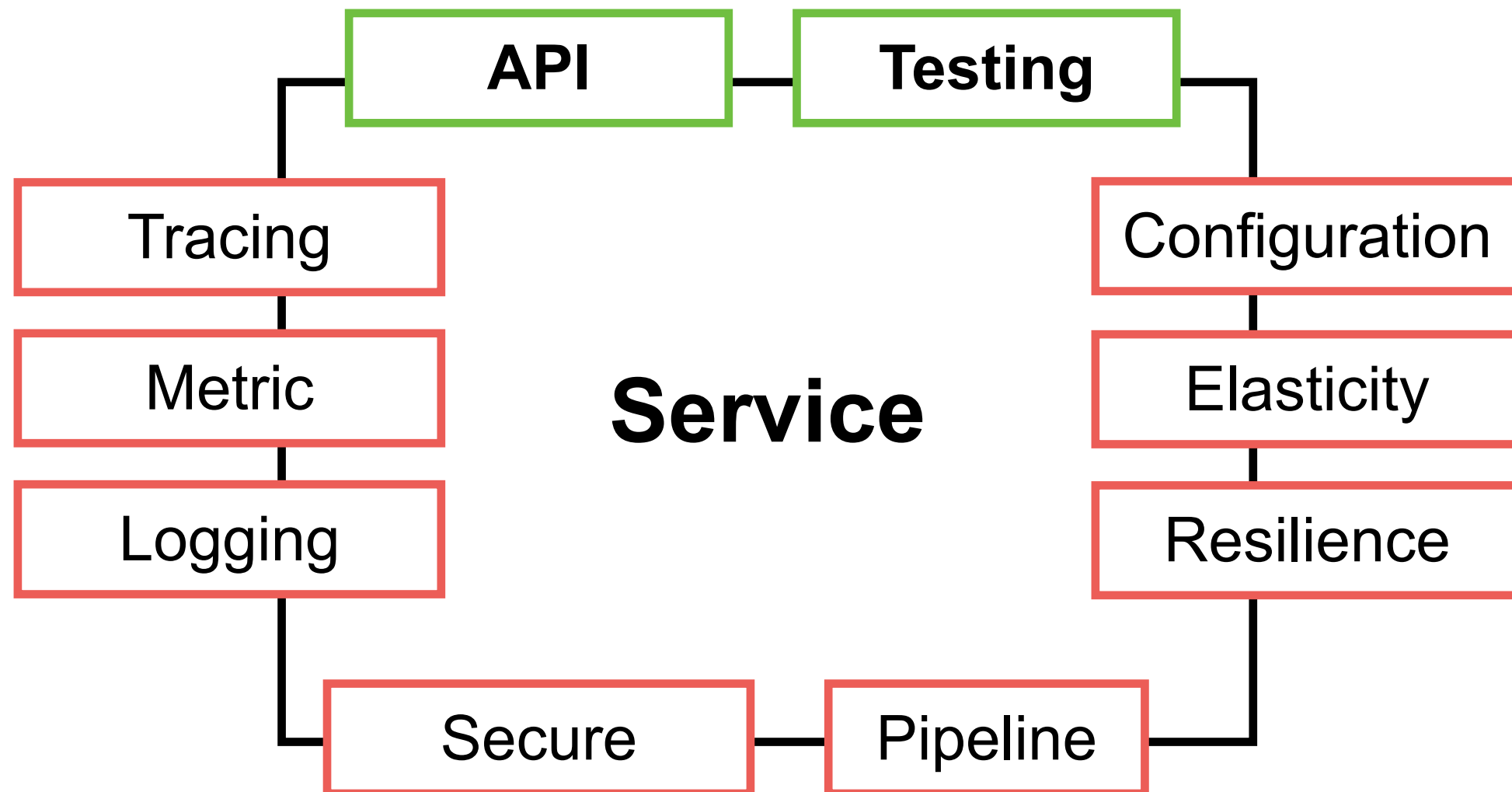
    @Autowired
    private RestTestClient restTestClient;

    @Test
    void case01() {
        String input = "java";
        restTestClient.get().uri("/api/hello/" + input)
            .exchange()
            .expectStatus().is2xxSuccessful()
            .expectBody(String.class).isEqualTo("Hello java");
    }
}
```

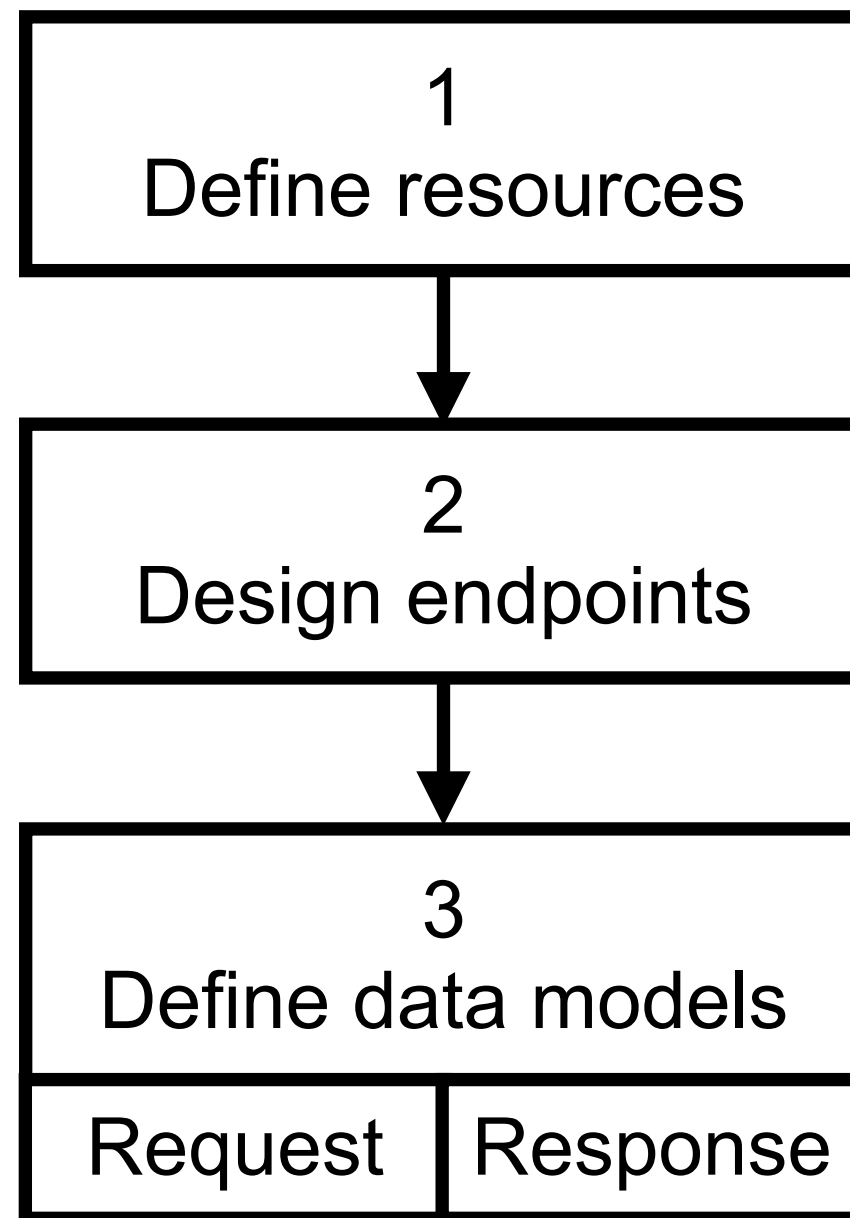
<https://docs.spring.io/spring-framework/reference/testing/resttestclient.html>



Properties of Service



Steps to Design APIs



Implementation

Coding

Testing

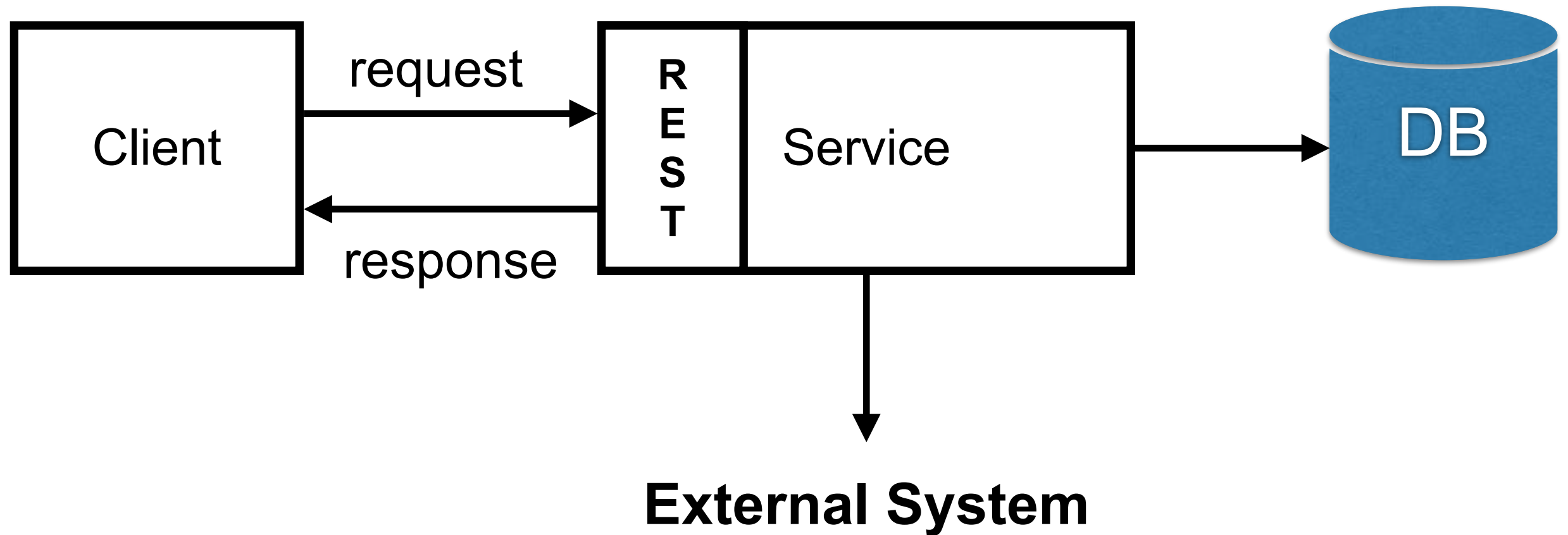
Observability

Performance

Security



Structure of Service



Spring Boot Structure

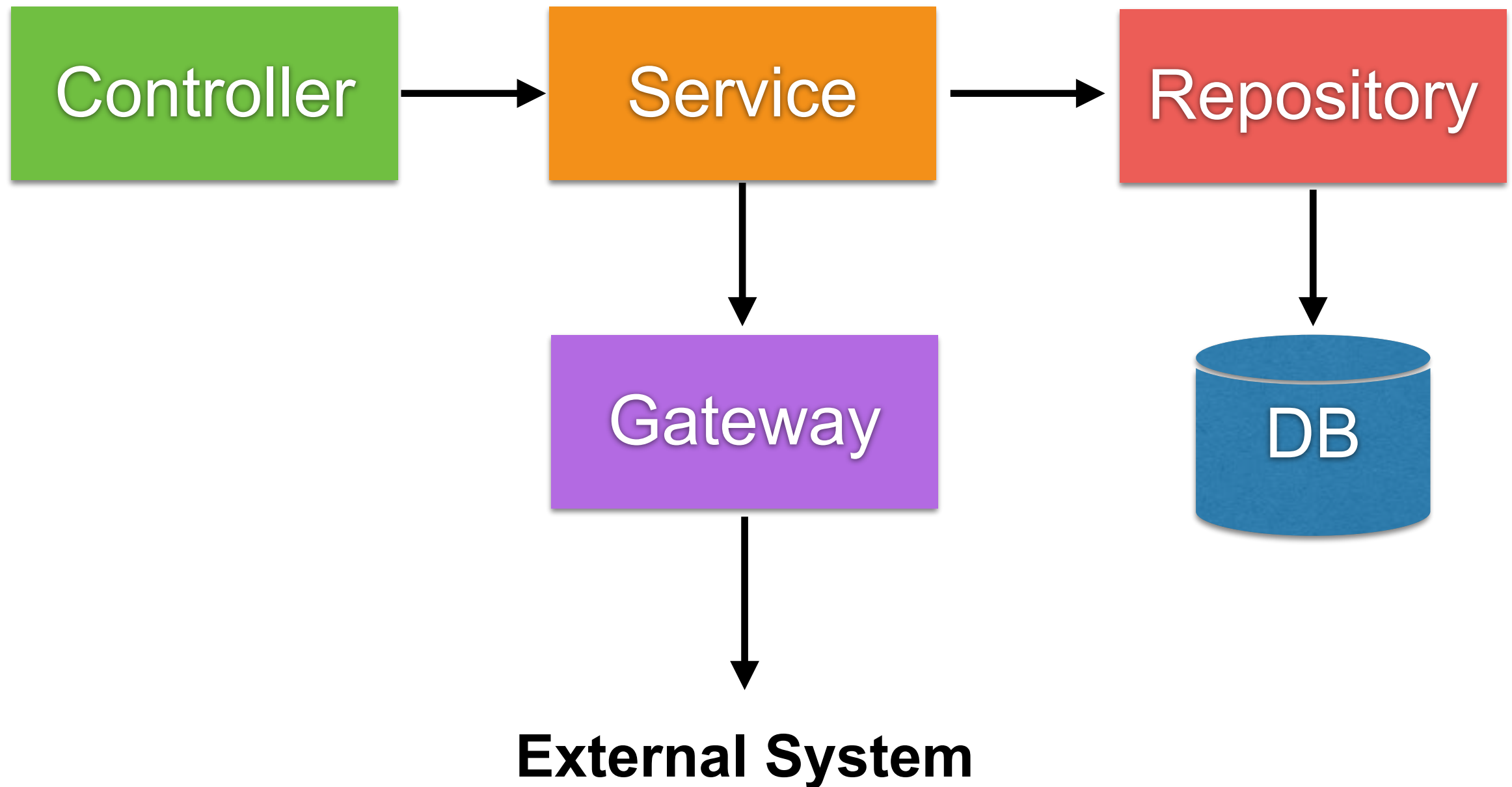
Separate by function/domain/feature

```
feature1
|
|   - controller
|   - service
|   - repository
feature2
|
|   - controller
|   - service
|   - repositor
```

<https://docs.spring.io/spring-boot/reference/using/structuring-your-code.html>



Spring Boot Structure



Controller

Request and Response

Input validation

Delegate to others classes such as service and repository

Generate response to client



Service

Control the flow of main business logics

Manage database transaction

Don't reuse service



Repository

Manage data in data store such as RDBMS and NoSQL



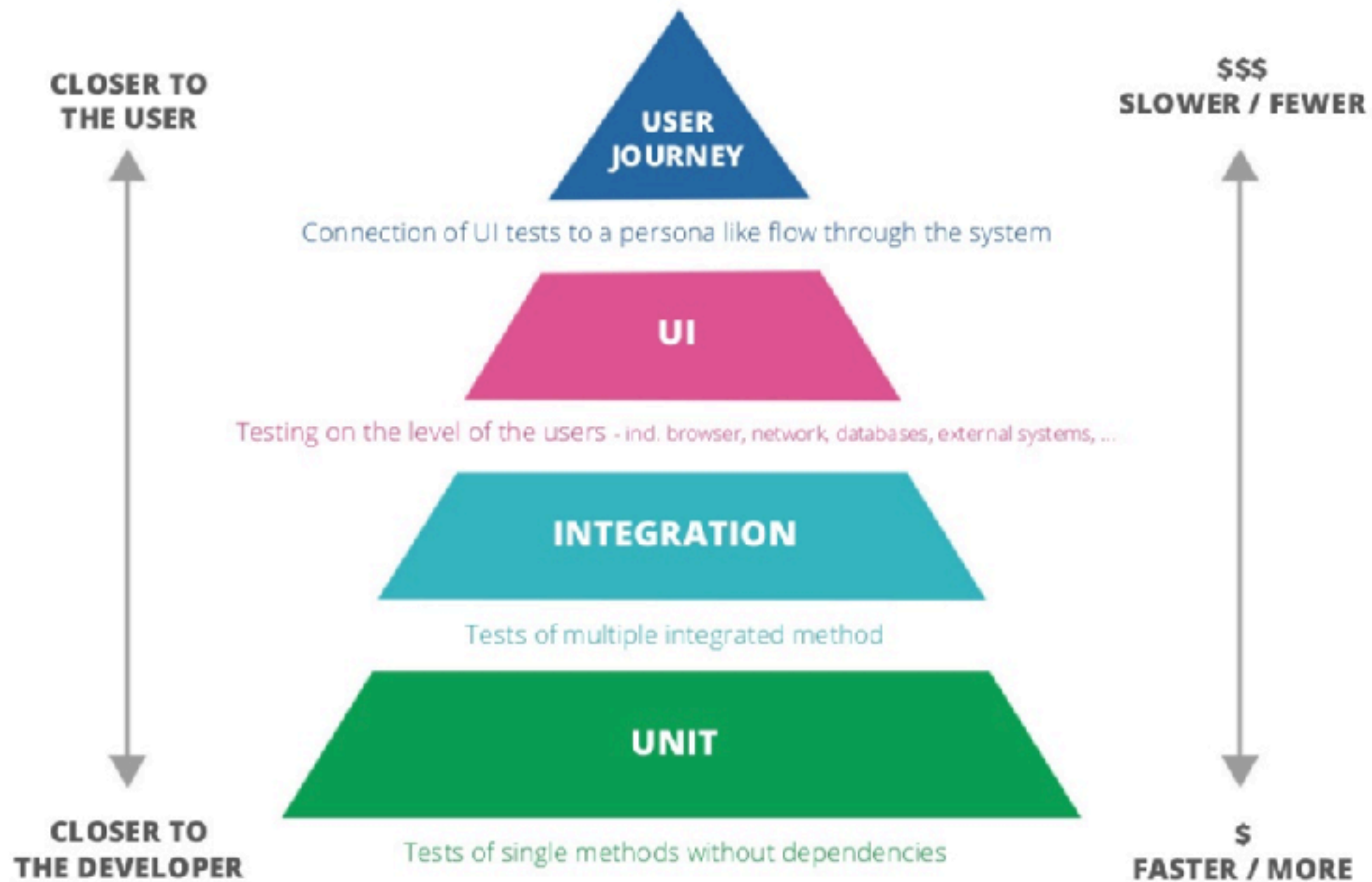
Gateway

Call external service via network such as
WebServices and RESTful APIs



Testing with Spring Boot





Spring Boot Testing

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-webmvc-test</artifactId>  
  <scope>test</scope>  
</dependency>
```



6



<https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing>



Testing in Spring Boot ?

@SpringBootTest

@WebMvcTest

@JsonTest

@DataJpaTest

@RestClientTest

<https://docs.spring.io/spring-boot/reference/testing/spring-boot-applications.html>



Testing in Spring Boot

@SpringBootTest

@WebMvcTest

@JsonTest

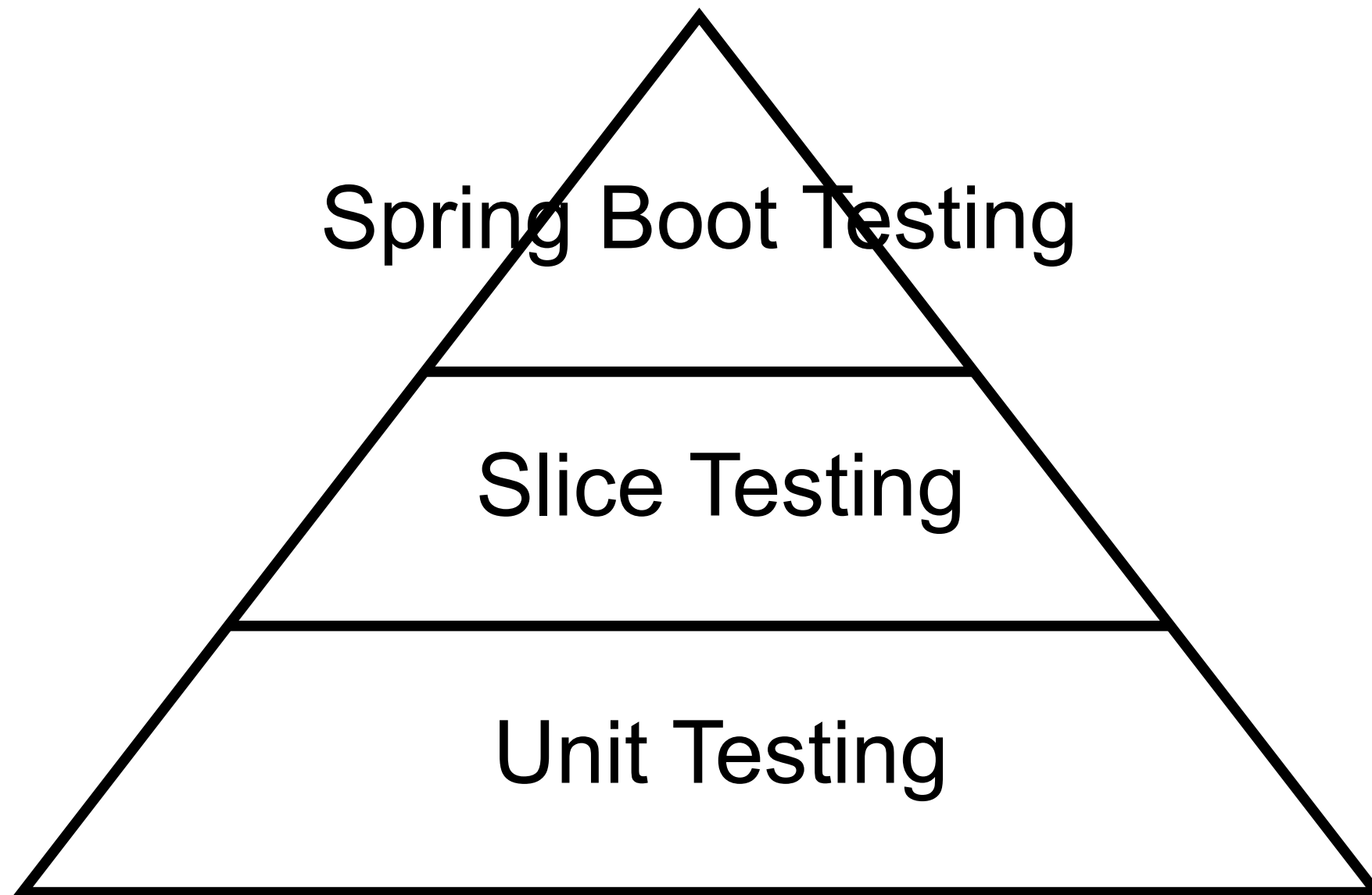
@DataJpaTest

@RestClientTest

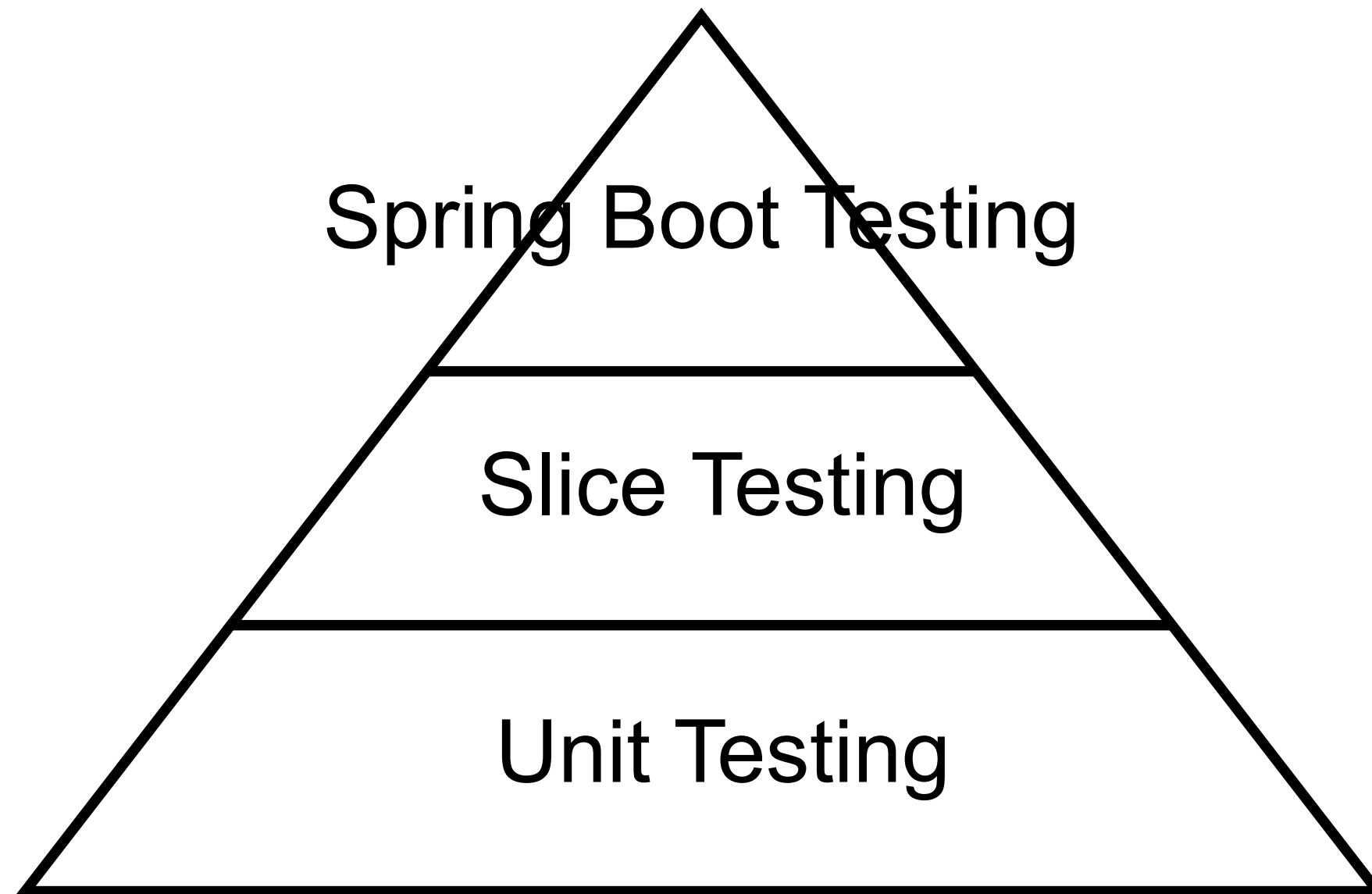
Slice testing



Testing in Spring Boot



Testing in Spring Boot



Controller testing

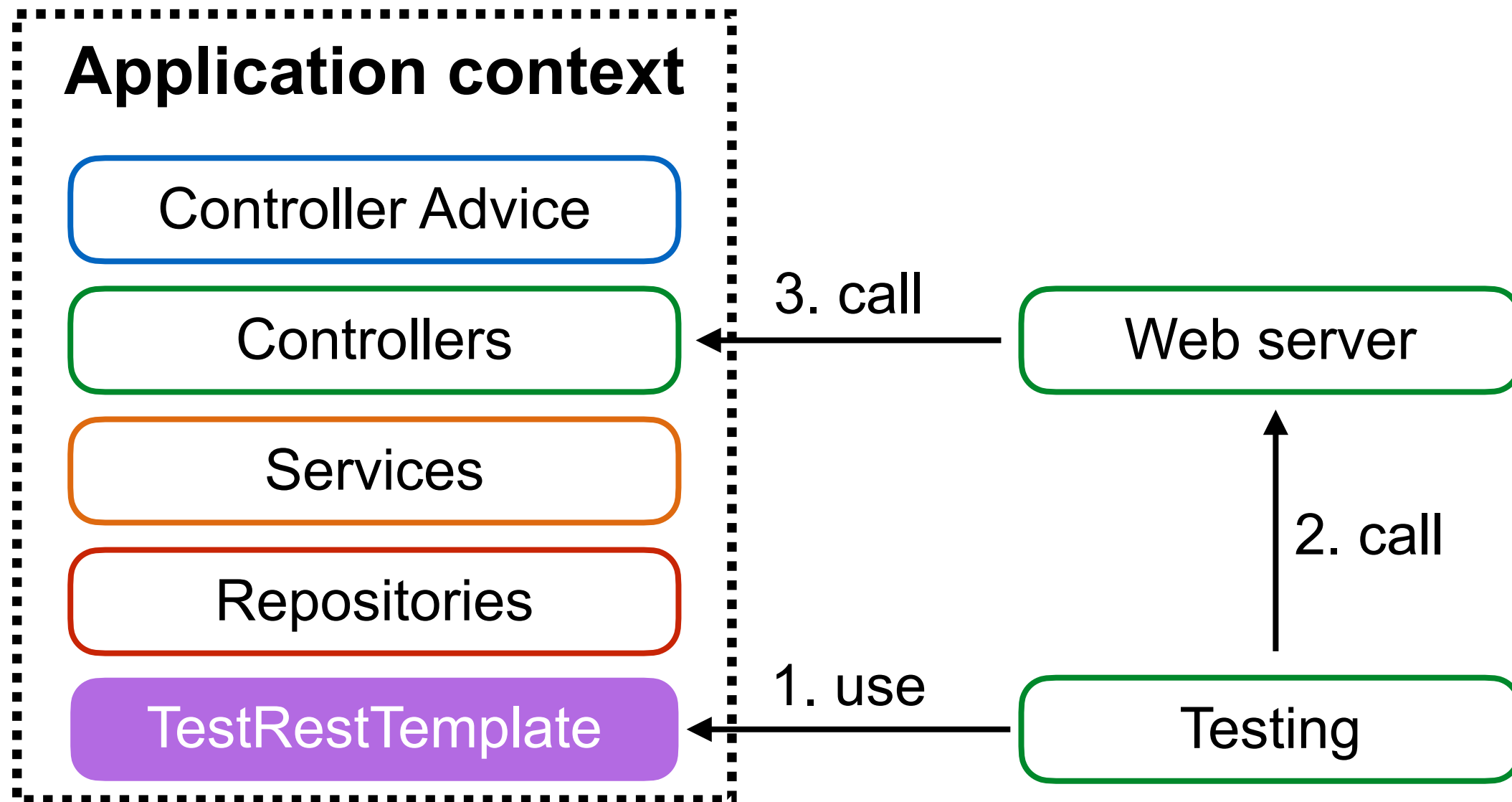
1. Spring Boot Testing
2. Slice Testing with MockMvc
3. Unit Testing



SpringBootTest



SpringBootTest



Example

```
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@AutoConfigureTestRestTemplate
class HelloControllerTest {

    @Autowired
    private TestRestTemplate testRestTemplate;

    @Test
    void case01() {
        // Act
        String input = "java";
        String actualResult
            = testRestTemplate.getForObject("/api/hello/" + input,
                                           String.class);

        // Assert
        assertEquals("Hello java", actualResult);
    }
}
```



Error handling



Working with Error Responses

ErrorResponse

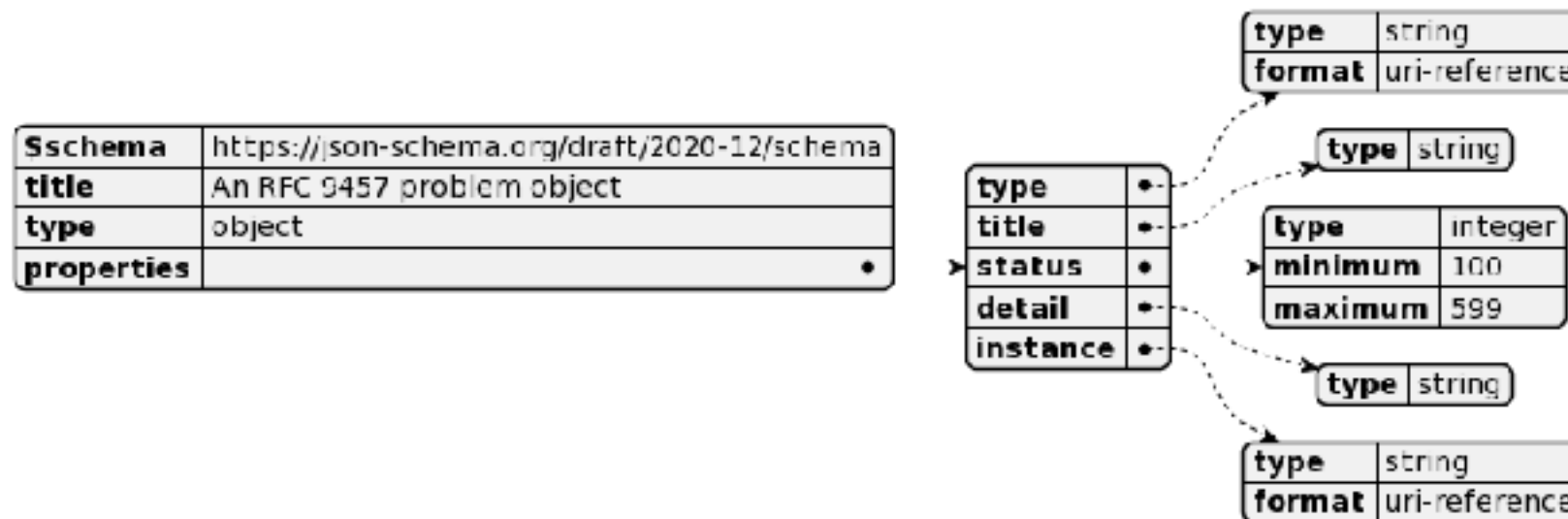
ProblemDetail

Customization

<https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-ann-rest-exceptions.html>



ProblemDetail



```
{
  "type": "https://problems-registry.smartbear.com/missing-body-property",
  "status": 400,
  "title": "Missing body property",
  "detail": "The request is missing an expected body property.",
  "code": "400-09",
  "instance": "/logs/regisrations/d24b2953-ce05-488e-bf31-67de50d3d085",
  "errors": [
    {
      "detail": "The body property {name} is required",
      "pointer": "/name"
    }
  ]
}
```

<https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/http/ProblemDetail.html>



ProblemDetail

Enable ProblemDetail for error by default

application.properties

```
spring.mvc.problemdetails.enabled=true
```



Example in ControllerAdvice

```
@RestControllerAdvice
class HelloControllerAdvice {

    @ExceptionHandler(InvalidInputException.class)
    public ProblemDetail handleInvalidInputException(
        InvalidInputException e, WebRequest request) {

        ProblemDetail problemDetail
            = ProblemDetail
                .forStatusAndDetail(HttpStatus.BAD_REQUEST, e.getMessage());
        problemDetail.setInstance(URI.create("demo-instance"));
        problemDetail.setTitle("demo");

        return problemDetail;
    }
}
```



Error handling

```
@Service
public class UserService {

    private AccountRepository accountRepository;

    @Autowired
    public UserService(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;
    }

    public Account getAccount(int id) {
        Optional<Account> account = accountRepository.findById(id);
        if(account.isPresent()) {
            return account.get();
        }
        throw new MyAccountNotFoundException(
            String.format("Account id=[%d] not found", id));
    }
}
```



MyAccountNotFoundException

```
public class MyAccountNotFoundException  
    extends RuntimeException {
```

```
    public MyAccountNotFoundException(String message) {  
        super(message);  
    }
```

```
}
```



Response Status

404 = Not Found

Status	Description
400	Request body doesn't meet API spec
401	Authentication/Authorization fail
403	User can't perform the operation
404	Resource does not exist
405	Unsupported operation
500	Error on server



Handling error in Spring Boot

```
@RestControllerAdvice  
public class AccountControllerHandler {
```

1

```
    @ExceptionHandler(MyAccountNotFoundException.class)  
    public ResponseEntity<ExceptionResponse> accountNotFound(  
        MyAccountNotFoundException exception) {  
  
        ExceptionResponse response =  
            new ExceptionResponse(exception.getMessage(),  
                                   "More detail");  
  
        return new ResponseEntity<ExceptionResponse>(response,  
                                                       HttpStatus.NOT_FOUND);  
    }  
}
```



Handling error in Spring Boot

```
@RestControllerAdvice
public class AccountControllerHandler {

    @ExceptionHandler(MyAccountNotFoundException.class)
    public ResponseEntity<ExceptionResponse> accountNotFound(
        MyAccountNotFoundException exception) {

        ExceptionResponse response =
            new ExceptionResponse(exception.getMessage(),
                                "More detail");

        return new ResponseEntity<ExceptionResponse>(response,
                                                    HttpStatus.NOT_FOUND);
    }
}
```

2



ExceptionResponse

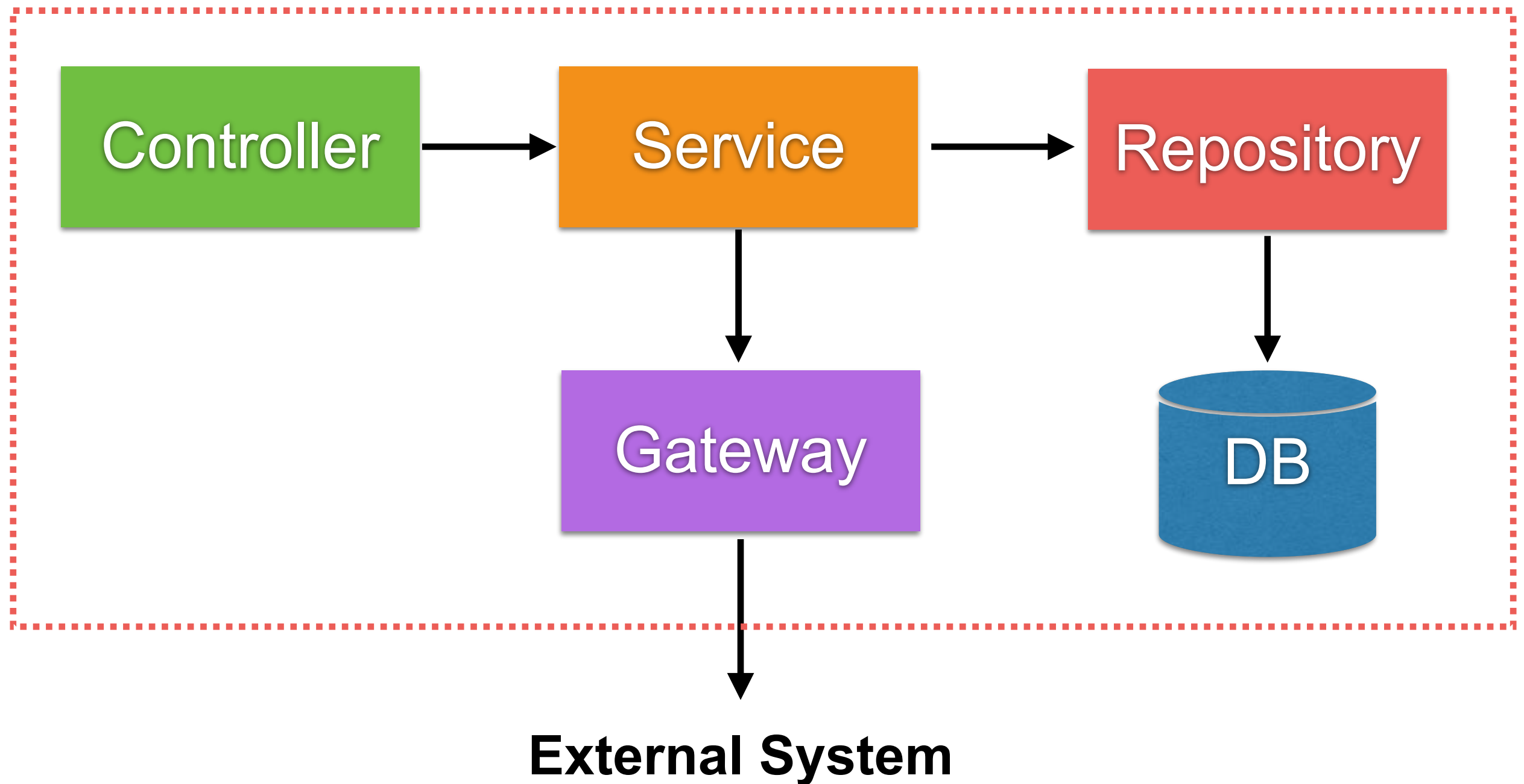
Response format of error

```
public class ExceptionResponse{  
  
    private Date timestamp = new Date();  
    private String message;  
    private String detail;  
  
    public ExceptionResponse(String message, String detail) {  
        this.message = message;  
        this.detail = detail;  
    }  
}
```



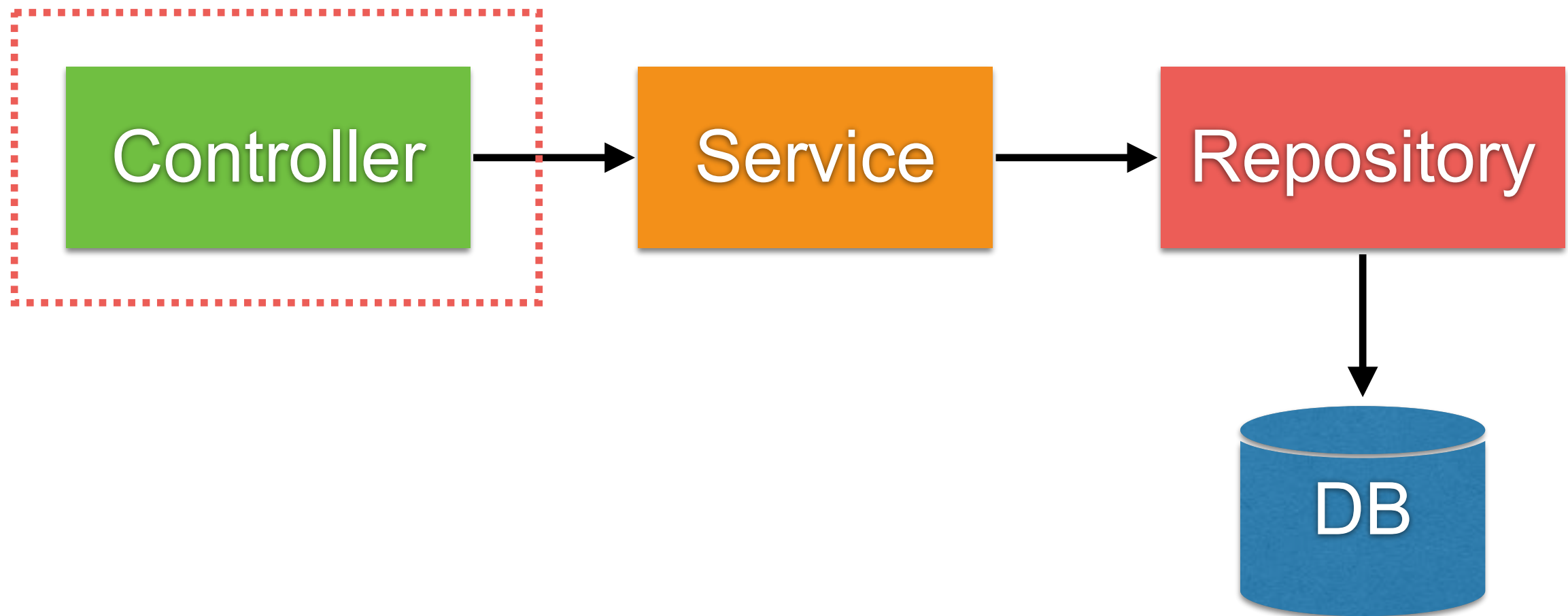
Scope of SpringBootTest

Loaded all classes as same as real system !!



How SpringBootTest ?

Only controller



Example code

```
@RestController
public class HelloController {
    @Autowired
    private HelloService service;

    @GetMapping("/api/hello/{name}")
    public String sayHi(@PathVariable String name){
        String result = service.process(name);
        return "Hello " + result;
    }
}
```

use

```
@Service
public class HelloService {
    public String process(String input){
        throw new RuntimeException("Underconstruction!!");
    }
}
```



SpringBootTest with Mockito

```
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@AutoConfigureTestRestTemplate
class HelloControllerTest {

    @Autowired
    private TestRestTemplate testRestTemplate;

    @MockitoBean
    private HelloService helloService;

    @Test
    void case01() {
        // Arrange
        when(helloService.process(anyString())).thenReturn("java");
        // Act
        String input = "java";
        String actualResult
            = testRestTemplate.getForObject("/api/hello/" + input,
                                           String.class);

        // Assert
        assertEquals("Hello java", actualResult);
    }
}
```



Slicing test with WebMvcTest

```
@WebMvcTest(HelloController.class)
class HelloControllerWebMvcTest {

    @Autowired
    private MockMvcTester mvcTester;

    @MockitoBean
    private HelloService helloService;

    @Test
    void case01() {
        // Arrange
        when(helloService.process(anyString())).thenReturn("java");
        // Act
        String input = "java";
        MockMvcRequestBuilder requestBuilder
            = mvcTester.get().uri("/api/hello/" + input);

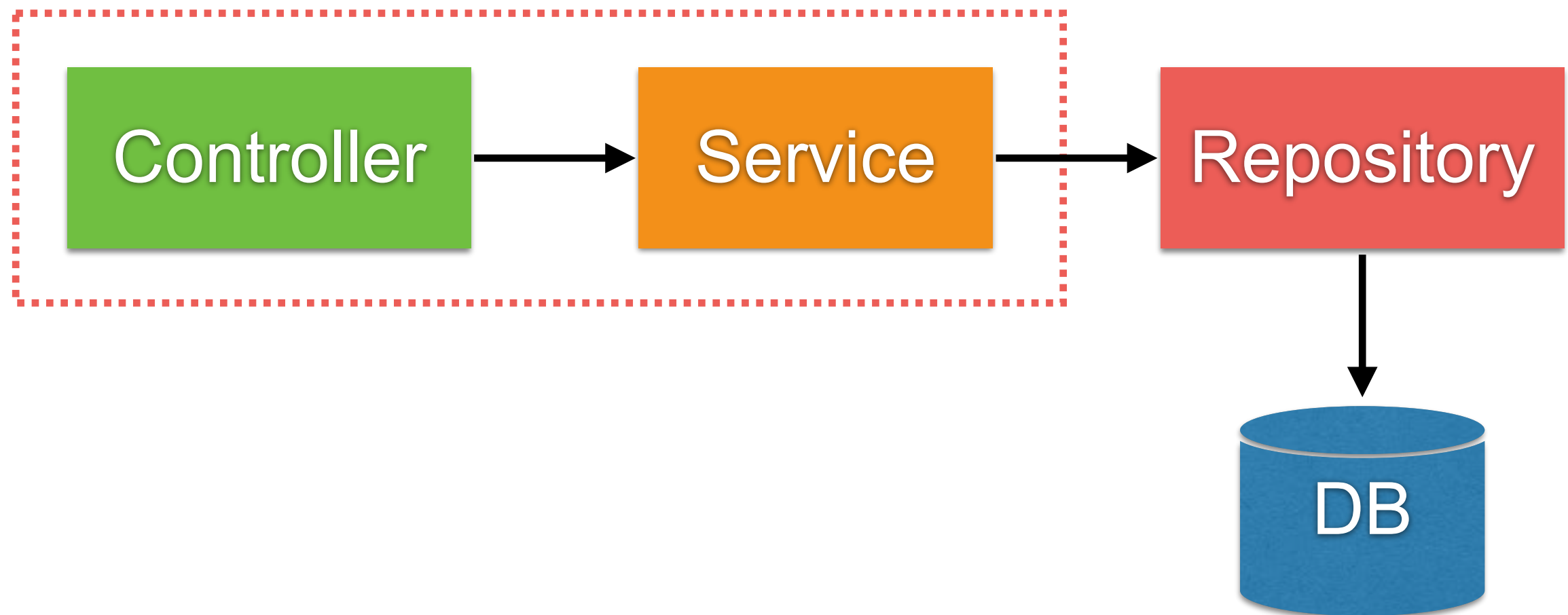
        // Assert
        requestBuilder.assertThat().hasStatusOk();
        requestBuilder.assertThat().hasBodyTextEqualTo("Hello java");
    }
}
```

<https://docs.spring.io/spring-boot/reference/testing/spring-boot-applications.html#testing.spring-boot-applications.spring-mvc-tests>



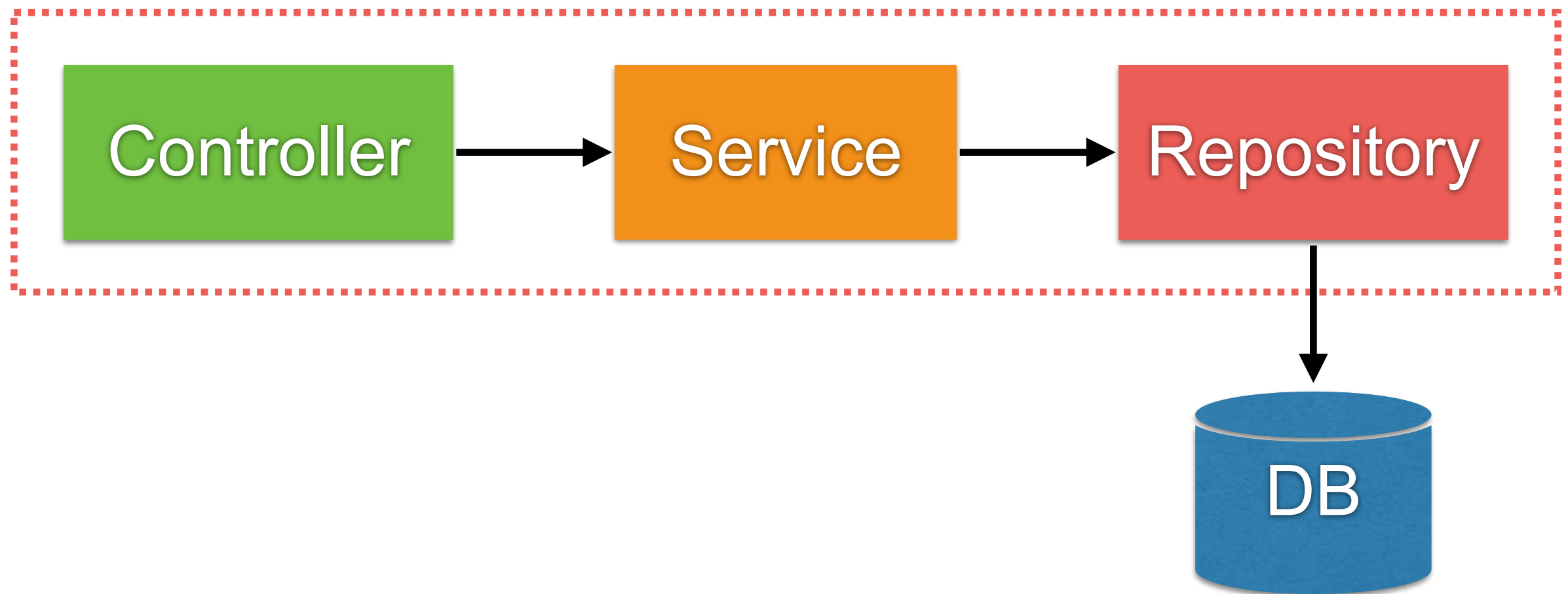
How SpringBootTest ?

Controller + Service



How SpringBootTest ?

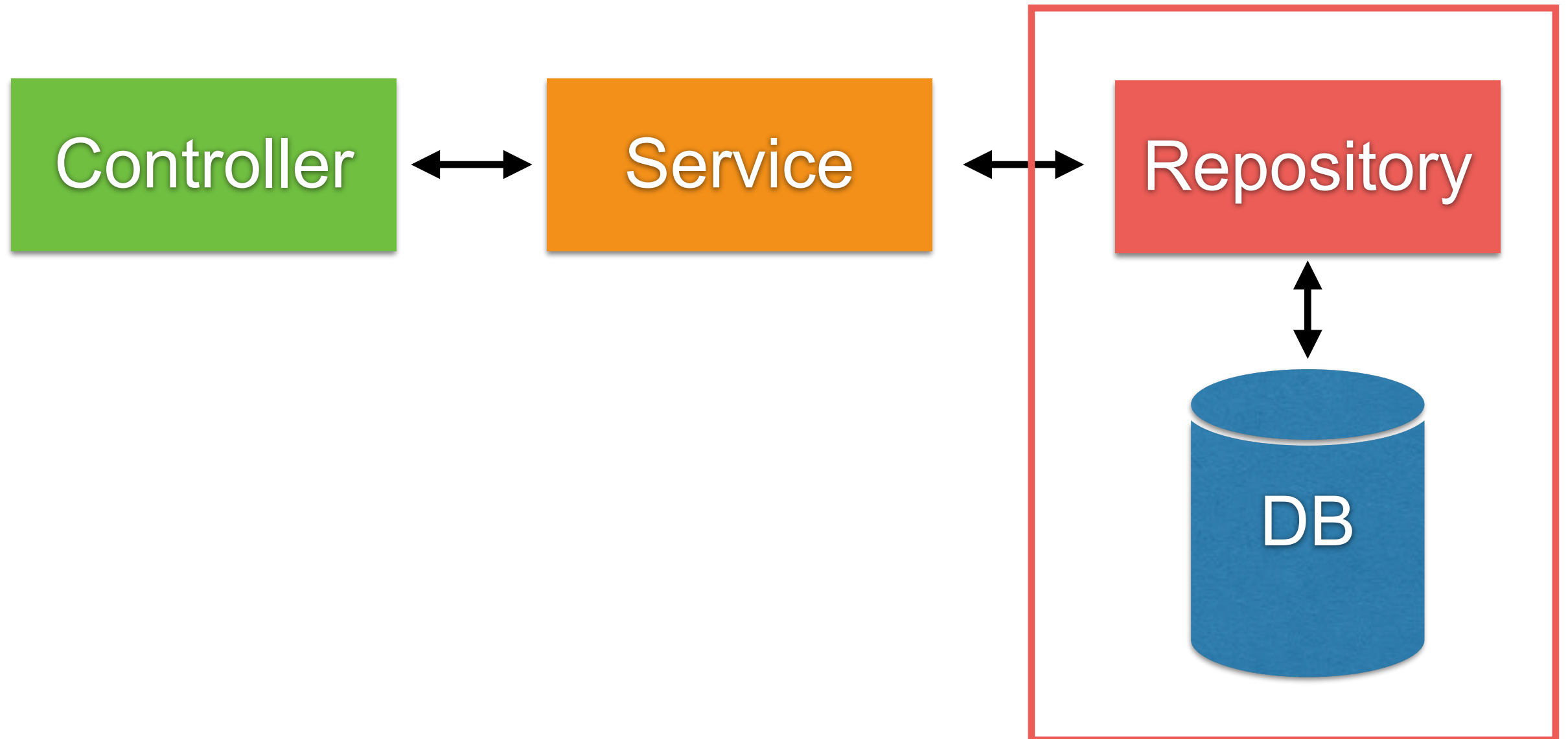
Controller + Service + Repository



Working with Database



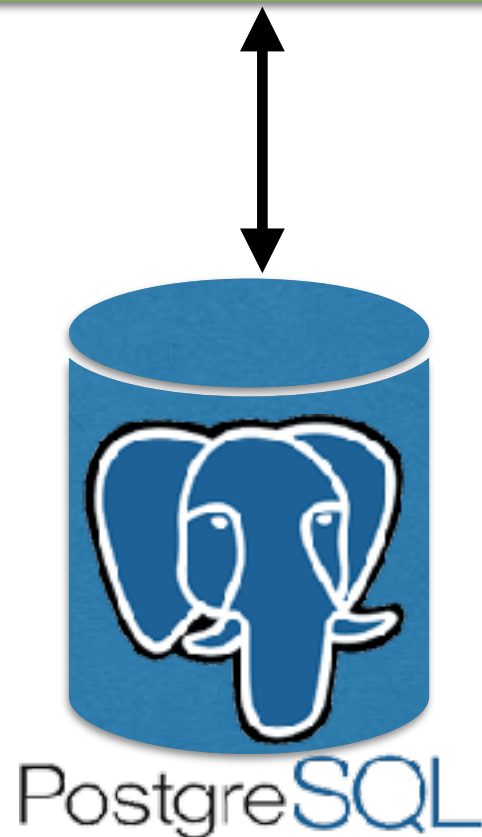
Repository Layer in Spring



Working with Database ?

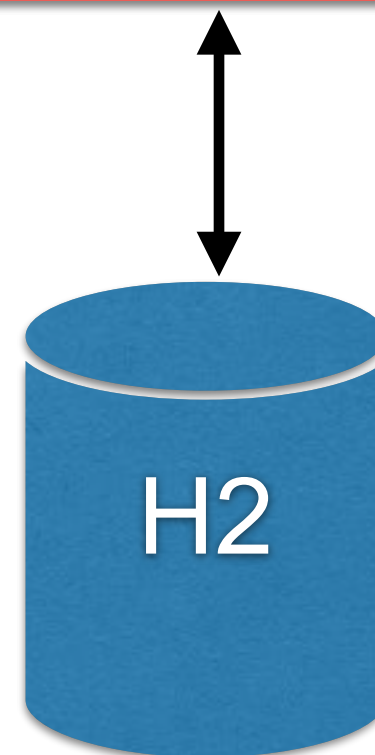
Production

Application



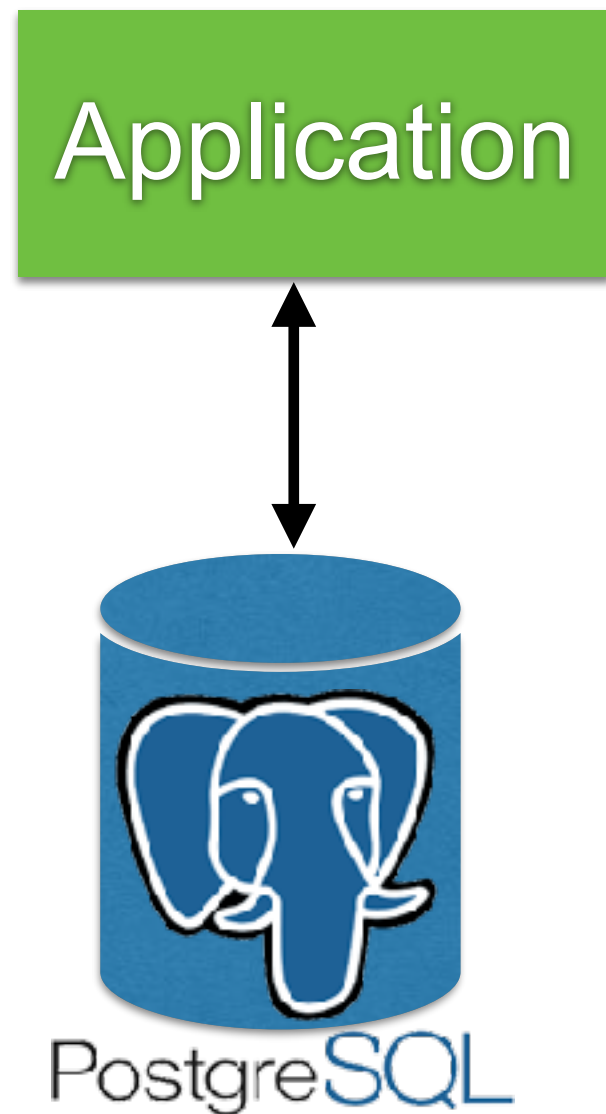
Testing

Application

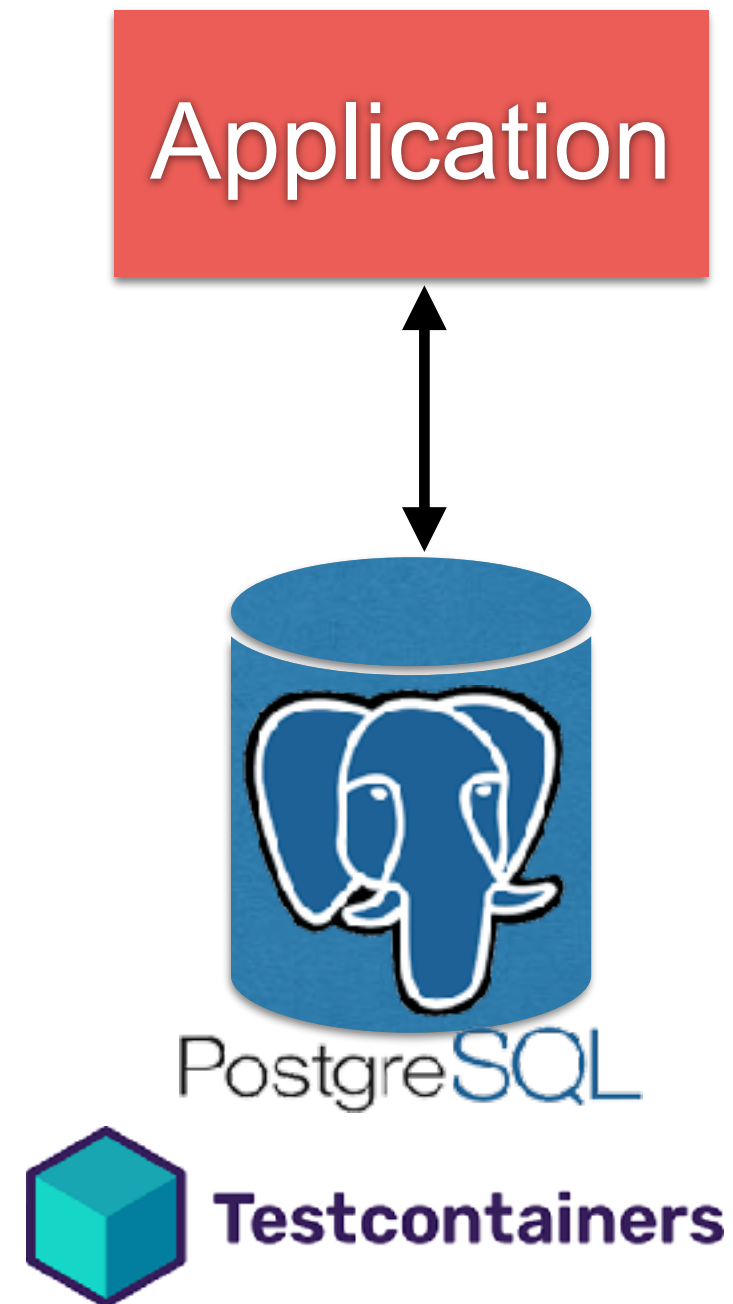


Working with Database ?

Production



Testing



Working with repository

We're using Spring Data

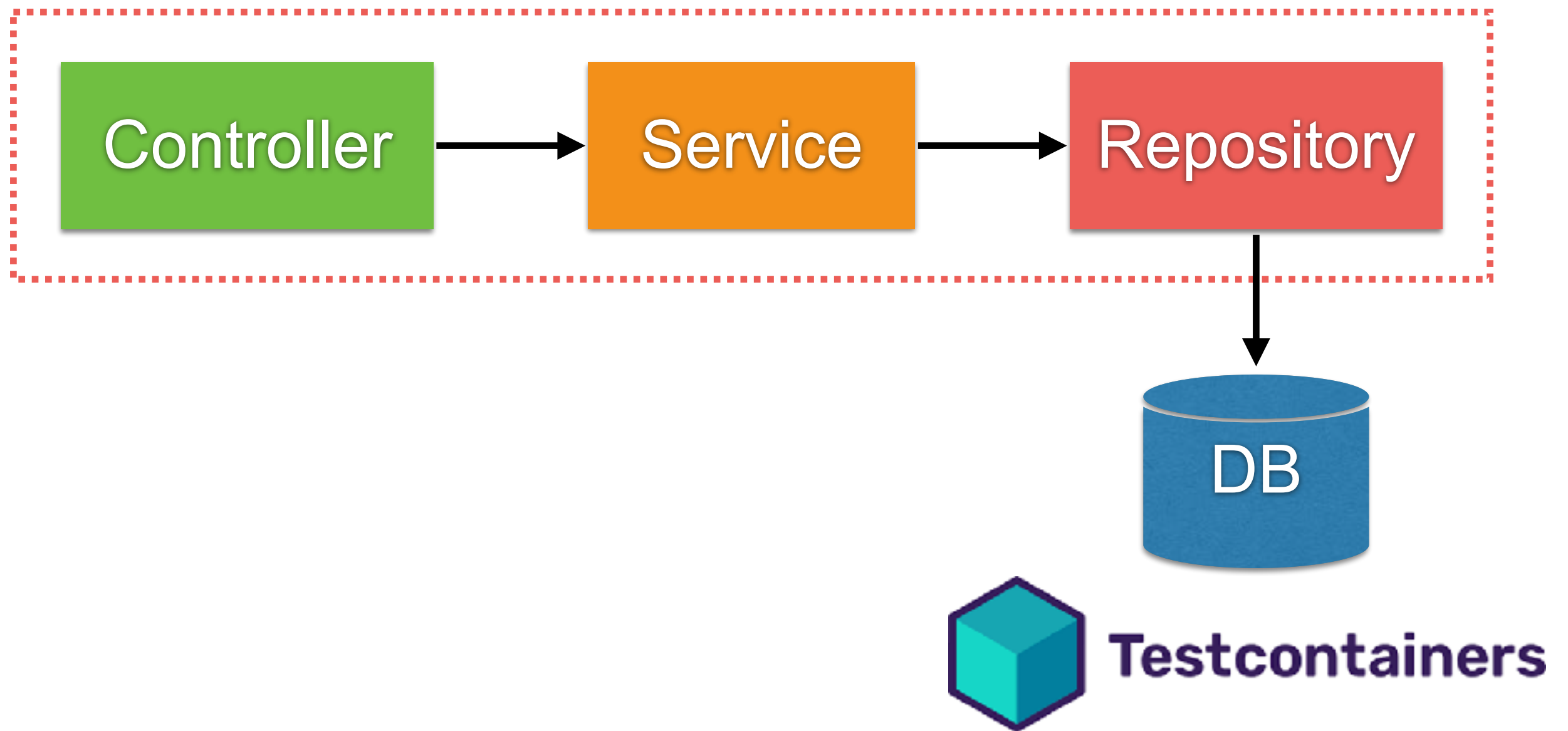


<https://spring.io/projects/spring-data>



How SpringBootTest ?

Controller + Service + Repository



Working with TestContainers



Project

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ **Java** ☐ Kotlin ☐ Groovy

☒ **Maven**

Spring Boot

☐ 4.1.0 (SNAPSHOT) ☐ 4.1.0 (RC1) ☐ 4.0.7 (SNAPSHOT) ☒ **4.0.6**

☐ 3.5.15 (SNAPSHOT) ☐ 3.5.14

Project Metadata

Group

Artifact

Package name

Packaging ☒ **Jar** ☐ War

Configuration ☒ **Properties** ☐ YAML

Java ☐ 26 ☒ **25** ☐ 21 ☐ 17

Dependencies ADD DEPENDENCIES... +8

Spring Web **WEB**
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA **SQL**
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

Testcontainers **TESTING**
Provide lightweight, throwaway instances of common databases, Selenium web browsers, or anything else that can run in a Docker container.

PostgreSQL Driver **SQL**
A JDBC and R2DBC driver that allows Java programs to connect to a PostgreSQL database using standard, database independent Java code.

<https://start.spring.io/>



SpringBootTest with TestContainers

```
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@Testcontainers
class HelloIntegrationTest {

    @Container
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres");

    @Autowired
    private HelloService helloService;

    @Autowired
    private HelloRepository helloRepository;

    @Test
    void case01() {

    }

}
```

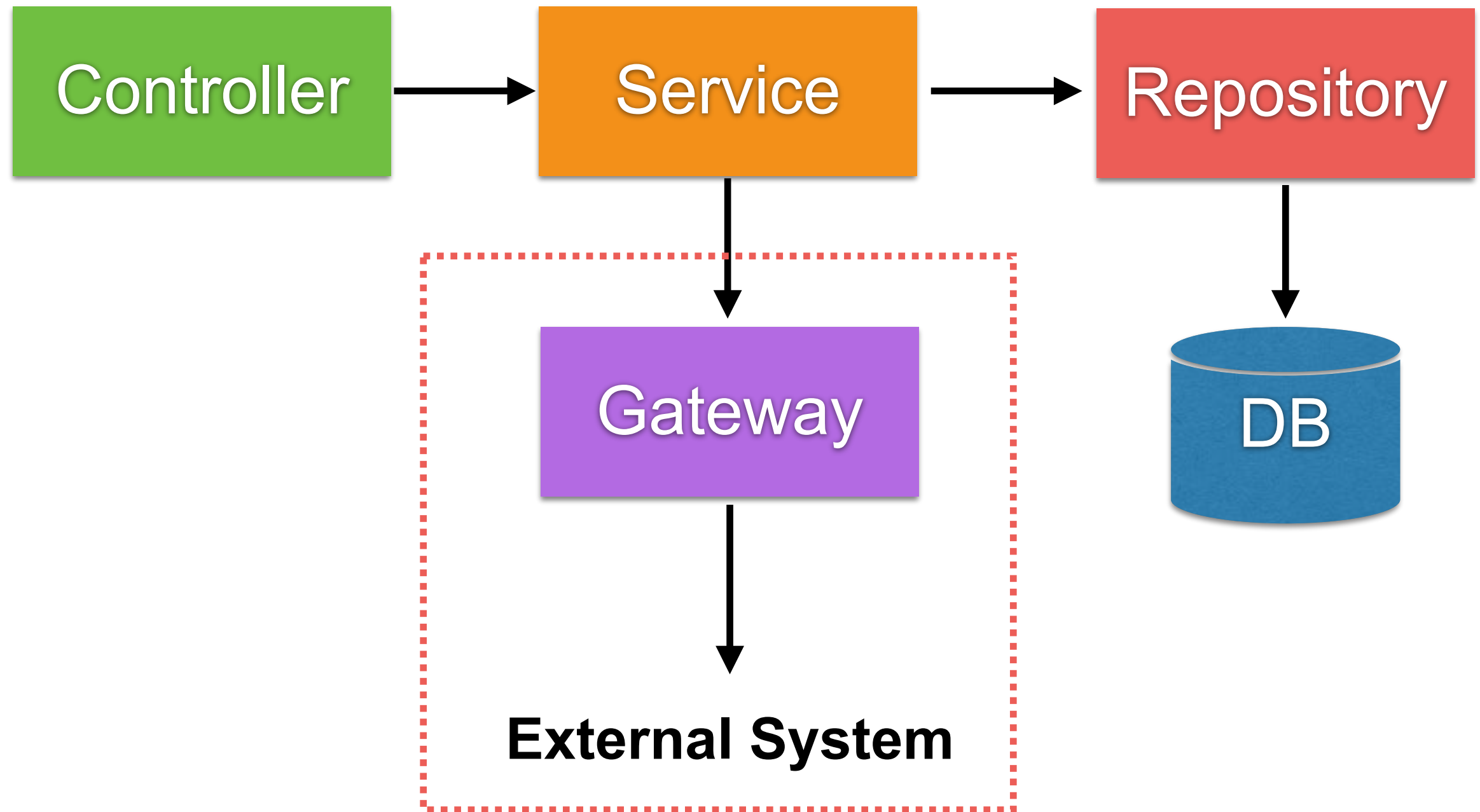
<https://docs.spring.io/spring-boot/reference/testing/testcontainers.html>



Working with External System via HTTP



How to testing ?









Q/A

